

fcbb:bb00

0002

0004

0006

e004

0000

0000

SRv6 深度學習

一個 packet、一張 topology、五個問題

Quintus Zhu

CCDE #20240069 · CCIE #15189 (R&S, SP, Wireless)

Cisco IOS XR 26.2.2 · uSID f3216 · 實驗平台：CML 上的 XRd vRouter

為什麼做這些實驗，它們有什麼特點

為什麼做這些實驗

- SRv6 的指令藏在 128-bit 的位址裡，光看設定看不到
- 成功的 commit 與 ping 不代表 data plane 正常
- 目標：看到一個 packet，就能說出誰在執行哪個指令
- 每個說法都要有實驗網路的證據

這些實驗的特點

- 依「packet 必須回答的五個問題」排列
- 每個結論都來自 capture 或 router 輸出；引用文件與實測分開標示
- 同一個 lab 從第一天用到畢業
- 每一步都是 Predict → Observe → Explain

1 張 topology

5 個問題

12 個 module

24 集

核心觀念：Destination Address 是一段程式



擁有 active instruction 的 router 執行它，把程式往左移 16 bits，再依新的 DA (Destination Address，目的位址) 轉送。

- 其他 router 只是 transit：對 DA 做 IPv6 longest-match，不看、不改這段程式。
- 所以學 SRv6 只需要一直問：現在的 active instruction 是哪一格？誰擁有它？

每個 packet 都要回答的五個問題

- | | | | |
|---|---------|----------------------|---|
| 1 | 我要去哪裡？ | Locator reachability | IS-IS 把 /48 locator 送進每台 router 的 RIB 與 CEF |
| 2 | 到了要做什麼？ | Local SID 與 behavior | uN、uA、uDT、uDX2：本地 SID table 決定動作 |
| 3 | 我載著誰？ | Service | BGP 把 tenant route 與 service SID 綁在一起 |
| 4 | 路斷了怎麼辦？ | Protection | TI-LFA 預先算好 repair list；microloop avoidance |
| 5 | 誰決定這條路？ | Intent | SR Policy、ODN、Flex-Algo 表達路徑意圖 |

學習方法：Predict → Observe → Explain

1 Predict

動手前寫下預測：哪個欄位會先改變？什麼證據可以推翻它？

2 Observe

只改一件事。讀 show 輸出，在 CML link 上抓 packet。

3 Explain

預測與觀察不同時，先解釋差異，再決定要不要改 config。

每個結論都帶一個證據標示

Standard

RFC 定義的行為

Lab

本實驗網路的輸出或 capture

Cisco guide

我們測不到時，引用 Cisco 文件

Design choice

為了可重現而做的選擇

Not verified

沒有文件，也沒有實驗證據

實驗地圖：5 個部分，12 個 module

01

The packet

我要去哪裡？到了做什麼？

- M1 SID 解剖
- M2 uSID shift
- M3 Encapsulation 與 MTU

02

Reachability

Locator 怎麼被全網知道

- M4 Bring-up 與驗收層級
- M5 IS-IS : LSP → RIB → CEF

03

Services

我載著誰？

- M6 L3VPN 與 resolution chain
- M7 EVPN : unicast 與 BUM

04

Resilience 與 intent

路斷了怎麼辦？誰決定路徑？

- M8 TI-LFA
- M9 SR Policy 與 ODN
- M10 Flex-Algo

05

Operate

把它維運起來

- M11 五層故障排除
- M12 Migration、邊界、畢業

實驗地圖：24 集，6 個單元

1 Ep 1–3

Foundations

- 1 狀態在哪裡
- 2 建立實驗網路
- 3 SID 與兩次查表

2 Ep 4–7

Data plane

- 4 Endpoint behavior
- 5 uSID shift
- 6 SRH 與封裝大小
- 7 MTU 與 ICMP

3 Ep 8–11

Underlay

- 8 IS-IS 到 CEF
- 9 TI-LFA
- 10 Microloop 與 BFD
- 11 Flex-Algo

4 Ep 12–16

Services

- 12 L3VPN
- 13 解析鏈
- 14 EVPN
- 15 SR Policy 與 BSID
- 16 ODN 需要 PCE

5 Ep 17–19

Operations

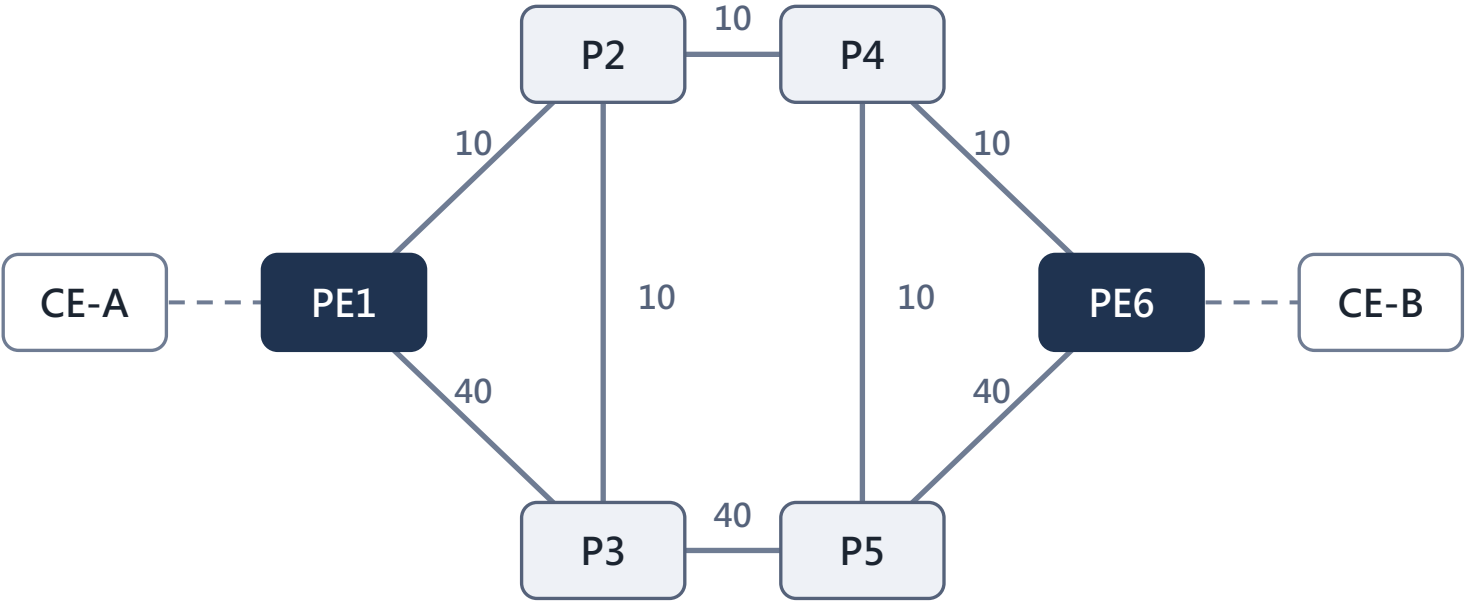
- 17 方法與 OAM
- 18 四個故障
- 19 安全邊界與規模

6 Ep 20–24

Migration and capstone

- 20 LDP 到 SRv6
- 21 SR-MPLS 到 SRv6
- 22 OSPF 到 IS-IS
- 23 硬體上的差異
- 24 畢業實作

一張 topology 用到底



上側 PE1–P2–P4–PE6 : Cost 30 · 唯一的最短路徑

下側每段 40 ; P2–P3 與 P4–P5 橫向各 10

項目	規劃
Loopback0	2001:db8:100::n/128
MAIN locator	fcbb:bb00:n::/48
FA128 locator	fcbb:bb01:n::/48
Core link	2001:db8:200:ab::/127
IS-IS	Level-2 · point-to-point
BLUE	RT 65000:100
RED	RT 65000:200
軟體	XRd vRouter · IOS XR 26.2.2

n 是節點編號 1–6。BLUE 與 RED 在 PE6 上刻意使用同一個位址 10.99.0.6，用來驗證隔離。GREEN 是第三個客戶。



第一部分：The packet

我要去哪裡？到了要做什麼？答案都寫在 128 bits 裡。

M1 · 狀態放在邊緣，不在核心

路由器	LDP + MPLS VPN	RSVP-TE (只有設計，沒建置)	SRv6
邊緣 PE1	VPN route 與 label ; 每個遠端 PE 一個 LDP label	Headend : 路徑與 signaling	客戶 route、service SID、封裝指令
核心 P2	每個 prefix 一筆 LDP binding ; 每個鄰居一條 session	每條經過的 tunnel 一筆 state，並持續 refresh	Locator 的 IPv6 route 與自己的 SID。沒有任何客戶資料

14 PE1 從兩個 LDP 鄰居各收到的 label 數

12 P2 的本地 SID 數 ; 客戶是 3 或 3000 都一樣

5 PE1 的 service SID : BLUE 2、RED 2、GREEN 1

消失的只有一種 state : 中間 router 上「每條路徑、每個客戶」的 state。IGP、本地 SID 與保護 state 都還在。

M1 · 同一個 100-byte ping，三種封裝

LDP

- 兩層 label：外層 24009 是 P2 對 PE6 的 label，內層 24000 是 PE6 的 VPN label
- 8 bytes

SR-MPLS

- 外層 label 16006 是 PE6 的 prefix SID
- 每一跳值相同，因為各 router 由 IGP 算出

SRv6

- 沒有 label。外層 IPv6 header 的 DA 是 PE6 的 service SID
- 40 bytes；frame 比 LDP 大 32 bytes

- 40 bytes 對 8 bytes 的差距一直存在。Ep 7 會示範它對大 packet 的影響。
- 同一份封裝有兩個任務：把 packet 送到 PE6，並告訴 PE6 這是哪個客戶。

M1 · SID 解剖：f3216



Locator = Block + Node

可路由的 /48，由 IS-IS 通告，回答「我要去哪裡」。PE6 的 locator metric 是 31：路徑 cost 30 加 1。

fcbb:bb00:6::/48

Function

本地 SID table 的索引，回答「到了做什麼」。本地 function 從 0xe000 開始，uA、uDT、uDX2 使用它。

e000、e001: 每台 router 的前兩個 uA

Argument

程式剩下的部分。uN 通告 Function 0、Argument 80：這 80 bits 放後面的 uSID。

IS-IS: uN 32/16/0/80 · uA 32/16/16/64

M1 · SID table 與 CEF 裡的 uN 和 uA

	SID (P3)	擁有者	CEF	Next hop
uN	fcbb:bb00:3::	SID manager	/48 · Endpoint uN, shift	下一跳不固定：shift 後再查一次
uA	fcbb:bb00:3:e005::	IS-IS	/64 · Block + Node + Function	固定：Gi0/0/0/1，往 P5

Protected 與 unprotected

- 同一個介面有兩個 uA：一個有 P 標記 (protected)，一個沒有
- Router 能繞過故障修復 protected 的 adjacency；只想走這條 link 時，用 unprotected 的

數量對得起來

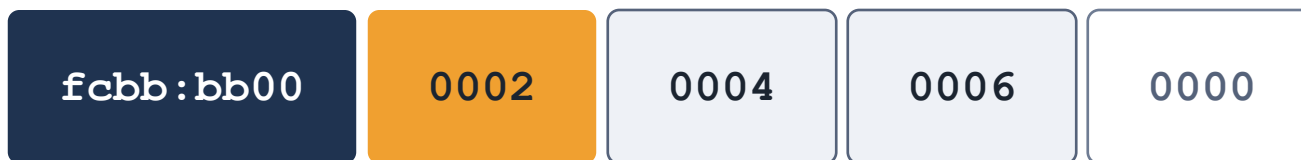
- P3 有 3 條 link × 2 個 algorithm × 2 (protected 與否) = 12 個 uA
- 拿 show segment-routing srv6 sid 和 show isis database verbose 對照

Transit 只要 route

- P2、P4 把 SID 當成一般 IPv6 位址轉送
- PE1 看 PE6：loopback metric 30，locator metric 31

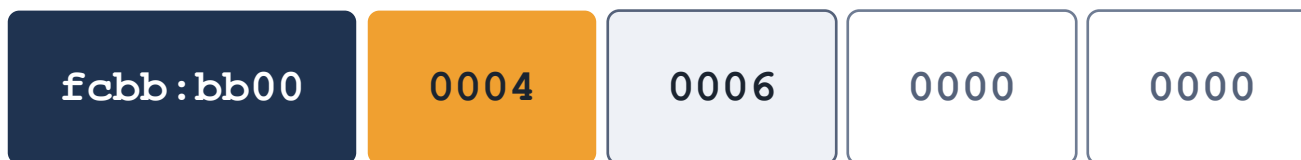
M2 · uSID shift : 三次觀察

PE1 送出



Hop limit 60 · P2 擁有 active uN

P2 處理後



Hop limit 59 · shift 16 bits · 低位補零

P4 處理後



Hop limit 58 · PE6 的 uN ; Argument 為零

Predict : 從 PE1 執行 ping fcbb:bb00:2:4:6::, 三條 link 上各是什麼 DA ?

Lab : DA 與 hop limit 和上表一致, 其他 link 沒有 packet。這個 ping 沒有 echo reply (見 M2 capture 規則)。

M2 · Waypoint、container 容量與 capture 規則

Node SID 是 waypoint，路徑由 IGP 選

- P3、P5、PE6 的清單實際走 PE1–P2–P3–P2–P4–P5–P4–PE6
- Hop limit 60 到 54，下側三條 link 沒有 packet
- 要指定 link，用 uA

Container 容量

- 32-bit block 之後有六個 16-bit 位置
- Service SID 佔一個位置
- 同一個 container 只能有同一個 block：bb00 與 bb01 不能混用（見 M3）

Capture 看四個欄位

- DA：active 指令與剩下的指令
- Hop limit：幾台 router 處理過
- Next header：43 表示後面有 routing header
- Segments Left：只有在有 SRH 時出現

Shift 過的 ping 沒有 echo reply：ICMPv6 checksum 用原本的 DA 計算（RFC 9800 6.5 要求用最後的 DA）。客戶流量不受影響。測試時看 capture，不要看 ping 回應。

M2 · Behavior 對照表

IOS XR	RFC	動作	由誰分配
uN	End + NEXT-CSID (PSP/USD)	shift，往下一個 instruction 轉送	SID manager (來自 locator)
uA	End.X + NEXT-CSID	shift，從指定的 adjacency 送出	IS-IS
uDT4 / uDT6 / uDT46	End.DT4 / DT6 / DT46	解封裝，在 VRF table 查表	BGP
uDX2	End.DX2	解封裝，把 L2 frame 送往 AC	EVPN (VPWS)
uDT2U / uDT2M	End.DT2U / DT2M	解封裝，在 bridge-domain 查 MAC 或 flood (平台拒絕 bridge domain)	EVPN (E-LAN)
uB6	Insert 形式 (RFC 8986 只定義 Encaps)	把流量導入一條 SR Policy (Ep 15)	SR-TE

- PSP 移除 SRH、USD 移除外層 header，都是 uN 的 flavor；uDT 的解封裝是 service behavior 自己的動作。
- Behavior 不能從 config 名稱猜，要讀 show segment-routing srv6 sid。清單裡有 behavior，不代表平台接受使用這個 behavior 的功能。

M2 · Endpoint 的處理順序與 flavor

1 DA 裡還有下一個 micro-SID ？

Shift 16 bits · hop limit 減一，用新的 DA 轉送。

2 沒有，但 SRH 的 Segments Left 大於 0 ？

下一個 SID 從 SRH 複製到 DA。

3 兩者都沒有

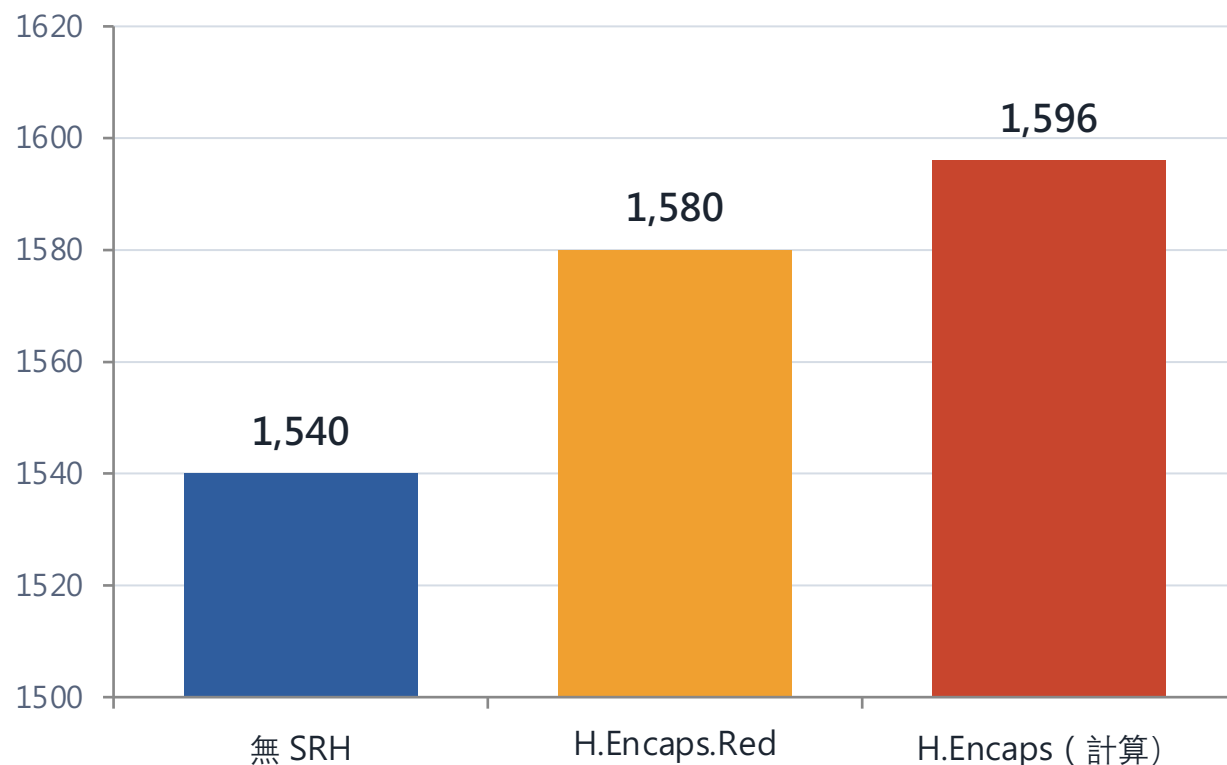
這是最後一條指令，處理 packet 的內容。

Flavor	移除什麼	證據
PSP	倒數第二個 endpoint 移除 SRH	Lab (Ep 6) : P5 之後 frame 從 182 變 158 bytes 。 Ep 4 較早版本的 capture : 178 變 154
USP	最後一個 endpoint 移除 SRH	RFC 8986
USD	最後一個 endpoint 移除整個外層 IPv6 header	Not verified : 沒有任何服務產生這種 packet

Flavor 只有在有 header 可移除時才動作。uDT4 的解封裝是它自己的功能，不是 USD。Segments Left 為 0 時，DA 仍可能含有指令。

M3 · Encapsulation 與 MTU : SRH 的代價

內層 1500 bytes 封裝後的外層長度



後兩者以 3 個 128-bit SID 為例：reduced 的 SRH 有 2 個 entry (40 bytes)，plain 有 3 個 (56 bytes)。

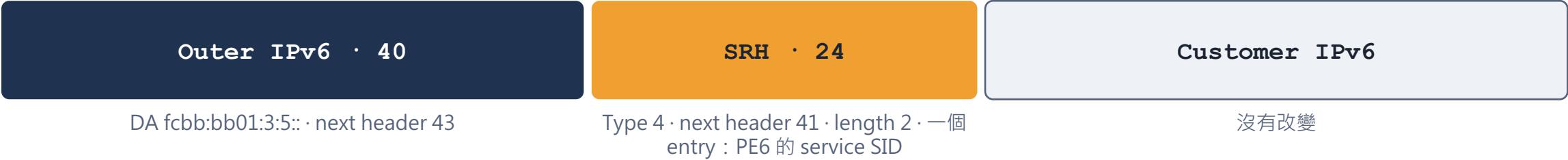
40

bytes：uSID 放得進單一 container 時，沒有 SRH，開銷只有外層 IPv6 header。

$$40 + 8 + 16 \times \text{entries}$$

- Lab：GREEN 的 frame 是 154 bytes，沒有 SRH。colour 300 的 frame 在 PE1-P3 是 182 bytes，多 64。
- P5 是倒數第二個 endpoint，PSP 移除 SRH，之後只多 40 bytes。
- 這個 headend 只有 reduced 形式，沒有 plain H.Encaps。1596 只是計算值，不是 capture。

M3 · colour 300 的 SRH (capture)



位置	Segments Left	發生什麼
PE1 → P3	1	DA 是 path container (P3 、 P5)
P3 之後	1	只有 uSID shift · SRH 不變
P5	0	Container 沒有 micro-SID : SL 減一 · service SID 複製進 DA · PSP 移除 SRH

SRH = 8 + 16 × 1 = 24

為什麼需要 SRH

- 一般 SID : fcbb:bb00 ; Flex-Algo SID : fcbb:bb01
- 一個 container 只能有一個 block

兩個 container · 一個 entry

- Reduced 形式 : 第一個 SID 只放進 DA
- PE1 先封裝 service SID · 再插入路徑

M3 · MTU 預算：1460、1436、1446

1460

一個 container，沒有 SRH

$1500 - 40$

1436

colour 300，一個 SRH entry

$1500 - 64$

1446

E-Line (Ep 14)

$1500 - 40 - 14$

先確認工具的「size」是什麼

- IOS XR ping：size 是整個 IP packet
- Linux ping：size 只是 data，IPv4 加 28、IPv6 加 48
- 一律用 capture 確認

核心 MTU

- 實驗用 1500，是為了容易碰到極限；實際網路的核心 MTU 通常大很多
- 核心 9000、一個 SRH entry： $9000 - 40 - 24 = 8936$
- E-Line： $9000 - 40 - 14 = 8946$

M3 · 三種 MTU 失敗 (Ep 7)

案例	上限	發生什麼	客戶
IPv4、DF，在 ingress PE1 (RED)	1460	1461 在 PE1 被丟棄。任何 link 上都沒有 ICMP	Black hole，不會恢復
IPv6，在 ingress，colour 300 (BLUE)	1436	1437 得到 Packet Too Big，MTU 1436，來源是 PE1 loopback。PTB 先經過核心到 PE6，再回來	Host 分段，恢復
IPv4，在中間：P4 的 link 設為 1300 (RED)	1260	P4 丟棄 1261，沒有 VRF，所以把 ICMPv4 frag needed (MTU 1236，來源 10.0.0.4) 沿原外層 header 送到 PE1，再由 PE1 經 PE6 回給 CE-B	CE-B 分段，恢復

- 最糟的是 ingress 的 IPv4，不是多數人預期的案例。
- Layer 2 (Ep 14)：PE1 對外層 IPv6 packet 分段，PE6 丟棄分段，沒有任何訊息。
- 依最大的 overhead 計算核心 MTU，不要倚賴 ICMP。每個案例都記錄 ICMP type、回報的 MTU、來源與路徑。

M3 · 回報的 MTU：有 repair path 時減 64

目的地的 route 狀態	ICMPv6 PTB 回報	實際轉送使用	設定 1400 時
已安裝 TI-LFA repair path	出口 MTU – 64	真正的 MTU	1336
沒有 backup	準確的 MTU	真正的 MTU	1400
Plain LFA backup	準確的 MTU	真正的 MTU	1400

證據

- XRd vRouter 26.2.2 與 XRv9000 26.1.1 結果相同
- 在 P2 移除 TI-LFA 後，回報值從 1336 變 1400
- 設定 1300 時，回報 1236 (Ep 7 的 ICMPv4 也是)

推論 · 未證實

- $64 = 40 + 24$ ：可能是替 repair header 保留的空間
- Cisco 沒有文件說明
- 不要把 -64 當成 SRH 的設計規則

實務影響

- Host 少用 64 bytes，什麼都沒壞
- 每次比較 PTB 的值時，記錄 repair 類型
- 轉送一律用真正的 MTU

M3 · 一個 service 的四個條件

1 Ingress

Locator 可達

PE1 的 IPv6 RIB 有 PE6 的 /48

```
show route ipv6 <locator>
```

2 Egress

Egress 有 service SID

PE6 的 SID table 有這個 SID

```
show segment-routing srv6 sid
```

3 Egress 之後

Behavior 有執行

外層 header 不見了：138 → 98 bytes，差 40

```
capture
```

4 Egress

在客戶 table 查表成功

VRF 有往 CE 的 route

```
show route vrf <VRF> <prefix>
```

- Loopback ping 一個條件也沒有檢查；BGP 顯示 best 也沒有。
- 健康時 PE6 的 locator metric 是 31；BGP 收到的 label 是十六進位 e0080，也就是 SID function e008。
- 第三種失敗：PE6 已換 SID，PE1 還送舊的，約 25 秒內收到 ICMPv6 destination unreachable。

M3 · 兩次查表



故障 A：遠端 locator 不見了

- PE6 沒有把 locator 掛進 IS-IS
- Loopback ping 正常，BGP 仍顯示 best
- BGP：locator inaccessible；CEF：drop
- 分歧點：PE1 的 IPv6 RIB；線上沒有 packet

故障 B：出口 service SID 不見了

- PE6 的 BGP VRF 沒有分配 SRv6 SID
- PE1 收到十進位 MPLS label 24004，不是 SID
- 核心沒有 MPLS：CEF unresolved，drop
- 分歧點：PE6 的 local SID table



第二部分：Reachability

Locator 只在本機 Up 還不夠，全網都要學到並裝進 CEF。

M4 · Bring-up : 四個驗收層級

4 End-to-end packet
`capture: hop limit 60/59/58`

3 遠端可達
`show cef ipv6 <locator>`

2 Local SID 已安裝
`show segment-routing srv6 sid`

1 Config 被接受
`commit`

```
! PE1 baseline
interface Loopback0
  ipv6 address 2001:db8:100::1/128
segment-routing
  srv6
    encapsulation
      source-address 2001:db8:100::1
    locators
      locator MAIN
        micro-segment behavior unode psp-usd
        prefix fcbb:bb00:1::/48
router isis CORE
  is-type level-2-only
  net 49.0001.0000.0000.0001.00
  address-family ipv6 unicast
  metric-style wide
  segment-routing srv6
    locator MAIN
```

M4 · 設計前先讀 manager

啟用了什麼

- Micro-segment 模式，格式 f3216
- 本地 function 從 e000 開始
- Encapsulation source 是 Loopback0

缺少什麼

- Headend 只有 H.Insert.Red 與 H.Encaps.Red，沒有 plain H.Encaps
- Hop limit 不能 propagate；traffic class 可以，預設不開

上限

- Max-SL 3、Max-H-Insert 3、Max-H-Encap 3、Max-End-D 4
- 一個 SID 是完整的 container，所以 3 是 container 的上限
- 16 個 locator、64000 個 SID 是虛擬 router 的上限

- 這個平台能證明：syntax、control plane、SID 分配、packet 格式、轉送路徑。
- 它不能證明：line card 支援、收斂時間、硬體資源、規模。
- Behavior 清單不等於平台接受：清單有 uDT2U 與 uDT2M，bridge domain 仍被拒絕。

M5 · IS-IS : LSP → RIB → CEF



三個故障，各自從不同的一層開始

故障	Adjacency	LSP	SPF route	RIB	CEF
介面沒有啟用 IPv6 address-family	✗ 往 P2	✓	metric 71	metric 71	出口改變
Locator 沒有掛進 IS-IS	✓	✗	✗	✗	✗
本機 RIB 安裝被 filter	✓	✓	✓	✗	✗

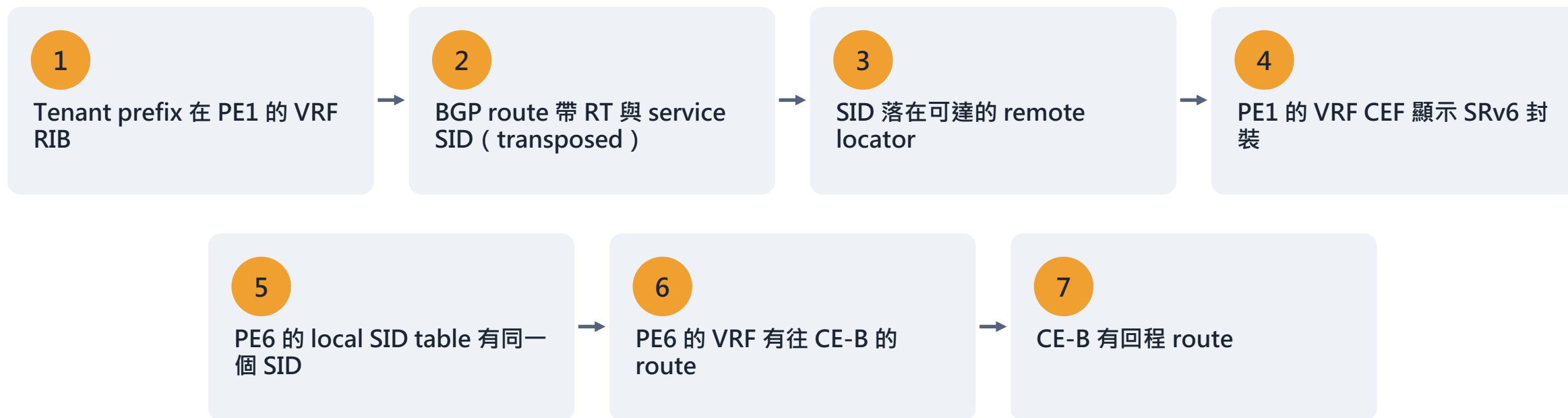
通告的 SID structure : uN 是 32/16/0/80 , uA 是 32/16/16/64 (不是 32/16/16/0) 。 MSD 是每個 packet 的 SID 深度 , Max SIDs 是每台 router 的上限。
Locator metric = 路徑 cost + 1 。



第三部分：Services

我載著誰？BGP 把 tenant route 與一個 service SID 綁在一起。

M6 · L3VPN : service resolution chain



BGP Established 一環也沒有證明

從 tenant prefix 出發，沿著鏈條逐環核對。第一個對不上地方就是 first divergence，停在那裡解釋，不要一次改很多處。

M6 · BGP update : transposition



Prefix-SID 屬性 : locator

屬性 : node

來自 NLRI label

NLRI

- Label 917568 = 0xE0040
- Label 欄位 20 bits : function e004 後面加 4 個 0 bits
- Next hop 是 PE6 的 loopback , 不是 SID

Prefix-SID 屬性

- Type 40 · SRv6 L3 Service TLV (type 5)
- SID 值只有 fcbb:bb00:6::
- Behavior End.DT4 NEXT-CSID , structure 32/16/16/0

重建

- Transposition length 16 , offset 48
- 接收端取 label 的前 16 bits , 放在 bit 48 之後
- 結果 : fcbb:bb00:6:e004::

	SID	Label	來源
BLUE IPv4	fcbb:bb00:6:e004::	917568	capture
BLUE IPv6	fcbb:bb00:6:e005::	917584	計算
RED IPv6	fcbb:bb01:6:e004::	917568	計算 : label 同 BLUE IPv4

M6 · 線上看到什麼

Hop limit

- PE1 把外層 hop limit 設為 255，不複製客戶的 TTL
- P2、P4 只減外層
- CE-A 的 traceroute 只看到 PE1、PE6、CE-B (pipe model, RFC 2983)

大小

- 客戶的 98-byte ping 在核心是 138 bytes
- 離開 PE6 後又是 98 bytes
- 客戶 packet 在 tunnel 中沒有改變

QoS 標記

- CE-A 標 EF (DSCP 46)
- 核心裡內層是 46，外層是 0
- Traffic class 預設不 propagate。核心若依外層排隊，要設定並用 capture 驗證

每一環失敗的樣子不同：RT 不符，update 到了但 VRF 不 import；locator 不可達，BGP 仍是 best 但 CEF 是 drop；PE1 正常但 CE-B 沒收到，看 PE6 的 SID。

M6 · 把隔離設計成可以被證明

BLUE

RT 65000:100

RD 65000:101 / 65000:106

Site prefix

10.1.1.0/24 · 10.1.6.0/24

重疊的位址 (PE6)

10.99.0.6/32

PE6 的 service SID

uDT4 e004 · uDT6 e005

RED

RT 65000:200

RD 65000:201 / 65000:206

Site prefix

10.2.1.0/24 · 10.2.6.0/24

重疊的位址 (PE6)

10.99.0.6/32

PE6 的 service SID

uDT4 e006 · uDT6 e004 (bb01)

1

正向測試

同 tenant 雙向 ping ; IPv4 與 IPv6 分開驗

2

負向測試

BLUE 來源 ping RED 的位址 (10.1.1.2 → 10.2.6.2) 沒有回應，而且 VRF BLUE 沒有那條 route。兩者都要檢查

3

沒有送錯 tenant

兩個 tenant 在 PE6 都有 10.99.0.6。抓 packet 比較外層 DA：BLUE 是 e004，RED 是 e006

RD 讓重疊的 prefix 在 BGP 裡保持唯一；RT 決定誰 import；service SID 決定 egress 用哪個 table。三者都要正確。

M6 · 一個 VRF 不等於一個 SID

SID (PE6)	Behavior	Context
fcbb:bb00:6:e004::	uDT4	BLUE
fcbb:bb00:6:e005::	uDT6	BLUE
fcbb:bb00:6:e006::	uDT4	RED
fcbb:bb01:6:e004::	uDT6	RED (FA128)
fcbb:bb00:6:e007::	uDX2	300:300 (L2 · 見 M7)
fcbb:bb00:6:e008::	uDT4	GREEN

另一個陷阱：這些 SID 的 function 值在 PE1 與 PE6 不同。

Per-VRF 分配

每個 VRF 與 address family 一個 SID。BLUE 有 uDT4 與 uDT6 兩個。

動態分配

Function 依分配順序從動態範圍取得。重建實驗後數值會變：讀它，不要抄它。

只建 VRF 不會有 SID

SID 要在 BGP 的 VRF 底下啟用 SRv6 才會分配。

M6 · RD：兩台 PE 用不同的值

VRF	PE1 的 RD	PE6 的 RD	Route target
BLUE	65000:101	65000:106	65000:100
RED	65000:201	65000:206	65000:200
GREEN	65000:301	65000:306	65000:300

```
PE1#show bgp vpnv4 unicast vrf BLUE 10.99.0.6/32 detail
BGP routing table entry for 10.99.0.6/32,
  Route Distinguisher: 65000:101
  Received Label 0xe0040
  Extended community: RT:65000:100
  Source AFI: VPNv4 Unicast, Source VRF: default,
    Source Route Distinguisher: 65000:106
```

RD 只讓路由唯一

BGP 因此能同時承載 BLUE 與 RED 的 10.99.0.6。RD 不是隔離。

兩台 PE 不必相同

每台 PE 對同一個 VRF 用不同的 RD 是常見的設計；RD 可以看出路由來自哪台 PE。

Route target 才決定匯入

兩台 PE 的 route target 相同，所以 VRF 仍會匯入對方的路由。

M7 · EVPN : E-Line 已建 , E-LAN 被拒

E-Line (VPWS)

Point-to-point · 已建置

SID uDX2

BGP route Type 1 (Ethernet A-D)

驗收 xconnect Up ; 外層 Next Header 為 Ethernet (143)

E-LAN · known unicast

Bridge-domain · 只有理論

SID uDT2U

BGP route Type 2 (MAC/IP)

預期 學到 MAC 之後 , unicast 走 uDT2U

E-LAN · BUM

Bridge-domain · 只有理論

SID uDT2M

BGP route Type 3 (IMET)

預期 Ingress PE 對每個其他 PE 各複製一份

Lab result

XRd vRouter 26.2.2 與 XRv9000 26.1.1 都拒絕 bridge domain (VPLS bridge domains not supported on this platform) , 雖然 manager 列出 uDT2U 與 uDT2M 。 E-LAN 只能講理論 。 這是 CML 實驗環境的限制 , 真實環境是否支援要看具體硬體。

M7 · E-Line 的 route、SID 與 MTU

Type 1 route

- ESI 為 0 (CE 只接一台 PE) ; Ethernet tag 300 就是 service 編號
- Label 917680
- L2 attribute 的 MTU 為 0 : 不檢查 MTU
- RT 65000:300 由 AS 與 instance 自動產生

Label 到 SID

- Prefix-SID TLV type 6 (L2 Service)
- Behavior End.DX2 NEXT-CSID , code 65
- 917680 = 0xE00B0 → function e00b → fcbb:bb00:1:e00b::
- EVPN 在這個平台顯示三個 byte (e00700) : 讀 bits

MTU 陷阱

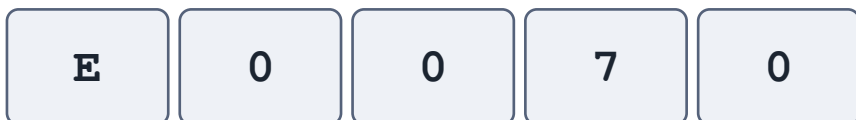
- 核心 1500 – 40 – 14 (客戶的 Ethernet header) = 1446 ; L3VPN 是 1460
- 1447 : PE1 對外層 packet 分段 (1510 + 75) , PE6 丟棄分段 , CE 沒收到任何訊息
- 核心 MTU 要能承載客戶 MTU + 54

CE-A 學到 CE-B 的 MAC , TTL 仍是 64 : 對客戶來說兩台 CE 之間沒有 router 。 98-byte 的 ping 在核心是 152 bytes 。

M7 · 為什麼 e0070 和 e00700 是同一個值

BGP 更新裡的 label 欄位有 24 位元，其中的 label 只有 20 位元。

Wireshark 顯示的 label : 917616 = 0xE0070 (20 位元)



PE1 上的 XR 顯示整個欄位 : 0xe00700 (24 位元)



前 16 位元 = function e007

最後 4 位元補零

```
Prefix-SID: transposition offset 48, length 16  
SID = fcbb:bb00:6 : e007 :: = fcbb:bb00:6:e007::  
PE1: 917680 = 0xE00B0 -> function e00b
```

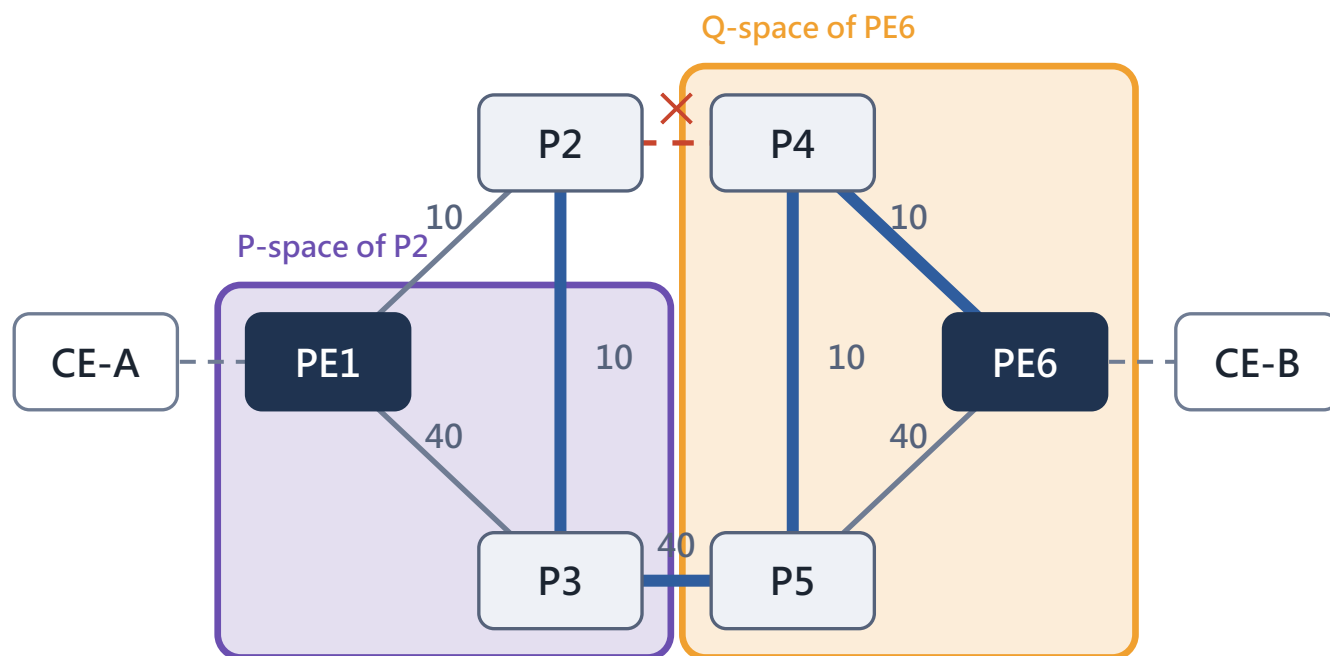
兩個數字是同一個值，只是寬度不同。路由器在最後多加四個零位元。要讀前 16 位元，不要數位數。



第四部分：Resilience 與 intent

路斷了怎麼辦？誰決定這條路？

M8 · TI-LFA : repair list 怎麼算



P2-P4 失效，PLR 是 P2。Repair path：P2-P3-P5-P4-PE6

1

P-space

P2 不經過故障 link 就能到的節點：PE1 與 P3。

2

Q-space

不經過故障 link 就能到 PE6 的節點：P4 與 P5。P3 不在裡面：P3 到 P5 的最短路徑會走向 P2-P4。

3

補上缺口

沒有節點同時在兩個 space。P3 與 P5 之間要用 adjacency SID (e005) 強制走那條 link。

Repair list (router 的輸出)

fcbb:bb00:3:e005::

uN 與 uA 同在一個 container。Metric 71，與手動計算一致。

M8 · 五個故障：手動計算與 router 的 repair list

失效	PLR	Repair path (自 PLR)	Router 的 repair list (Ep 9)	與手動計算比較
Link PE1-P2	PE1	PE1-P3-P2-P4-PE6	Plain LFA 經 P3，沒有 SID	相同：一般 LFA 已足夠
Link P2-P4	P2	P2-P3-P5-P4-PE6	fcbb:bb00:3:e005:: (71)	相同
Link P4-PE6	P4	P4-P5-PE6	fcbb:bb00:5:e001:: (51)	相同：最後一跳也要 uA
Node P2	PE1	PE1-P3-P5-P4-PE6	fcbb:bb00:3:e005:: (101)	Router 較短：一個 container，手動計算是三段
Node P4	P2	P2-P3-P5-PE6	fcbb:bb00:3:e005:e001:: (91)	相同；第二個 uA 前面沒有節點 uSID

Repair 是 H.Insert.Red

Router 把 repair 插進目前的 header，不加第二層外層 header。

Node protection 要另外要求

這個 release 要設 node-protecting tiebreaker。到 PE6 的流量不能防 PE6 自己故障：Path not found。

每次變更後看 coverage

show isis ipv6 fast-reroute summary 要是 100%。Link 斷成單向時，P4 曾只有 87%。

M8 · TI-LFA 在 wire 上：三個階段

階段	Packet	在 wire 上看到
1	前 44 個	P2 插入 node-protecting repair，送往 P3。Routing header 內是 GREEN service SID，Segments Left 為 1。
2	接著 15 個	較短的 list：P3 與 P3→P5 的 adjacency，仍帶 routing header。成因沒有驗證。
3	之後	PE1 自己的 explicit path：fcbb:bb00:3:e005:6:e008::，約 4 秒後改走新的最短路徑（M8 Microloop）。

第一次：repair 有出現

- P3 執行指令並 shift；剩下 e001（P5 到 PE6）
- P5 移除 routing header；PE6 只收到 service SID
- 4000 個 ping 全有回應；沒有 packet 從 P3 回 P2

第二次：同一個故障

- 看不到任何 repair 過的 packet
- 8 個 ping 遺失，wire 上約 1 秒沒有 packet
- 沒有驗證兩次為什麼不同

軟體 forwarder 證明 repair 存在並顯示它的格式，不能告訴你保護有多快。

M8 · Microloop avoidance 與 BFD

時間	事件 (P2-P4 失效)
34.253	P2-P4 的 BFD down ; IS-IS 在同一毫秒移除 adjacency
34.354	P2 安裝新 route (metric 71) , 並有一條 exp path : P3 , 再用 uA 到 P5
34.719	PE1 安裝新 route (metric 81) , 也有 exp path ; 一般 next hop 仍是 P2
40.219	exp path 消失 (安裝後 5.5 秒) ; route 再次受保護
還原	Adjacency 6.5 秒後 up ; PE1 再 6 秒後回到 metric 31 ; 沒有 ping 遺失

Microloop 的手動計算例：P2 已收斂，把流量送往 P3；P3 還沒收斂，又送回 P2。這不是 capture。

RIB update delay : 5000 ms

Cisco 的預設值，沒有 RFC 規定。這是 router 保留 exp path 的時間，不是中斷時間。

BFD : 100 ms × 3

偵測時間 300 ms。沒有 BFD 時，IS-IS 要等 hello holdtime，是好幾秒。

證據與缺口

開啟 avoidance 時，P2-P3 上看不到從 P3 回 P2 的客戶 packet。關閉 avoidance 的情況沒有測。

M8 · 虛擬 lab 量不到的時間

這個 lab 量不到

- 軟體 forwarder 跑在共用的 vCPU 上
- 同一個故障：第一次 0 個遺失，第二次 8 個遺失
- 5000 ms 是 exp path 的保留時間，不是中斷
- 約 0.1 秒的 repair 只是一次測試，不是保護時間

在硬體上這樣量

- 用 traffic generator 送固定速率、有序號的 stream，不用 ping
- 知道速率：10000 pps 時，每個遺失的 packet 是 0.1 ms
- 同一個物件反覆讓它故障；報告最差值與範圍
- 也測還原：link 回來時也會掉 packet

三種證據按時間排列

Log

BFD 與 adjacency 事件，在故障兩側的 router

Route 狀態

安裝時間與 exp path，在修復的 router 與遠端 PE

Wire

舊 path、repair path、新 path 上的 capture

M9 · SR Policy 的五個部分與 Binding SID

1

Headend

Policy 在 headend (PE1) 上，由 color 與 endpoint 識別。

2

Candidate path

一條或多條，各有 preference；最高且 valid 的那條是 active。

3

Segment list

Headend 放進 packet 的指令。

4

Binding SID

一個本地 SID，代表整條 policy。

5

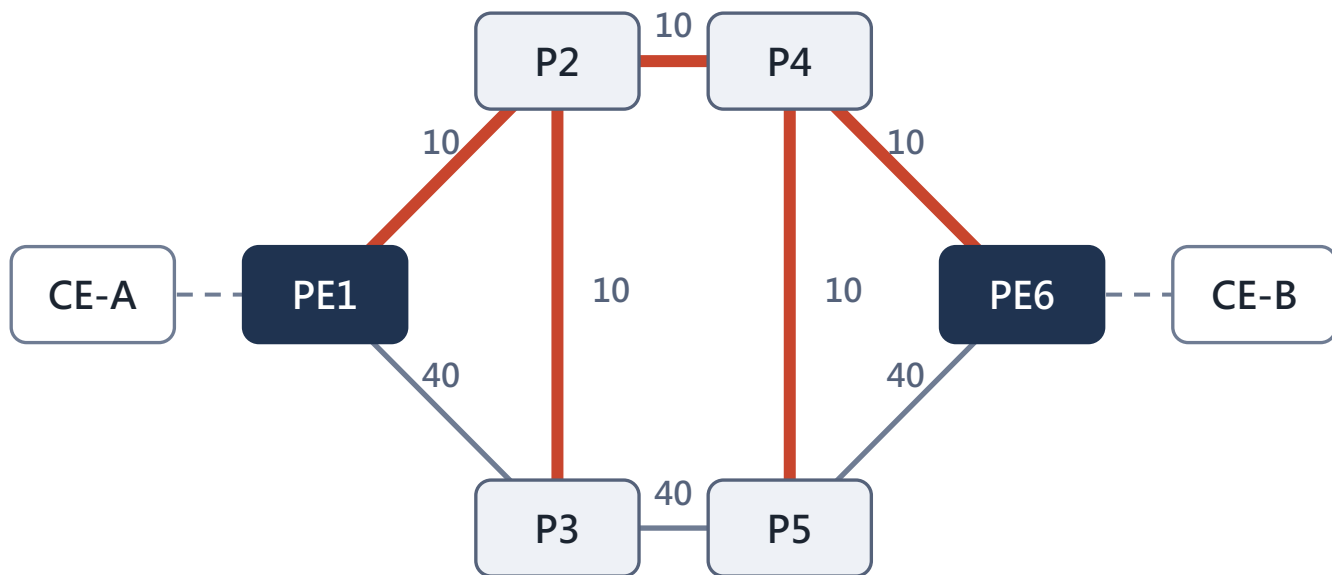
Color

由 egress PE 加在 route 上；ingress 用 color 與 next hop 找 policy。

項目	Lab 的值 (Ep 15)
Policy	color 100，endpoint 是 PE6 的 loopback
Candidate path	preference 100，list：P3、P5、PE6 (都是 uN)
Binding SID	fcbb:bb00:1:e008::
最後的 node SID	在 list 裡，但 PE1 不放進 packet
PE6 的 export	只有 BLUE IPv4 加 color 100：一個 update 帶三個 prefix
CEF (PE1)	H.Encaps.Red 加 service SID，再 H.Insert.Red {fcbb:bb00:3:5::}

Color 不改變 PE6 對 packet 做什麼，只改變 PE1 到 PE6 用哪條路。Binding SID 是動態分配的，不要設計成依賴這個值。

M9 · 意外的路徑：只用 uN 走不到下側



Segment list : P3, P5, PE6 (全部是 uN) · 138 bytes

PE1-P2-P3-P2-P4-P5-P4-PE6

Hop limit 255 → 249 : 七跳，兩次迴轉

段	IGP 最短路徑	Cost
PE1 → P3	經 P2	20，小於直連的 40
P3 → P5	經 P2、P4	30，小於直連的 40
P5 → PE6	經 P4	20，小於直連的 40

uN 只指定「到哪個節點」

- 節點之間仍走 IGP 最短路徑
- 要固定某一條 link，用這條 link 的 uA
- uA 的值是動態分配的：從 SID table 讀，不要寫死
- Policy up 且 valid，不代表流量走了你畫的那條線

M9 · Strict list、單向導流與 fallback

Strict list (preference 200)

- uA P3→P5 (e005) 與 uA P5→PE6 (e003)
- Container
fcbb:bb00:3:e005:5:e003:6:e004 : 六個 micro-SID , 138 bytes , 沒有 SRH
- Path PE1–P2–P3–P5–PE6 , hop limit 255 → 252
- PE1 到 P3 仍經 P2 : 我們沒有修那一段 , 只修必須修的
- Binding SID 不變

Color 只管一個方向

- Color 只加在 PE6 的 export
- Reply 走 IGP : PE6、P4、P2、PE1
- 要兩個方向都導流 , 另一側也要加 color

Policy 失效時

- Shutdown : Admin down , Binding SID 釋放
- BGP route 保留 color , 但不再指向 policy
- CEF 回到 locator 與 plain encapsulation : PE1–P2–P4–PE6 , ping 通
- Invalidation drop 預設關閉。要隔離 , 就打開並測試
- no shutdown 後 Binding SID 值相同 , 但它是動態的

M9 · Strict list 裡的 adjacency SID：為什麼，以及風險

```
segment-list VIA-P3-P5-UA
  srv6
    index 10 sid fcbb:bb00:3:e005:: ! uA P3 to P5
    index 20 sid fcbb:bb00:5:e003:: ! uA P5 to PE6
policy BLUE-C100
  candidate-paths
    preference 200
    explicit segment-list VIA-P3-P5-UA
```

請在測試之前，才在 P3 和 P5 上讀取這兩個值。

為什麼用 adjacency SID

- node SID 只說明封包去哪裡，路徑由 IGP 決定
- 只有 adjacency SID 能指定某一條鏈路

風險

- 這個平台上，動態 uA 值在重新開機後，或中斷超過 3 分鐘後，可能改變
- list 裡的舊 SID 可能指到另一條鏈路，或什麼都指不到

怎麼做

- 這類事件之後，重新讀 SID 並和 list 比較
- 不要把它寫死；讓 PCE 計算（第 16 集），或使用 node SID 與 Flex-Algo

M9–M10 · Policy、ODN、Flex-Algo 是三件不同的事

Explicit SR Policy

你自己寫 segment list

- 誰算路徑：人
- 觸發：BGP route 的 color 與 next hop 對上 Policy 的 color 與 endpoint
- 驗收：candidate path valid、業務確實導入、抓到預期的 DA

ODN

Color 觸發的 template

- 誰算路徑：PCE。這個 release 的 headend 不能算 SRv6 path
- 觸發：帶 color 的 route 到達；template 只有 intent，沒有 path
- 驗收：policy up、PCEP session up、CEF 的 SID、wire 上的 packet

Flex-Algo

IGP 多算一張 topology

- 誰算路徑：每台參與的 router
- 觸發：FAD 被選出，節點明確參與
- 驗收：FAD winner、參與狀態、獨立的 locator 與路徑

M9 · ODN 需要 PCE：三個測試

測試	設定	結果	流量走哪裡
A	沒有 PCE，headend 自己算	Policy down：Headend computation not supported for SRv6 policy	最短路徑 PE1–P2–P4–PE6，沒有任何告警
B	P4 當 PCE（三行設定）；dynamic pcep、metric latency	Policy up：preference 100 委派給 PCE 且 active；三個 protected uA；latency 3000	PE1–P3–P5–PE6（低 latency）
C	B 再加 constraints segments sid-algorithm 128	Policy down：SRv6 SIDs failed verification；PCE 回傳空 path	最短路徑，沒有任何告警

文件與 lab 一致

Cisco 25.x 的 guide 列為不支援：headend 計算的 dynamic path（A）、搭配 Flex-Algo constraint 的 PCE 委派（C）。XRv9000 26.1.1 結果相同。

清除測試 A 的陷阱

只移除 pcep 或 PCE 位址不夠：已委派的 path 還在。要刪掉 template 再重建。

沉默的 fallback

客戶付了低 latency 的錢，卻走高 latency，而且沒有任何設備告知。開機後 ODN policy 可能要一分鐘以上才 up。

M9 · ODN 的封包，與兩個可用的方法

```
CEF:  H.Insert.Red {fcbb:bb00:e004:e002::}  
!      PE1 own uA = outgoing interface to P3  
PE1→P3  fcbb:bb00:e004:e002:6:e006::  
P3→P5   fcbb:bb00:e002:6:e006::  
P5→PE6  fcbb:bb00:6:e006::
```

- 三個 adjacency 只用兩個 micro-SID：PE1 自己的 adjacency 就是出口 interface
- e004、e002 在 e000 範圍，是本地 micro-SID：每台 router 讀成自己的 adjacency
- PCE 選的是 algorithm 0 的 adjacency SID（依 latency），不是 Flex-Algo path

方法 1：Algo 128 的 service SID

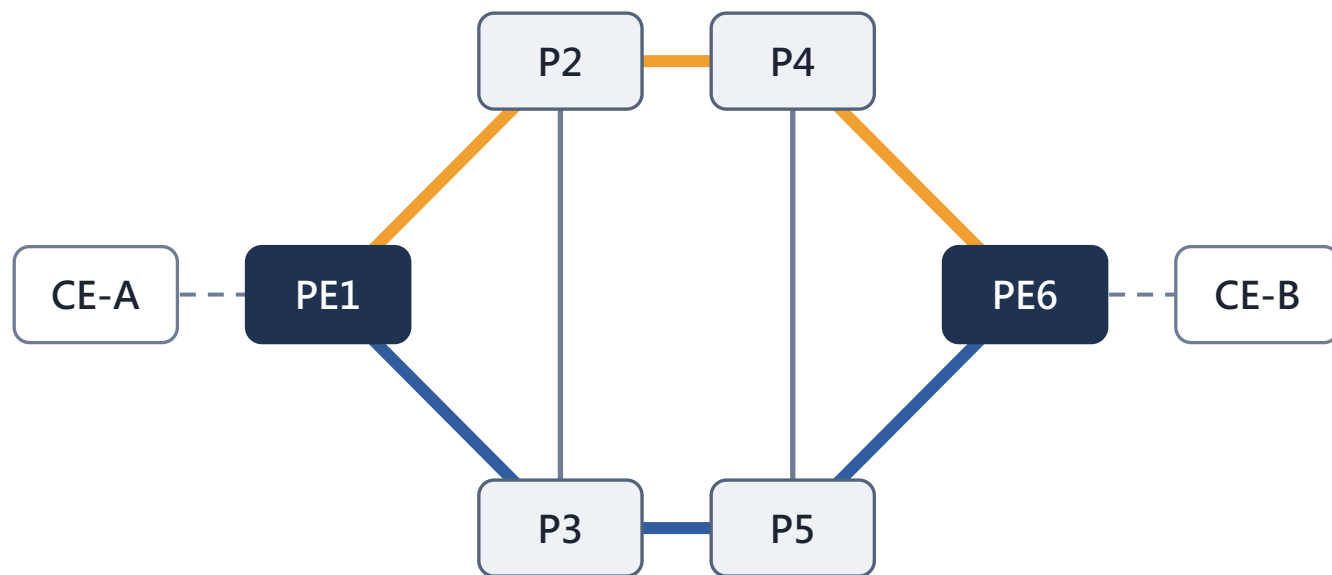
- RED IPv6 用 FA128 的 uDT6 fcbb:bb01:6:e004::
- Metric 3001，經 P3；沒有 policy，也沒有 color

方法 2：經 PCE 的 ODN

- Metric type latency；選到同樣三條低 delay link
- 兩個方法不能合併（測試 C）

Color 400 的 policy 只放 PE6 的 Algo 128 node SID：policy up，但最後一個 node SID 不在 packet 裡，流量經 P3 後改走 Algo 0。Up 不等於走了那條路。

M10 · Flex-Algo 128 : 同一張圖，兩條最短路徑



Algo 0 : IGP cost 30

Algo 128 : delay 3000 μ s

靜態廣告的 delay : 下側三條 link 各 1000 μ s · 其餘 5000 μ s

	Algo 0	Algo 128
Metric	IGP cost	Delay
PE1 看 PE6 locator 的 metric	31	3001
PE6 的 locator	fcbb:bb00:6::/48	fcbb:bb01:6::/48
PE1 的 next-hop	P2	P3

- 收到 FAD 不等於參與：每台 P 都要明確設定 flex-algo 128
- 不同 Block 的 uSID 不能放進同一個 container
- Algo 128 不等於低延遲：是 FAD 與實際 metric 決定結果

M10 · 128 代表什麼

使用者範圍

RFC 9350 : 128 到 255 留給營運者，意思由你決定。0 到 127 不是用在這裡。

不必連續

可以只用 128、140 和 200，不需要中間的數字。

只有一個定義勝出

好幾台 router 可以為同一個數字通告定義。
。選最高的 priority；相同時選最高的 System-ID。

數字本身沒有意思

被選中的定義才給出意思：metric type、限制條件。

我們的 lab

只用 128。XRd 顯示 Max Locators: 16，所以可以有很多 algorithm。你的平台可能不同。

```
P3#show isis flex-algo 128
Definition Priority: 200
Definition Source: PE1.00
Definition Metric Type: Delay
Local Priority: 128
```

M10 · 三個條件，以及怎麼把客戶放上去

1 Definition

- PE1 (priority 200) 與 PE6 (100) 廣告 128 的意義：metric type delay
- 選 priority 最高的；相同時選 system ID 最高的
- P3 沒有廣告 definition，也看得到選出的那一個 (PE1)

2 Participation

- 每台 router 都要設定 flex-algo 128
- 收到 definition 不等於參與
- P2-P5 只參與，不廣告 definition

3 Locator

- FA128：fcbb:bb01:n::/48，另一個 block (這裡的選擇，不是 RFC 規則)
- Algo 128 有自己的 node SID，每條 link 也有自己的 uA
- TI-LFA 留在同一個 topology：repair SID 是 fcbb:bb01:4::

Service：不用 policy，也不用 color

- PE6 從 FA128 分配 RED IPv6 的 SID：uDT6 fcbb:bb01:6:e004::
- PE1 到 PE6 走 PE1-P3-P5-PE6
- Return 的 SID 仍在 MAIN，走上側：兩個方向不同。要對稱，兩端都要分配

Delay 實驗：一個數字移動客戶

- P3-P5 的廣告 delay 兩端都從 1000 改成 20000 μ s；cost 不變
- Algo 128 移到 P2、P4，metric 15001；Algo 0 不變
- RED IPv6 跟著移到上側，GREEN 不動；還原後移回



第五部分：Operate

故障排除、邊界、遷移：把同一套問題用在真實的維運上。

M11 · 五層故障排除階梯

1

Local : locator Up · SID 已分配

`show segment-routing srv6 locator / manager`

2

IGP : locator 已廣告並安裝成 route

`show isis segment-routing srv6 locators`

3

Forwarding : CEF 有出口與 backup

`show cef ipv6 <locator> detail`

4

Service : BGP path、SID、color、policy

`show bgp vpnv4 unicast vrf ... detail`

5

End to end : 客戶流量與 wire

雙向 ping、不同大小、packet capture

動手前寫下假設

以及什麼證據可以推翻它

由第 1 層往上取證

停在第一個預期與實際不同的地方；不因上一層正常就跳過這一層

一次只改一處

每次改變都要連到一個觀察結果

修好後做回歸

重測原症狀，再測一個沒受影響的 service

M11 · 方法的速度：故障 A，與健康的基準

層	檢查	結果
1 Local · PE6	locator MAIN	Up：正常
2 IGP · PE1	show route ipv6 fcbb:bb00:6::/48	Network not in table：失敗
2 IGP · PE6	IS-IS 設定	IS-IS 底下少了 locator 那一行
修復	在 PE6 補回一行	所有層恢復正常

症狀：GREEN 不通，但 PE1 ping PE6 的 loopback 有回應。兩個指令就找到故障。沒有方法時，Ep 3 的故障排除從 BGP 開始，而 BGP 顯示 best。

健康時先留下基準

- 每台 PE、每個 service 記錄五層的指令輸出
- 保存 running config：每個 module 一個 snapshot
- 動態值要標成動態：uA 與 Binding SID 在重開機或超過三分鐘的中斷後會變
- 用 diff 比對，不靠記憶

M11 · SRv6 OAM : ping 與 traceroute 能證明什麼

測試 (從 PE1)	結果	能證明什麼
ping PE6 的 uN 、 uA 、 uDT4	3/3 有回應 · 來源是 PE6 的 loopback · 不是 SID	只證明 PE6 可達且擁有這個 SID ； 不證明 uDT4 在 BLUE 查了表
traceroute 到 node SID	P2 、 P4 、 PE6 · 走 IGP 路徑	一般的 IPv6 traceroute
traceroute ipv6 ... srv6	被 router 拒絕 (% Invalid input)	Cisco 記載兩種 SRv6 形式 · XRd 26.2.2 的 ipv6 說明樹裡都沒有 。 我們的說法只限 ipv6 語法
traceroute segment-routing srv6 policy name <name>	在這個版本的說明樹裡 ； 沒有執行	用來追蹤 policy 。 文件不一定和你的版本相符 ； 用 ? 並測試語法
traceroute 到 fcbb:bb00:3:5:6::	P2 、 P3 、 P2 、 P4 、 P5 、 P4 · 然後是星號	看得到 U-turn ； 最後一個節點丟棄 probe (checksum 用原本的 DA)

文件說 ping 到 SRv6 SID 不支援 ； lab 看到回應 。 文件是起點 ， 不是保證 ； 先試 。

工具不夠時 ， 在五個位置 capture

入口 access link

第一條 core link

中間的 link

最後一條 core link

出口 access link

M11 · 四個故障，四個 first divergence

症狀	第一個假設	First divergence 與證據	修復
L2 service：小 frame 通，大的不通（1000、1300 通；1400 不通）	某條核心 link 的 IPv6 MTU 較小	大 frame 到 P2，沒有出現在 P2-P4。P2 送 PTB 給 PE1 的 loopback，MTU 欄位 1336（不是 1400）；CE 什麼都沒收到	移除 P2 的 ipv6 mtu
CE-A ping CE-B 不通	回程缺 route	Request 到 CE-B，CE-B 有回應，reply 停在 PE6。PE6 的 BLUE 沒有 CE-A 的 route：PE1 export RT 65000:110，PE6 import 65000:100	RT 改回 65000:100
只有 GREEN 不通	SID 的 locator 不可達	Layer 4：locator fcbb:bb00:66::/48 inaccessible，CEF unresolved、drop。PE6 的 TEST locator 在 IS-IS 是 Inactive	GREEN 移回 MAIN
平時正常，P3-P5 一斷 RED IPv6 就全斷	保護或參與狀態消失了	故障前的 layer 2：P2、P4 移除 flex-algo 128 後，Algo 128 的 locator route 失去 Protected 與 backup	還原 flex-algo 128

M11 · 故障 1 與故障 3：變更內容

```
! Fault 1, on P2, toward P4
interface GigabitEthernet0/0/0/1
  ipv6 mtu 1400
! 1400 - 40 (outer IPv6) - 14 (inner Ethernet) = 1346
! Repair: no ipv6 mtu
```

```
! Fault 3, on PE6. Step 1: new locator, RED uses it
segment-routing srv6 locators locator TEST
micro-segment behavior unode psp-usd
prefix fcbb:bb00:66::/48
router bgp 65000 vrf RED address-family ipv4 unicast
segment-routing srv6 locator TEST
! missing: the locator is not added under IS-IS
! Step 2: RED back to MAIN, GREEN moves to TEST
! Repair: GREEN to MAIN, then: no locator TEST
```

故障 1

介面 MTU 仍是 1514，IPv6 只能用 1400。大於 1346 位元組的客戶封包過不去：1300 通過，1400 失敗。

故障 3

locator 沒有加到 IS-IS：show isis segment-routing srv6 locators 顯示 Inactive。RED 第一步時沒壞，因為它的 ODN policy 以 P5 的 adjacency SID 結尾。

M11 · 四個故障教了什麼

故障之前：lab 自己壞了

Lab 放了幾小時後，四條 link 變成單向：只有 12 個 adjacency，不是 16 個。兩端 shut 再 no shut 後恢復。每次記錄前先做 health check。

故障 1：PTB 少 64

PTB 的 MTU 欄位是 1336，不是 1400。目的地有 TI-LFA repair path 時，回報值是出口 MTU 減 64；轉送仍用真正的 MTU。L2 service 用不上這個 PTB，客戶看不到訊息。

故障 2：看兩個方向

Ping 不能告訴你哪個方向壞。Capture 可以。去看回覆停下來的那台 PE，即使回報問題的客戶在另一側。

故障 3：policy 會掩蓋故障

SID 變更也是中斷：PE1 用了 PE6 已移除的舊 SID 約 40 秒。RED 有 color，policy 最後的 uA 把 packet 送到 PE6，所以沒壞。

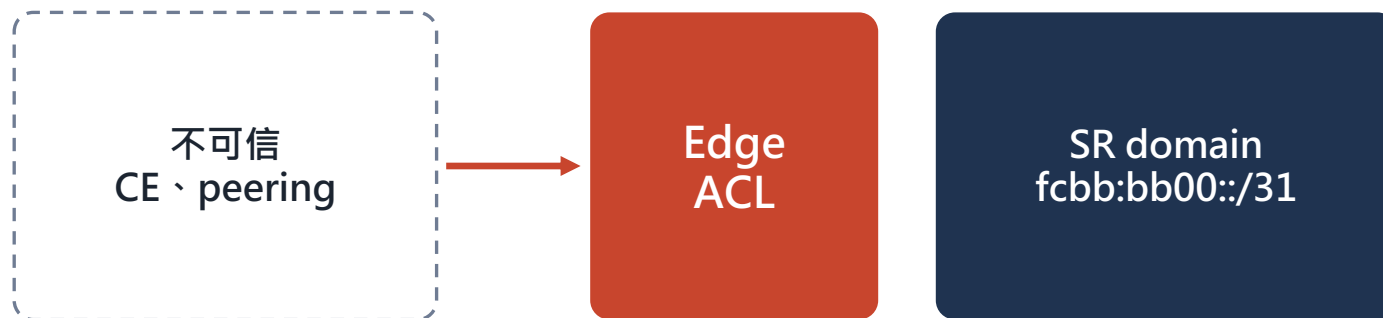
故障 4：等 topology 事件

修改時什麼都沒壞，所有測試都通；只有 layer 2 與 reference 的差異看得出來。P3-P5 斷了，只有付費買特殊路徑的客戶斷線。

Commit 失敗之後

先看 show configuration failed，再 abort。我們在 E-LAN 測試看過後續 commit 全部失敗，但沒有重做驗證：當成好習慣，不是已證實的規則。

M12 · 邊界：誰可以對 SID 說話



```
ipv6 access-list SRV6-BOUNDARY-IN
 10 deny ipv6 any fcbb:bb00::/31 log
 20 deny ipv6 any any routing log
 30 permit ipv6 any any
!
interface GigabitEthernet0/0/0/4
 ipv6 access-group SRV6-BOUNDARY-IN ingress
```

負向測試：從外部 host ping 四種 SID (uN、P4 的 uN、uA、BLUE uDT4) 必須沒有回應，PE6 的 loopback 仍要回應。ping 要加 -I 指定來源位址。

SID 是位址，不是 label

沒有 filter 時，外部 host 對 BLUE 的 uDT4 SID 也得到回應。MPLS 的預設會擋；SRv6 沒有。

Routing header 也要擋

Line 20。這個 release 不接受 routing-type 4，接受 routing 這個 keyword。

/31 剛好涵蓋兩個 block

整個 base block 是 /24，也涵蓋以後新增的 block。選一個，記下理由。

用 capture 驗證，不用 counter

Deny 行的計數維持 0，packet 卻被丟棄。在 PE1 的 core link 上看不到請求。

M12 · 規模：三種不同的上限

上限	這個 lab	注意
每個 block 的本地 SID	7680 (e000 到 fdff)	每個 VRF 用一到兩個；per-CE 分配會多很多。Adjacency 與 Binding SID 也在同一個範圍
SID 深度	Max-H-Encap 3、Max-H-Insert 3、Max-End-D 4	用 micro-SID，一個 SID 最多放六個指令；深度通常不是上限
Locator 數量	16 (XRd 的 manager 顯示)	每個 algorithm 與每個遷移步驟各要一個；硬體的數字不同 (M12 硬體)

Base block 與 ACL

Manager 顯示 base block 是 fcbb:bb00::/24。
ACL 用 /31 或 /24：見上一頁。

500 個 VRF 的算術

PE6 用了四個 L3 service SID (BLUE 兩個、RED 一個、GREEN 一個)。雙協定 500 個 VRF 最多 1000 個，放得進 7680。

MSD

Max segments left 與 Max End D 限制節點能處理與移除的 routing header。定義在 RFC 9352 與 RFC 8491。

M12 · SID 分配模式：500 個 VRF 的例子

模式	每一個計一個 SID	500 個 VRF 的 SID 數	已測試
per-vrf	VRF 與位址族	$500 \times 2 = 1000$	是
per-vrf-46	VRF ; IPv4 與 IPv6 共用一個 (uDT46)	$500 \times 1 = 500$	否
per-CE	CE 與位址族	$500 \times 4 \text{ 個 CE} \times 2 = 4000$	否
本地範圍	一個 block	7680	

這些數字是範例，不是量測值。per-CE 分配可能用掉本地範圍的一半以上；adjacency SID 與 Binding SID 也在同一個範圍。Cisco 8000 的指南描述 per-vrf-46，我們沒有測試。

M12 · Migration : 順序比指令重要



LDP → SRv6 (Ep 20)

- Egress 決定：每台 PE 可單獨切換，每個方向跟著它的 egress
- LDP 留著，rollback 只要一行；LDP 最後才拆
- SRv6 packet 比兩個 MPLS label 大 32 bytes：先測 MTU

SR-MPLS → SRv6 (Ep 21)

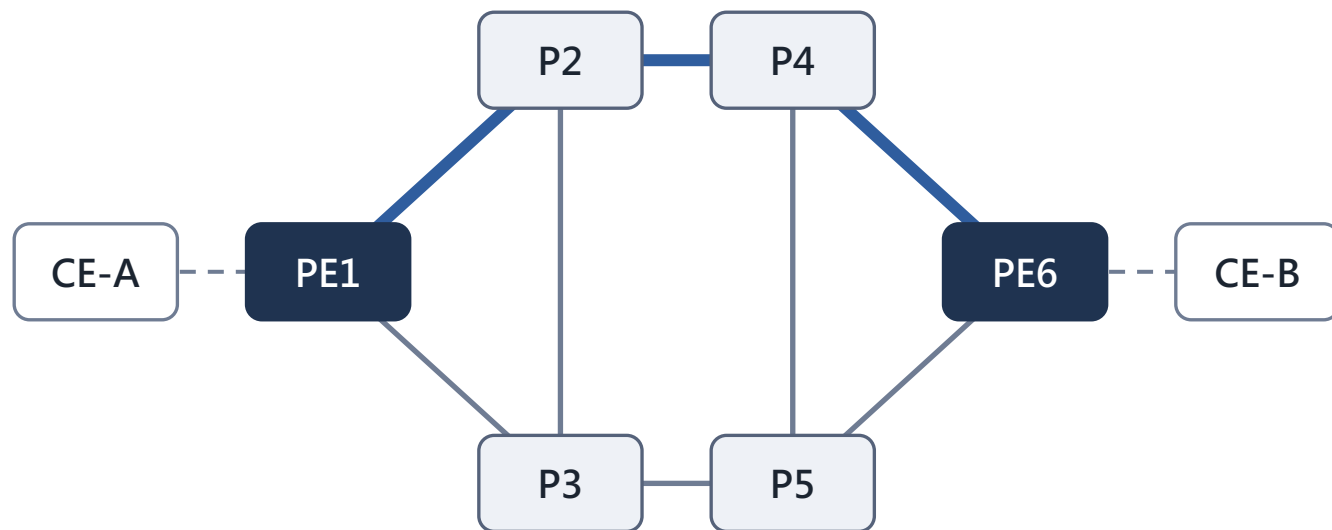
- Service 的切換與 Ep 20 相同
- 每一跳 label 相同，要 SRGB 相同
- 舊 transport 還載著 IPv4 BGP session：60 秒 + 99 bytes
- IPv4 VPN session 移除後，計數器停止

OSPF → IS-IS (Ep 22)

- 兩個 IGP 並存，不做 redistribution；用 distance 切換
- OSPF (110) 擁有 loopback、沒有 backup；IS-IS (115) 擁有 locator
- OSPF 最後才移除；選定的 route 不應改變

Rollback 的觸發條件在切換前就寫定。Rollback 是恢復已驗證的原狀並重新驗收，不是切回一個開關。

M12 · LDP 的起點：topology 與指令



GREEN 的路徑：PE1-P2-P4-PE6，每條核心鏈路都有 LDP

PE1 與 PE6 在 IPv4 loopback 10.0.0.1 與 10.0.0.6 之間有 VPNv4 會話；BLUE 與 RED 留在 SRv6。

```
PE1#show mpls ldp neighbor brief
10.0.0.2:0 ... 10.0.0.3:0 ...

PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32
detail
    Received Label 24000

PE1#show cef vrf GREEN 10.30.6.6/32 detail
    next hop 10.12.0.1/32 Gi0/0/0/0
    labels imposed {24009 24000}
```

24009 是通往 PE6 的 LDP label；24000 是 VPN label。

M12 · LDP 到 SRv6：每個方向跟著 egress

步驟	動作 (GREEN)	PE1 → PE6	PE6 → PE1
起點	LDP 與 MPLS VPN : label {24009 24000} · 122 bytes · BLUE 在 SRv6 : 154 bytes	MPLS	MPLS
1	PE6 為 GREEN 分配 SRv6 SID (fcbb:bb00:6:e008::)	SRv6	MPLS
2	PE1 做同樣的設定	SRv6	SRv6
Rollback 步驟 1	PE6 一行設定移除 SID 分配 (PE1 已切換)	MPLS	SRv6
3	先移除 IPv4 BGP session · 再移除 LDP	SRv6	SRv6

Session 不決定封裝 · SID 才決定

- GREEN route 從 IPv4 session 來也用了 SRv6；兩個 session 的 path 屬性相同，由最後的 tie-break 選出

清理後留下的東西

- IS-IS 仍廣告 IPv4 · IPv4 loopback 仍是 router ID · Ep 7 的 ICMPv4 來自 P4 的 IPv4 位址，它只因這個遷移設定而存在：清理後重測 MTU

M12 · SR-MPLS 到 SRv6 : label 16006 是誰在用

項目	LDP	SR-MPLS	SRv6
Transport	本地分配的 label	廣告的 index 加 SRGB	Locator route
GREEN 的 label	{24009 24000}	{16006 24000}	fcbb:bb00:6:e008::
100-byte ping	122 bytes	122 bytes	154 bytes
Transport 訊號	另一個協定 (LDP)	IGP	IGP

- Label 16006 = SRGB 起點 16000 + index 6 ; PE6 的 micro-SID node ID 也是 6 。一個數字，兩種 data plane
- 每一跳 label 都是 16006，因為各 router 的 SRGB 相同；SRGB 不同，label 值就可能不同。LDP 則各 router 獨立分配

GREEN 搬走後，16006 還在計數

- 60 秒沒有客戶流量：counter +99 bytes
- Capture：兩個有 label 的 packet，都是 10.0.0.1 到 10.0.0.6 的 TCP 179：BGP keepalive 59 bytes 加 TCP ACK 40 bytes
- 原因：IPv4 BGP session 的 next hop 10.0.0.6 經 SR-MPLS 解析
- 移除 session 後 counter 不再增加；GREEN 仍然通

M12 · SR-MPLS 到 SRv6 : 四個步驟與指令

```
! 1 SR-MPLS (here P2, index 2)
router isis CORE address-family ipv4 unicast
  segment-routing mpls
router isis CORE interface Loopback0 address-family ipv4 unicast
  prefix-sid index 2
! GREEN: labels imposed {16006 24000}

! 2 GREEN to SRv6, on PE6 and PE1
router bgp 65000 vrf GREEN address-family ipv4 unicast
  segment-routing srv6 locator MAIN alloc mode per-vrf
! CEF: SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
```

```
! 3 remove the IPv4 VPN session, then read the counter
router bgp 65000
  no neighbor 10.0.0.6
PE1#show mpls forwarding prefix 10.0.0.6/32 (00:28:51)
16006 16006 SR Pfx (idx 6) Gi0/0/0/0 10.12.0.1 1251
PE1#show mpls forwarding prefix 10.0.0.6/32 (00:29:53)
16006 16006 SR Pfx (idx 6) Gi0/0/0/0 10.12.0.1 1251

! 4 remove SR-MPLS, on every router
no router isis CORE address-family ipv4 unicast
no router isis CORE interface Loopback0 address-family ipv4
unicast
no router isis CORE interface Gi0/0/0/0 address-family ipv4
unicast
```

第 3 步之後計數器停止 (1251 → 1251) 。第 4 步要最後才做。第 4 步之後，IS-IS 只剩 SRv6 的設定；再執行 ping 確認 GREEN 正常。

M12 · SR-MPLS 與 SRv6 的對應，與遷移順序

SR-MPLS	SRv6
Prefix SID	uN：到這個節點
Adjacency SID	uA：用這條 link
TI-LFA 的 label stack	Repair SID list，由修復點加上
Binding label	uB6 Binding SID
VPN label	Service SID (可路由：所以要有邊界)

概念不變，變的是 data plane 與 service 的編碼。

順序

- 先遷移每個 service，包含所有 VRF 與 L2 service
- 找出還在用 label 的流量：增加中的 counter、IPv4 BGP session、SR-MPLS policy
- 移除 IPv4 VPN session，確認 counter 停止
- 最後才移除 SR-MPLS 與 prefix SID

第 4 步：移除 SR-MPLS

- 在每台 router 上，從 IS-IS 移除 IPv4 位址族 (instance 與每個介面)
- 這也移除 prefix SID 與 MPLS 的 segment routing
- 要在計數器停止之後才做；之後 IS-IS 只剩 SRv6 的設定

M12 · OSPF 到 IS-IS : 兩個 IGP 同時跑

Route	兩個 IGP · 預設 distance	步驟 1 之後 (IS-IS distance 100)
PE6 的 loopback	OSPF (110) · metric 30 · 一條 path · 沒有 backup	IS-IS · 有 TI-LFA 保護
PE6 的 locator	IS-IS (115) · metric 31 · 有 TI-LFA backup	IS-IS
BGP 看到的	Next-hop metric 30 (OSPF) · locator metric 31 (IS-IS)	兩個都來自 IS-IS

步驟 0

兩個 IGP 同時跑 · cost 相同 (下側 40 · 其餘 10) 。改變偏好之前 · 確認選到的 path 相同。

步驟 1

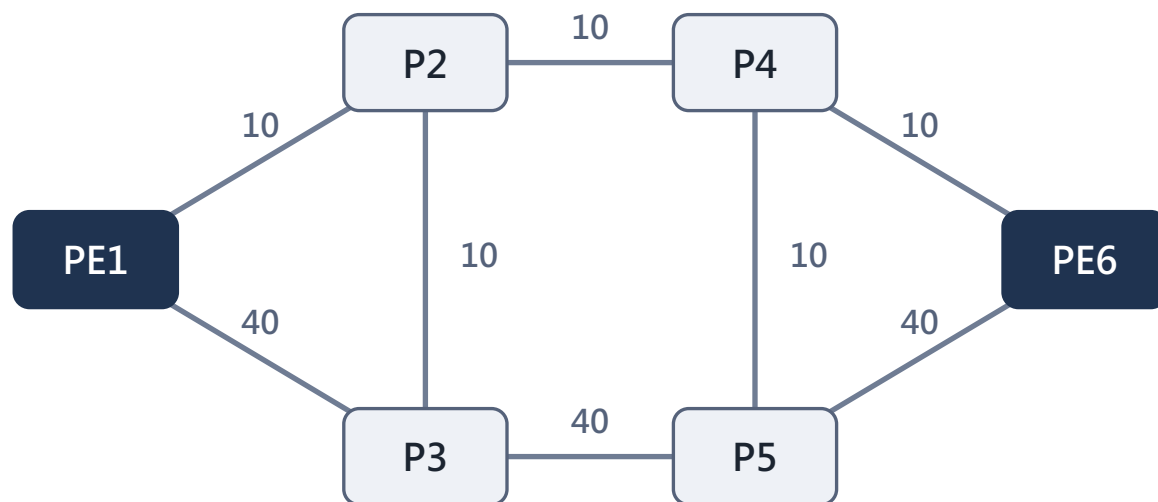
每個 address family · 每台 router 設 IS-IS distance 100 。逐台驗證。

步驟 2

等過穩定期 · 再移除 OSPF 。若選定的 route 改變 · 就停下來 · 找出還依賴 OSPF 的 route 。

若 OSPF 的 loopback route 消失 · IS-IS 的 loopback route 可以接手；若沒有可用的 next-hop route · BGP 會失去這條 path 。遷移期間兩行都要看。

M12 · OSPF 到 IS-IS：拓撲與三個狀態



兩個 IGP 的 cost 相同：下側 40，其餘 10。IS-IS 通告 loopback 與 locator；OSPF 與 OSPFv3 在 area 0。

狀態	PE6 的 loopback	PE6 的 locator
1 兩個 IGP，預設 distance	OSPF 110	IS-IS 115
2 第 1 步：IS-IS distance 100	IS-IS 100	IS-IS 100
3 第 2 步：移除 OSPF	IS-IS 100	IS-IS 100

真正的變更是第 1 步。第 2 步只是移除一個沒有人使用的協定，選定的路由不應改變。

M12 · 動態的 SID 值：e008 變成 e002

測試 (PE1 的兩條 core link)	結果
T1	只 bounce Gi0/0/0/1 (policy 仍經 P2 up) : 所有值不變
T2	兩條 link 都 down 約 10 秒 (policy down , uA 與 uB6 全部釋放) : 所有值回來 , 不變
T3	兩條 link 都 down 240 秒 (超過 holdtime) , 先恢復 Gi0/0/0/1 , 一分鐘後恢復 Gi0/0/0/0 : 值依 owner 回來的順序 , 從最低的空位重新分配

Manager 顯示 SID holdtime : 3 分鐘

- Ep 15 的 color 100 Binding SID 是 e008 ;
Ep 22 變成 e002
- 設定沒有改 : binding-sid dynamic

超過三分鐘的中斷之後 , uA 與 Binding SID 都要重新檢查 , 或改用明確的值。

M12 · 硬體上要重新驗證什麼

主題	你的硬體：讀你的指南	我們的虛擬 router
Micro-SID profile	某些硬體需要 hardware profile，要 reload 才生效	不需要這個步驟
每個 carrier 的 SID 數	依路由器角色而不同：讀單位 (micro-SID 或完整 SID)	Max-H-Encap 3、Max-H-Insert 3
Locator	讀你的指南：locator 與 block 的上限	XRd 顯示 16
SID format	檢查你的版本是否同時允許兩種 format	只有 f3216
要測試的系列	Cisco NCS 540、ASR 9000、Cisco 8800：每個系列有自己的指南與限制；每張可能載運 packet 的 card 都要查	不適用
E-LAN	要有支援的 forwarding 實作	XRd 與 XRv9000 都拒絕 bridge domain

M12 · 硬體測試計畫：六件事

1

Profile 與 reload

依文件的順序套用 profile 並 reload，再驗證狀態，之後才設定 locator

2

Encapsulation 上限

拿最長的 policy 與最長的 repair path，對照 encapsulation 的上限

3

MTU 與 ICMP

每個案例都 capture。記錄已安裝的 backup 類型：TI-LFA、plain LFA、無。有 TI-LFA 時 PTB 少 64

4

故障時間

用持續的流量量測。每秒一個 ping 量不出短暫的中斷

5

E-LAN

把 Ep 14 的理論拿到支援的硬體上測：route type、MAC learning、broadcast 複製、multihoming

6

EVPN IRB anycast gateway

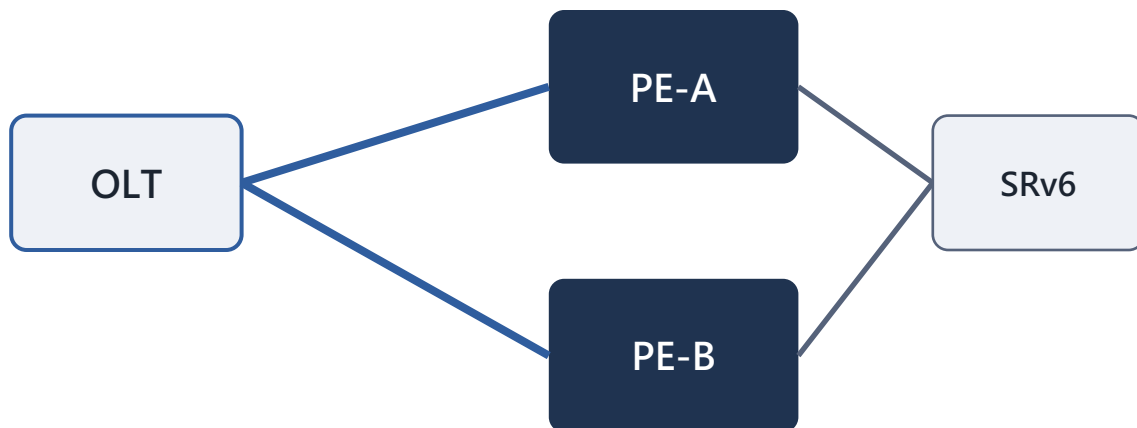
OLT bundle 與兩台防火牆兩種情況：route type、故障、MAC 移動（下一頁）

量測要特別小心的虛擬結果

Repair 中斷、BFD 偵測、microloop delay；SID ping 的回應；ACL deny counter；預設不複製 DSCP。這些都要在目標硬體上重測。

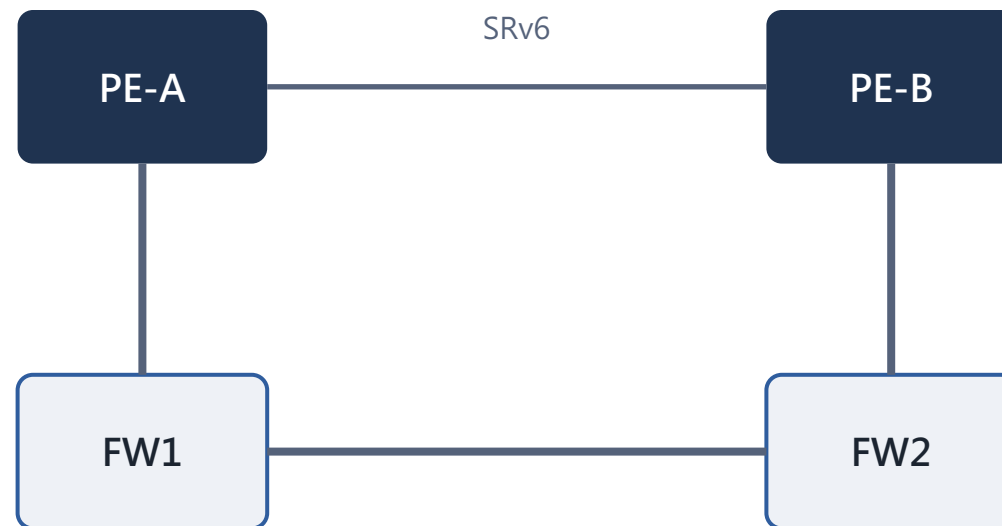
M12 · 只能在硬體上做的測試：EVPN IRB anycast gateway

情況 1：OLT 以 bundle 接到兩台 PE



同一個 ESI；IRB 在兩台 PE 上使用相同的 IP 與相同的 MAC

情況 2：兩台防火牆，方形連接



做法 1：兩台 PE 使用相同的 anycast gateway。做法 2：防火牆之間用 HSRP 或 VRRP，PE 只承載 VLAN。我們沒有測試。

- 控制平面：擷取 route type 1、2、4 (ESI、gateway 的 MAC 與 IP)
- Gateway：兩台 PE 都以相同 MAC 回應 ARP 或 ND
- 故障：關閉一條 PE 鏈路，量測中斷時間並確認恢復
- 迴圈與 MAC 移動、路徑 (哪台 PE、哪個 SID)：在兩台 PE 與核心上擷取



Lab 與學習計畫

同一個 lab，從第一天用到畢業。

Lab 平台：CML 上的六台 XRd vRouter 與兩台 CE

6 + 2

台 XRd vRouter 26.2.2 + Alpine CE-A、
CE-B

2 / 7

vCPU / GiB · 每台 XRd

2.10.0

我們使用的 CML 版本

可以證明

- Config 語法是否被接受
- Control plane：IS-IS、BGP、Flex-Algo、PCEP
- SID 分配與 behavior
- Packet 格式：每條 link 都能抓包
- TI-LFA repair list、CEF 安裝，以及 wire 上的 repair
- LDP、SR-MPLS 與 VPWS 的 data plane
- 結果標記為 Virtual lab

不能證明

- 收斂時間、50 ms、流量間隙：同一個故障曾 0 個、也曾 8 個遺失
- NPU 資源、profile 與 reload、line card 差異、規模
- E-LAN：XRd 26.2.2 與 XRv9000 26.1.1 都拒絕 bridge domain
- ODN 搭配 Flex-Algo constraint：不支援
- Link 可能沒有任何變更就變成單向：先確認 16 個 adjacency

12 個 lab 的階梯，對應 24 集

L1 State map

誰保存哪一種 state

Ep 1

L2 Two lookups

分別弄壞 locator 與 SID

Ep 3

L3 MTU 與 PMTUD

讓 PTB 真的出現

Ep 6-7

L4 uSID shift

ping 一個 container，抓三條 link

Ep 4-5

L5 Bring-up

把一台節點重建到可驗收

Ep 2

L6 IS-IS 五層

三個只壞一層的故障

Ep 8

L7 L3VPN

雙 tenant、dual-stack、隔離

Ep 12-13

L8 EVPN

VPWS；E-LAN 只有理論

Ep 14

L9 TI-LFA

手動計算與 CEF 的 repair list 對照，加 microloop

Ep 9-10

L10 Policy、Flex-Algo

先導流，再證明；ODN 要 PCE

Ep 11、15-16

L11 故障排除

五層方法，四個只給症狀的故障

Ep 17-18

L12 邊界、遷移、畢業

邊界 ACL、三種遷移、五個案例

Ep 19-24

四週學習節奏

第 1 週

第 2 週

第 3 週

第 4 週

The packet 與 bring-up

M1-M4

L1-L5

Ep 1-7

完成時

能手動計算 container 並用抓包驗證

Reachability 與 services

M5-M7

L6-L8

Ep 8、12-14

完成時

能沿 resolution chain 找出第一個分歧

Resilience 與 intent

M8-M10

L9-L10

Ep 9-11、15-16

完成時

能預測 repair list 與 Policy 的實際路徑

Operate、遷移與畢業

M11-M12

L11-L12

Ep 17-24

完成時

能交付畢業實作：五個區域、五個有證據的案例

每週約 3 小時教材加 3 小時 lab，是設計上的估計值，不是已量測的工時。第 4 週有 8 集，最重；可以拆成兩週。

畢業實作：建五個區域

拓撲：兩台 PE、四台 P、兩台客戶 host，各 PE 接一台 host

Link 要有不同的 cost，讓 loose、strict 與 repair path 都有替代 link。平台要支援 SRv6 forwarding：我們用 XRd vRouter；只有 control plane 的 XRd image 沒有所需的 forwarding。

1 Underlay

IS-IS、兩個 block 的 locator、node SID 與 adjacency SID

2 保護與 algorithm

TI-LFA、microloop avoidance、以 delay 為基礎的 Flex-Algo

3 Services

兩個位址重疊的 L3 tenant，加一條點對點 L2 service

4 Steering

一條 explicit policy，加一條由 PCE 計算的 ODN policy

5 Operations

健康基準、邊界 ACL、有 rollback 的遷移演練

建置順序：每一步先通過 checkpoint 才做下一步，每個 checkpoint 都保存每台 router 的 config

IS-IS 與 locator

保護

Tenant

VPWS

Policy 與 ACL

畢業實作：五個案例與評分

案例	做什麼	證據
A	抓一個 BGP update，由 label 欄位與 Prefix-SID 屬性還原 service SID	update 的 capture 與解碼 (Ep 13)
B	流量進行中讓 link 故障，數遺失的 packet；至少重做一次	兩條候選 path 的 capture (Ep 9 的兩次結果不同)
C	找出 L3VPN 與 L2 service 最大的 packet；各測高一個 byte	通過與失敗的 ping，加 capture
D	請同事弄壞一個 service 的一個方向；在兩台 PE 上用 capture 找出	兩條 access link 與兩條 PE core link 的 capture
E	從不可信的 lab port 測 SID 可達性，過濾前後各一次	第一條 core link 的 capture；指定來源位址

Pass

五個區域都做完，而且案例 A、C、E 有證據

Merit

五個案例都有 capture 證據

Distinction

另外要解釋與我們結果的差異：指出改變的條件，並提出它造成影響的證據

畢業實作：每個案例的做法、量測與證明

案例	做法	量測	證明
A	在 PE6 上讓某個 VRF 的測試 loopback 抖動	更新送出時，擷取 PE6 的核心鏈路	解碼更新；917568 = 0xE0040 = e004
B	CE-A 到 CE-B 送連續流量；讓 P2-P4 故障	計算遺失封包；至少測兩次	兩條路徑的擷取：故障前 P2-P4，故障後 P3-P5
C	CE-A ping CE-B，逐漸加大；先 L3VPN 再 VPWS	找出通過的最大封包；再加一位元組	L3VPN 1460 / 1461；VPWS 1446 / 1447
D	同事弄壞一個服務的一個方向	CE-A ping CE-B；找出封包在哪裡停止	兩台 PE 的接入與核心鏈路擷取
E	外部主機設定來源位址，ping 核心的 SID	ACL 之前有回應，之後沒有；不要信計數器	擷取 PE 的第一條核心鏈路

畢業實作：我們 lab 的參考值

項目	我們的值
PE1 看 PE6 的 locator metric	31：三條 link 各 10，locator 加 1
Algo 128 的 locator metric	3001，經 P3 與 P5
L3VPN frame	客戶 98 bytes，核心 138 bytes (不含 FCS)
Label 917568	依 transposition 還原為 function e004
VPWS	核心 frame 152 bytes，外層 next header 143
L3VPN 最大 packet	1460 bytes；一個 SRH entry 時 1436
VPWS 最大 packet	1446 bytes；多一個 byte 就遺失，客戶沒有錯誤訊息
Ping 多節點 container	沒有最後的 reply：checksum 用原本的 destination
ACL deny counter	維持 0，capture 卻顯示 packet 被丟棄
PTB	有 TI-LFA repair path 時是出口 IPv6 MTU 減 64；沒有 backup 或 plain LFA 時準確。轉送用真正的 MTU

畢業實作：我們錄製時遇到的五個陷阱

Link 單向

Link 沒有任何設定變更就變成單向。每次記錄前，確認每個 adjacency。

dot1q subinterface

在 CML 的 vRouter，帶 tag 的 frame 收得到，回覆卻不帶 tag。Untagged port 正常。

停止的節點

停止的 CML 節點會從 day-0 設定啟動。保存 snapshot，並確認你的 image 是否保存設定。

多個位址的 host

Host 有多個位址時，可能選到意外的來源。每個測試都指定來源位址。

Commit 失敗

先看 show configuration failed，再 abort。從 snapshot 還原已知的設定，並驗證還原後的狀態。

十六個最常見的錯誤結論（上）



「SR 的 core 沒有 state」

IGP、FIB、local SID 與 protection state 都還在



「SL = 0 就是處理完了」

還要看 DA、Argument、behavior 與 Next Header



「六個 slot 就能放六個節點」

Service function 與 Block 切換都會佔位



「DA 和我算的一樣，所以沒有 SRH」

有沒有 SRH 要看 Next Header 鏈



「commit 成功，data plane 就正常」

Config 接受、SID 安裝、遠端可達、端到端是四層



「TI-LFA 一定 50 ms」

同一個故障：一次 0 個遺失，一次 8 個。速度是硬體的問題



「BGP Up 又有 route，tenant 就走 SRv6」

沿 resolution chain 逐環核對，再抓包



「Algo 128 就是低延遲」

由 FAD、參與節點與實際 metric 決定

十六個最常見的錯誤結論（下）

✗ 「Policy up，流量就走我的路」
第一條 policy up 且 valid，卻走了七跳、兩次迴轉

✗ 「ODN 會自己算路徑」
這個 release 需要 PCE；沒有 PCE 時流量沉默地走最短路徑

✗ 「SID 有回應，這個 SID 就有作用」
回應來自 loopback，uDT4 沒有查 VRF

✗ 「Traceroute 停了，path 就斷了」
最後一個節點丟棄 probe (checksum)；用 capture 確認

✗ 「Deny counter 是零，ACL 沒作用」
Capture 顯示 packet 被丟棄；反過來，counter 也不是證明

✗ 「所有測試都通過，沒有問題」
故障 4 通過所有測試，直到 link 斷了

✗ 「兩台 PE 必須在同一個維護窗口一起切」
每台 PE 各自移動，GREEN 全程可用

✗ 「PTB 的 MTU 就是真正的 MTU」
有 TI-LFA repair path 時，PTB 是出口 MTU 減 64；轉送用真正的 MTU

依據的標準

RFC 8402

Segment Routing 架構

RFC 8754

IPv6 Segment Routing Header

RFC 8986

SRv6 Network Programming

RFC 9800

Compressed SRv6 Segment List (uSID)

RFC 9352

IS-IS 的 SRv6 擴充

RFC 9252

基於 SRv6 的 BGP overlay service

RFC 9256

SR Policy 架構

RFC 9350

IGP Flexible Algorithm

RFC 9259

SRv6 OAM

RFC 8231

Stateful PCE 的 PCEP 擴充

RFC 8664

Segment Routing 的 PCEP 擴充

RFC 8214

EVPN 的 VPWS 支援

RFC 8660

SR 的 MPLS data plane

RFC 9513

OSPFv3 的 SRv6 擴充

RFC 8491

用 IS-IS 廣告 MSD

Implementation 類的結論依 Cisco IOS XR 各平台的 Segment Routing Configuration Guide，並以目標版本的實際輸出為準。證據見各集影片。



附錄：實驗步驟頁

每個實驗的指令、已記錄的輸出和判讀，依集數排列。在自己的 lab 照著做，並比較輸出。動態值（SID、計數器、時間）會不同：讀取它們，不要照抄。

Ep 1.1 · What P2 actually holds

路由器 / 工具 : P2 · 來源 : lab-evidence/01-state/P2.log

```
RP/0/RP0/CPU0:P2#show bgp summary
% BGP instance 'default' not active

RP/0/RP0/CPU0:P2#show route vrf all summary
VRF: **iid
connected          0          0          0          0
Total              0          0          0          0

RP/0/RP0/CPU0:P2#show segment-routing srv6 locator
Name                ID      Algo  Prefix                Status  Flags
FA128               1      128   fcbb:bb01:2::/48      Up      U
MAIN                2      0     fcbb:bb00:2::/48      Up      U

RP/0/RP0/CPU0:P2#show segment-routing srv6 manager | include Number of SIDs
Number of SIDs: 12 (0 stale)
```

- 現在我們檢查完成後實驗裡的 P2。這個實驗網路的所有服務都經過它。
- P2 上沒有執行 BGP。
- 沒有任何客戶 VRF。唯一的項目是一個沒有路由的內部項目。
- P2 有兩個 locator。MAIN 是我們最常用的 locator。第二個 locator 用於 Flex-Algo。第 11 集會說明它。
- P2 有 12 個本地 SID:它的 node SID 和 adjacency SID。不論有三個客戶還是三千個客戶,P2 保存的狀態都一樣。

Ep 1.2 · And what PE1 holds

來源：lab-evidence/01-state/PE1.log (service SIDs and route counts only)

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
fcbb:bb00:1:e004::          uDT4          'BLUE'          bgp-65000        InUse   Y
fcbb:bb00:1:e005::          uDT4          'RED'           bgp-65000        InUse   Y
fcbb:bb00:1:e006::          uDT6          'BLUE'          bgp-65000        InUse   Y
fcbb:bb00:1:e007::          uDT6          'RED'           bgp-65000        InUse   Y
fcbb:bb00:1:e00a::          uDT4          'GREEN'         bgp-65000        InUse   Y

RP/0/RP0/CPU0:PE1#show route vrf all summary
VRF: RED      ...  bgp 65000      2
VRF: BLUE     ...  bgp 65000      2
VRF: GREEN    ...  bgp 65000      1
```

- 現在看邊緣。這是同一個完成後實驗裡的 PE1。
- PE1 為每個客戶的每個位址族各有一個 service SID:BLUE 兩個,RED 兩個,GREEN 一個。
- PE1 也保存每個客戶來自 BGP 的遠端路由。每個客戶的狀態在邊緣,而且只在邊緣。

Ep 1.3 · What the customer sees

路由器 / 工具 : CE-A · 來源 : lab-evidence/11-ce/v3-CE-A-ping.log

```
CE-A:~$ sudo traceroute -n -s 10.1.1.2 10.1.6.2 -w 1 -q 1 -m 8
traceroute to 10.1.6.2 (10.1.6.2) from 10.1.1.2, 8 hops max, 46 byte packets
 1  10.1.1.1  1.259 ms
 2  10.1.6.1  2.902 ms
 3  10.1.6.2  1.610 ms
```

- 我們從客戶這一側檢查同一件事,從 CE-A 到 CE-B 做一次 traceroute。
- 第一跳是 PE1,也就是客戶的閘道。
- 下一跳是 PE6,然後是 CE-B。四台核心路由器沒有出現。它們不碰客戶的封包,只處理外層標頭。

Ep 2.1 · Know what you are running

來源 : lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show version
Cisco IOS XR Software, Version 26.2.2 LNT
Copyright (c) 2013-2026 by Cisco Systems, Inc.

Build Information:
  Built By      : cisco
  Built On      : Thu Sep 10 00:39:05 UTC 2026
  Workspace     : /auto/srcarchive13/prod/26.2.2/xrd-vrouter/ws/
  Version       : 26.2.2
  Label         : 26.2.2

cisco XRd vRouter
cisco XRd-vRouter-C-01 processor with 9GB of memory
XRd vRouter Container
```

- 在設定任何東西之前,先記錄你使用的軟體。這是每份實驗報告的第一行。
- 版本是 IOS XR 26.2.2。
- 平台是 XRd vRouter,一個容器。這個系列裡所有的輸出,都只來自這個平台和這個版本。

Ep 2.2 · Step 1: addresses

```
interface Loopback0
  description router-id and SRv6 encapsulation source
  ipv6 address 2001:db8:100::1/128
!
interface GigabitEthernet0/0/0/0
  description TO-P2 cost 10
  ipv6 address 2001:db8:200:12::/127
  no shutdown
!
interface GigabitEthernet0/0/0/1
  description TO-P3 cost 40
  ipv6 address 2001:db8:200:13::/127
  no shutdown
```

- 設定有三個步驟。這是 PE1。其他五個節點使用同樣的模式。
- 第一步是位址。Loopback0 使用規劃中的 /128。它是 router ID,也是每個封裝封包的來源位址。
- 兩個核心介面配上各自的 /127 位址。沒有 IPv4 位址,沒有 MPLS,也沒有 LDP。

Ep 2.3 · Step 2: SRv6 and the locator

```
segment-routing
srv6
  encapsulation
    source-address 2001:db8:100::1
  !
  locators
    locator MAIN
      micro-segment behavior unode psp-usd
      prefix fcbb:bb00:1::/48
    !
  !
!
```

- 第二步啟用 SRv6。只有四行是重要的。
- 封裝來源位址是 loopback。當這台路由器封裝客戶封包時,這個位址就是外層的來源位址。
- Locator 的名稱是 MAIN。它的 prefix 是這個節點的 /48。
- 這一行讓它成為 micro-SID locator。node SID 是帶有 PSP 和 USD flavor 的 uN。第 4 集會說明這些 flavor。
- 在硬體平台上,你可能還需要 hardware profile 並重新開機。這台虛擬路由器兩者都不需要。

Ep 2.4 · Step 3: IS-IS carries the locator

```
router isis CORE
 is-type level-2-only
 net 49.0001.0000.0000.0001.00
 address-family ipv6 unicast
  metric-style wide
  segment-routing srv6
   locator MAIN
!
interface Loopback0
 passive
 address-family ipv6 unicast
!
interface GigabitEthernet0/0/0/0
 point-to-point
 address-family ipv6 unicast
 metric 10
```

- 第三步是 IS-IS。這段設定大部分是一般的 IS-IS。
- 我們啟用 IPv6 address family,並使用 wide metric。
- 這兩行是 SRv6 的部分。它們把 locator 掛到 IS-IS 上。然後 IS-IS 才會通告 locator,並建立 adjacency SID。
- 如果少了這兩行,提交仍然成功。Locator 在這台路由器上,但其他路由器都到不了它。
- 每個核心介面都是點對點,metric 依照我們的規劃。

Ep 2.5 · Check 1: IS-IS neighbors

路由器 / 工具 : P2 · 來源 : lab-evidence/01-baseline/P2.log

```
RP/0/RP0/CPU0:P2#show isis neighbors

IS-IS CORE neighbors:
System Id      Interface      SNPA           State Holdtime Type IETF-NSF
PE1            Gi0/0/0/0      *PtoP*         Up    28      L2    Capable
P3             Gi0/0/0/2      *PtoP*         Up    23      L2    Capable
P4             Gi0/0/0/1      *PtoP*         Up    28      L2    Capable

Total neighbor count: 3
```

- 檢查一,在 P2 上。首先,數一數鄰居,並和拓撲圖比較。
- P2 有三個鄰居:PE1、P3 和 P4。全部是 up、level 2、點對點。
- 拓撲圖上也是三個。每台 PE 有兩個鄰居,每台 P 路由器有三個。

Ep 2.6 · Check 2: what the router says it can do

來源 : lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 manager
Parameters:
  SRv6 Enabled: Yes
  SRv6 Operational Mode:
    Micro-segment:
      SID Base Block: fcbb:bb00::/24
  Encapsulation:
    Source Address:
      Default: 2001:db8:100::1
      Configured: 2001:db8:100::1
  SID Formats:
    f3216 <32B/16NFA> (2)
    uSID LIB Range:
      LIB Start    : 0xe000
      ELIB Start   : 0xfe00
    uSID WLIB Range:
      EWLIB Start  : 0xffff7
```

- 檢查二只有一個指令:show segment-routing srv6 manager。輸出很長,所以我們分三段來讀。
- SRv6 已啟用,運作模式是 micro-segment。
- 封裝來源位址就是我們設定的 loopback。
- SID 格式是 f3216。也就是 32 位元的 block,後面是 16 位元的 node ID 和 function ID。
- 本地 function 從十六進位 e000 開始。請記住這個數字。這個系列的每一張 SID 表裡都會看到它。

Ep 2.7 · Check 2, continued: behaviors

來源：lab-evidence/01-baseline/PE1.log (uSID entries; the full list also has End and End.DT forms)

```
Platform Capabilities:
SRv6: Yes
TILFA: Yes
Microloop-Avoidance: Yes
Endpoint behaviors:
  uN (PSP/USD)
  uA (PSP/USD)
  uDT6
  uDT4
  uDT46
  uDX2
  uDT2U
  uDT2M
  uB6 (Insert.Red)
Headend behaviors:
  T
  H.Insert.Red
  H.Encaps.Red
  H.Encaps.L2.Red
```

- 第二段列出這台路由器支援的 behavior。
- uN 和 uA 是節點和鄰接的 behavior。每一條路徑都使用它們。
- uDT 和 uDX 這幾項是 service behavior。它們用於三層 VPN 和點對點乙太網路。
- uB6 是 Binding SID。清單裡有某個 behavior,不代表每個相關功能都能用。例如,這個平台拒絕 bridge domain。
- headend 清單裡有 H.Insert.Red 和 H.Encaps.Red,都是 reduced 形式。它沒有一般的 H.Encaps。
- 少了這一行,後面每一集的封包都會不同。在設計任何東西之前,先讀這段輸出。

Ep 2.8 · Check 2, continued: depth

來源：lab-evidence/01-baseline/PE1.log

```
Signaled parameters:
  Max-SL      : 3
  Max-End-Pop-SRH : 3
  Max-H-Insert : 3 sids
  Max-H-Encap  : 3 sids
  Max-End-D    : 4
Default parameters (under srv6):
  Encapsulation:
    Hop-Limit      : value=0, propagate=No
    Traffic-class   : value=0, propagate=No
Max Locators: 16
Max SIDs: 64000
```

- 第三段是關於限制。
- 路由器把這些深度通告給鄰居。它可以插入或封裝 3 個 SID。
- 這裡的 SID 是完整的 128 位元容器。一個容器可以放好幾個 micro-SID。所以 3 這個上限比看起來更大。
- 預設情況下,路由器不會把客戶封包的 hop limit 和 traffic class 複製到外層標頭。第 13 集會在線路上檢查這一點。
- 上限是 16 個 locator 和 64000 個 SID。這是虛擬路由器的上限。不要把它們用在硬體上。

Ep 2.9 · Check 3: the locator and the local SIDs

來源：lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 locator
```

Name	ID	Algo	Prefix	Status	Flags
MAIN	1	0	fcbb:bb00:1::/48	Up	U

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
```

```
*** Locator: 'MAIN' ***
```

SID	Behavior	Identification	Owner	State	RW
fcbb:bb00:1::	uN (PSP/USD)	'default':1	sidmgr	InUse	Y
fcbb:bb00:1:e000::	uA (PSP/USD)	[Gi0/0/0/0, Link-Local]:0	isis-CORE	InUse	Y
fcbb:bb00:1:e001::	uA (PSP/USD)	[Gi0/0/0/1, Link-Local]:0	isis-CORE	InUse	Y

- 檢查三使用兩個指令。在這個系列裡,你會比其他任何指令都更常用到它們。
- locator MAIN 是 up,屬於 algorithm 0,prefix 就是我們設定的那個。
- 第一個 SID 是 node SID,也就是 uN。它屬於 SID manager。locator 一旦 up,它就存在。
- 接下來兩個 SID 是 adjacency SID,每個核心介面一個。它們屬於 IS-IS。這就是第三步重要的原因。
- 它們的 function 是 e000 和 e001。這是本地範圍的頭兩個值。

Ep 2.10 · The remote locator, as PE1 sees it

來源 : lab-evidence/01-baseline/PE1.log (four of sixteen routes)

```
RP/0/RP0/CPU0:PE1#show route ipv6 isis

i L2 2001:db8:100::6/128
    [115/30] via fe80::5454:ff:fe9b:6d1f, 01:12:13, GigabitEthernet0/0/0/0
i L2 fcbb:bb00:2::/48
    [115/11] via fe80::5454:ff:fe9b:6d1f, 01:12:32, GigabitEthernet0/0/0/0
i L2 fcbb:bb00:4::/48
    [115/21] via fe80::5454:ff:fe9b:6d1f, 01:12:13, GigabitEthernet0/0/0/0
i L2 fcbb:bb00:6::/48
    [115/31] via fe80::5454:ff:fe9b:6d1f, 01:12:13, GigabitEthernet0/0/0/0
```

- 本地 SID 只有在其他路由器到得了時才有用。所以我們在 PE1 上檢查來自 IS-IS 的路由。
- PE6 的 loopback metric 是 30,經由 GigabitEthernet 0。這是上方路徑,和我們的預測一樣。
- PE6 的 locator 是另一條路由,metric 是 31。在這個版本上,路由器會為 locator 在路徑 cost 上加 1。
- P2 和 P4 的 locator 也在這裡,metric 分別是 11 和 21。每個節點的 locator 都是一條一般的 IPv6 路由。
- 這是 SRv6 的一個重要特性。中間的路由器不需要新的功能,就能把封包轉送到 SID。它只需要一條路由。

Ep 2.11 · Check 4: ping the node SID

來源：04-behaviors/ua-PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:: source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to fcbb:bb00:6::, timeout is 2 seconds:
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms
```

- 這是 PE1 上的 ping 指令。目的位址是 PE6 的 node SID,fcbb:bb00:6::。
- source 選項設定來源位址。我們使用 PE1 的 loopback,因為每台路由器都到得了它。回覆需要這個位址才能回來。count 選項表示送出兩個封包。
- 路由器送出兩個 echo request,每個 100 位元組。
- 每個驚嘆號代表一個收到的回應。
- 成功率是 100%,2 發 2 中。Ping 成功了。但光是 ping 看不到路徑。為此,我們需要擷取檔。

Ep 2.12 · Check 4: the packet on three links

路由器 / 工具 : tcpdump · 來源 : 04-behaviors/ua-PE1-P2, ua-P2-P4, ua-P4-PE6 (first request on each link)

```
# PE1 to P2
IP6 (hlim 60) 2001:db8:100::1 > fcbb:bb00:6::: ICMP6, echo request, seq 0
# P2 to P4
IP6 (hlim 59) 2001:db8:100::1 > fcbb:bb00:6::: ICMP6, echo request, seq 0
# P4 to PE6
IP6 (hlim 58) 2001:db8:100::1 > fcbb:bb00:6::: ICMP6, echo request, seq 0
IP6 (hlim 64) 2001:db8:100::6 > 2001:db8:100::1: ICMP6, echo reply, seq 0
```

- 這是 tcpdump 的輸出。每一行是一個封包。每行開頭是 hop limit,接著是來源位址、目的位址和封包類型。我們顯示每個擷取檔中的第一個 echo request。
- 在第一條鏈路上,目的位址就是這個 SID,hop limit 是 60。
- 在第二條鏈路上,目的位址相同,hop limit 是 59。
- 在第三條鏈路上,hop limit 是 58。P2 和 P4 沒有做任何特別的事。它們只是照路由轉送封包。每台路由器把 hop limit 減 1。
- 最後這一行是第三條鏈路上的回覆。它來自 PE6 的 loopback,2001:db8:100::6,不是來自那個 SID。

Ep 3.1 · The egress tables behind it

路由器 / 工具 : PE6 · 來源 : lab-evidence/03-faults/v3wire/red-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid fcb::bb00:6:e006:: detail
fcb::bb00:6:e006::          uDT4          'RED'          bgp-65000          InUse  Y
  SID Function: 0xe006
  SID Identification (Context): { table-id=0xe0000001 ('RED':IPv4/Unicast) }
```

```
RP/0/RP0/CPU0:PE6#show cef ipv6 fcb::bb00:6:e006::/64
fcb::bb00:6:e006::/64, version 179, SRv6 Endpoint uDT4
  next hop VRF - 'RED', table - 0xe0000001
    0      Y      recursive          Lookup in table
```

```
RP/0/RP0/CPU0:PE6#show route vrf RED 10.2.6.2
Routing entry for 10.2.6.0/24
  Known via "connected", distance 0, metric 0 (connected)
    directly connected, via GigabitEthernet0/0/0/3
```

- 這些是完成這項工作的出口路由表。
- 這個 SID 是 uDT4。它的 context 是 VRF RED 的 IPv4 路由表。
- 在 CEF 裡,這個 SID 是一筆一般的 /64 項目。它的動作是在 RED 路由表裡查詢內層封包。
- 在 RED 裡,客戶位址直接連接在通往 CE-B 的埠上。這是條件四的證據。

Ep 3.2 · Healthy 1: the loopback ping and the locator route

來源 : lab-evidence/03-faults/v3/0-healthy-PE1.log

```
RP/0/RP0/CPU0:PE1#ping 2001:db8:100::6 source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to 2001:db8:100::6, timeout is 2 seconds:
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
Routing entry for fcbb:bb00:6::/48
  Known via "isis CORE", distance 100, metric 31, SRv6-locator, type level-2
  Routing Descriptor Blocks
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
      Route metric is 31
    fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)
      Repair Node(s): 2001:db8:100::5
      Route metric is 101
```

- 我們從健康的實驗環境開始。先做大多數工程師最先做的檢查:ping PE6 的 loopback。
- 目的位址是 2001:db8:100::6,也就是 PE6 的 loopback。來源位址是 PE1 的 loopback。
- 兩個回應都收到了,成功率是 100%。這個 ping 只測試 loopback,並沒有用到 locator。
- 現在看 locator。PE1 有一條到 fcbb:bb00:6::/48 的路由。IS-IS 安裝了它,路由類型是 SRv6-locator。
- 主要路徑從 PE1 的 Gi0/0/0/0 送出。路由 metric 是 31。
- 備援路徑從 Gi0/0/0/1 送出,metric 是 101。請記住:在健康狀態下,PE1 到這個 locator 有一條主要路徑和一條備援路徑。

Ep 3.3 · Healthy 2: the customer route in BGP

來源 : lab-evidence/03-faults/v3/0-healthy-PE1.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
Paths: (1 available, best #1)
  2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
    Received Label 0xe0080
    Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best
    Extended community: RT:65000:300
    SID Locator: fcbb:bb00:6::/48, RIB metric: 31
    T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:63
    Length [Loc-blk,Loc-node,Func,Arg]:[32,16,16,0], Tpose-len:16, Tpose-offset:48

RP/0/RP0/CPU0:PE1#show route vrf GREEN 10.30.6.6/32
Routing entry for 10.30.6.6/32
  Known via "bgp 65000", distance 200, metric 0, type internal
    2001:db8:100::6, from 2001:db8:100::6
      Nexthop in Vrf: "default", Table: "default", IPv6 Unicast, Table Id: 0xe0800000
```

- 接著看客戶路由。在 PE1 上,我們讀 VRF GREEN 裡 10.30.6.6 這個 prefix 的 BGP 詳細輸出。
- BGP 有一條路徑,來自 PE6。下一跳是 PE6 的 loopback,到它的 IGP metric 是 30。
- 收到的 label 是 0xe0080。它裡面放著 SID 的 function e008。故障 B 會改變這一行。
- 這條路徑是 valid 和 best。Route target 65000:300 把這條路由放進 VRF GREEN。
- BGP 也會檢查 SID 的 locator。這條 locator 路由在 RIB 裡,metric 是 31。故障 A 會改變這一行。
- 在 VRF 裡,這是一條 BGP 路由。它的下一跳在 default VRF 的 IPv6 路由表裡。PE1 必須在那張表裡解析下一跳。

Ep 3.4 · Healthy 3: the forwarding entry and the customer ping

來源：lab-evidence/03-faults/v3/0-healthy-PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
10.30.6.6/32, version 9, SRv6 Headend
  via fcbb:bb00:6::/128, 3 dependencies, recursive
    next hop VRF - 'default', table - 0xe0800000
    next hop fcbb:bb00:6::/128 via fcbb:bb00:6::/48
  SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
  Hash  OK  Interface          Address
    0    Y  GigabitEthernet0/0/0/0  fe80::5454:ff:fe66:75e1

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Type escape sequence to abort.
Sending 3, 100-byte ICMP Echos to 10.30.6.6 timeout is 2 seconds:
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/1/3 ms
```

- 現在看轉送。CEF 項目顯示 PE1 如何處理送往這個客戶的封包。
- 這筆項目是 SRv6 Headend, 有 3 個 dependencies。這代表路由器已經把這筆項目解析完成。
- 下一跳是 PE6 的 locator。PE1 用那條 /48 路由來解析它。這就是條件一。
- 動作是 H.Encaps.Red。SID list 裡有一個 SID:fcbb:bb00:6:e008::。這是 GREEN 在 PE6 上的 service SID。
- 最後一行顯示結果。封包從 PE1 的 Gi0/0/0/0 送出。
- 現在做客戶的 ping。來源位址是 PE1 在 VRF GREEN 裡的一個位址。三個封包都成功。

Ep 3.5 · Healthy 4: the customer ping on the wire

路由器 / 工具 : tshark · 來源 : /healthy2-PE1-P2.pcap (fields of two frames)

```
# PE1 to P2: the first request and its reply
frame 113, 154 bytes, request
  IPv6  2001:db8:100::1 > fcbb:bb00:6:e008::      hlim 255
  IPv4  10.30.1.1 > 10.30.6.6                    ICMP echo request
frame 114, 154 bytes, reply
  IPv6  2001:db8:100::6 > fcbb:bb00:1:e00a::      hlim 253
  IPv4  10.30.6.6 > 10.30.1.1                    ICMP echo reply
```

- 現在看封包。我們在 PE1 到 P2 的鏈路上擷取了客戶的 ping。每個訊框是 154 位元組:14 是乙太網路,40 是外層 IPv6 標頭,100 是客戶的封包。
- 請求帶有外層 IPv6 標頭。目的位址是 PE6 的 service SID,fcbb:bb00:6:e008::。來源位址是 PE1 的 loopback。
- 裡面是客戶的封包:一個從 10.30.1.1 到 10.30.6.6 的 IPv4 echo request。沒有 SRH,因為 SID list 只有一個 SID。
- 回覆往相反方向走。它的外層目的位址是 fcbb:bb00:1:e00a::。這是 GREEN 在 PE1 上的 uDT4 SID。
- 回覆的 hop limit 是 253。PE6 送出時是 255。P4 和 P2 各把它減 1。
- 這是健康狀態的基準。在兩個故障裡,我們擷取同一條鏈路,然後比較。

Ep 3.6 · Fault A: PE6 stops advertising its locator

```
! PE6
router isis CORE
  address-family ipv6 unicast
    segment-routing srv6
      no locator MAIN
!
commit
```

- 故障 A。我們在 PE6 上刪掉一行:IS-IS 底下的 locator。第 2 集已經提醒過你這一行。
- IS-IS 仍然通告 loopback。Locator 也還在 PE6 上。只有 locator 的通告消失了。

Ep 3.7 · Fault A: what PE1 sees

來源 : lab-evidence/03-faults/v3/A-PE1.log

```
RP/0/RP0/CPU0:PE1#ping 2001:db8:100::6 source 2001:db8:100::1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

RP/0/RP0/CPU0:PE1#show route ipv6 fcb:b00:6::/48
% Network not in table

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
Paths: (1 available, best #1)
  Received Label 0xe0080
  Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best
  SID Locator: fcb:b00:6::/48, RIB metric: inaccessible
```

- 現在我們從 PE1 做同樣的檢查。
- Loopback 的 ping 成功了。這正是在那通電話裡給出錯誤答案的檢查。
- Locator 路由不見了:Network not in table。條件一不成立。
- 但 BGP 仍然把這條路徑顯示為 valid 和 best。
- 只有這一行顯示了問題:SID locator 是 inaccessible。只有讀詳細輸出時,你才會看到它。

Ep 3.8 · Fault A: forwarding

來源 : lab-evidence/03-faults/v3/A-PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
10.30.6.6/32, version 9, SRv6 Headend
  via fcbb:bb00:6::/128, 0 dependencies, recursive
  unresolved
  SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
    0      Y      recursive              drop

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 0 percent (0/3)
```

- 對於一條 locator 不可達的最佳路徑,轉送會怎麼處理?
- 這筆項目還在,但沒有任何東西可以解析它:0 dependencies,unresolved。
- SID list 仍然在這筆項目裡。匆匆一看,你可能會以為封裝已經就緒。
- 最後一欄顯示 drop。
- 客戶的 ping 失敗了。

Ep 3.9 · Fault A: PE6 still looks healthy

路由器 / 工具 : PE6 · 來源 : lab-evidence/03-faults/v3/A-PE6.log (FA128 locator and other SIDs omitted)

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 locator
Name          ID      Algo  Prefix          Status  Flags
-----
MAIN          2       0     fcbb:bb00:6::/48 Up       U

RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid
SID           Behavior      Identification      Owner          State  RW
-----
fcbb:bb00:6:: uN (PSP/USD)  'default':6         sidmgr         InUse  Y
fcbb:bb00:6:e008:: uDT4         'GREEN'             bgp-65000      InUse  Y
```

- 現在我們在故障 A 期間檢查出口。這項檢查告訴你,故障不在哪裡。
- locator MAIN 是 up。PE6 仍然擁有 fcbb:bb00:6::/48 這個 prefix。
- GREEN 的 service SID e008 是一個 uDT4。BGP 擁有它,而且它在使用中。條件二成立。
- PE6 擁有它需要的一切。但網路的其他部分不知道這個 locator。故障出在通告。

Ep 3.10 · Fault B: PE6 stops allocating the service SID

```
! PE6
router bgp 65000
  vrf GREEN
    address-family ipv4 unicast
      no segment-routing srv6
  !
commit
```

- 故障 B 不一樣。我們在 PE6 的 BGP 底下,移除 VRF GREEN 的 SRv6 分配。
- Locator、IS-IS 和 loopback 都沒有改變。只有這一個客戶的 service SID 消失了。

Ep 3.11 · Fault B: the egress

路由器 / 工具 : PE6 · 來源 : lab-evidence/03-faults/v3/B-PE6.log (adjacency SIDs omitted)

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid
*** Locator: 'MAIN' ***
fcbb:bb00:6::          uN (PSP/USD)      'default':6      sidmgr
fcbb:bb00:6:e004::      uDT4                'BLUE'           bgp-65000
fcbb:bb00:6:e005::      uDT6                'BLUE'           bgp-65000
fcbb:bb00:6:e006::      uDT4                'RED'            bgp-65000
fcbb:bb00:6:e007::      uDX2                300:300          l2vpn_srv6

RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
  Local Label: 24004 (with rewrite);
  Advertised to peers (in unique update groups):
    2001:db8:100::1
```

- 從故障所在的位置開始:出口。
- PE6 的 SID 表裡有 BLUE、RED 和一個二層服務的 service SID。沒有 GREEN 的。條件二不成立。
- 這個結果出乎意料。沒有 SRv6 SID 時,BGP 使用了一個 MPLS 標籤 24004。
- BGP 把這個標籤通告給了 PE1。沒有出現任何警告。

Ep 3.12 · Fault B: the ingress

來源 : lab-evidence/03-faults/v3/B-PE1.log

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 100, metric 31, SRv6-locator, type level-2

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
  Received Label 24004
  Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
  via 2001:db8:100::6/128, 0 dependencies, recursive
  unresolved
  labels imposed {24004}
  0      Y      recursive          drop
```

- 現在看 PE1。
- 這一次,locator 路由正常。條件一成立。
- BGP 收到了標籤 24004。這條路徑仍然是 valid 和 best。
- 轉送想推入一個 MPLS 標籤,送往 PE6 的 loopback。這個核心沒有 MPLS 路徑。所以這筆項目是 unresolved,封包被丟棄。
- 出口少了一行,就把 SRv6 服務變成了一個無法運作的 MPLS 服務。

Ep 3.13 · Fault B: the ping and the wire

來源：lab-evidence/03-faults/v3/B-PE1.log, B-cap-PE1-P2.pcap, B-cap-P4-PE6.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Type escape sequence to abort.
Sending 3, 100-byte ICMP Echos to 10.30.6.6 timeout is 2 seconds:
...
Success rate is 0 percent (0/3)

# Captures on PE1 to P2 and on P4 to PE6, during this ping
no echo request on either link
```

- 我們在故障 B 期間執行客戶的 ping。
- 每一個點是一次逾時。有三個點,沒有任何回應。
- 成功率是 0%。
- 我們也在 PE1 到 P2 的鏈路,以及 P4 到 PE6 的鏈路上做了擷取。兩條鏈路上都沒有 echo request。PE1 在封包上線之前就丟棄了它。
- 在兩個故障裡,客戶看到的現象相同:ping 不通,線路上也沒有封包。只有路由器的輸出能顯示差別。

Ep 3.14 · Read the label field

來源 : lab-evidence/03-faults/v3 (A-PE1.log, B-PE1.log)

```
Healthy:
  Received Label 0xe0080
  SID Locator: fcb:b00:6::/48, RIB metric: 31
  T:1(Sid information), Sid:fcb:b00:6::(Transposed), Behavior:63
  Length [Loc-blk,Loc-node,Func,Arg]:[32,16,16,0], Tpose-len:16, Tpose-offset:48

Fault B:
  Received Label 24004
```

- 這是分辨這兩種情況的快速方法:讀收到的 label。
- 健康狀態時,label 是一個十六進位值 e0080。這是 SID 的 function e008,放在 label 欄位裡。
- 接下來幾行說明了方法。BGP 只送一次 locator,並把 16 位元的 function 放進 label 欄位。接收端再重組出完整的 SID。
- RFC 9252 把這種方法稱為 transposition。第 13 集會從擷取到的更新訊息中逐欄解讀。
- 在故障 B 裡,label 是一個一般的十進位 MPLS 標籤。在只有 SRv6 的核心裡,這就告訴你出了問題。

Ep 3.15 · The moment in between: on the wire

路由器 / 工具 : tshark · 來源 : lab-evidence/03-faults/stale-sid-P4-PE6.pcap (fields of two frames)

```
# P4 to PE6, in an earlier version of the lab (VRF RED)
frame 116, 154 bytes
  IPv6  2001:db8:100::1 > fcbb:bb00:6:e004::    hlim 253
  IPv6  2001:db8:a:201::1 > 2001:db8:a:206::6    ICMPv6 echo request
frame 117, 206 bytes
  IPv6  2001:db8:200:46::1 > 2001:db8:100::1    hlim 63
  ICMPv6 type 1, code 0: destination unreachable
```

- 這份擷取來自這個實驗的早期版本。客戶在 VRF RED 裡,使用 IPv6 位址。概念是相同的。
- 請求到達 PE6。外層目的位址是舊的 SID,fcbb:bb00:6:e004::。
- 裡面是客戶的封包:一個 ICMPv6 echo request。
- PE6 沒有這個 SID。它從自己的位址回覆 ICMPv6 type 1、code 0:destination unreachable。
- 封包到了正確的路由器,但路由器沒有這個 SID 的指令。這就是為什麼你要讀 PE6 的 SID 表。

Ep 4.1 · UN in the forwarding table

路由器 / 工具 : P3 · 來源 : lab-evidence/04-behaviors/P3-cef.log

```
RP/0/RP0/CPU0:P3#show segment-routing srv6 sid
fcbb:bb00:3::          uN (PSP/USD)      'default':3          sidmgr          InUse   Y

RP/0/RP0/CPU0:P3#show cef ipv6 fcbb:bb00:3::/48 detail
fcbb:bb00:3::/48, version 5, SRv6 Endpoint uN (shift)
  via ::/128, 0 dependencies, weight 0, class 0 [flags 0x0]
  next hop ::/128
```

- 我們現在在一台真實路由器 P3 的轉送表裡看這些 behavior。
- P3 擁有 uN fcbb:bb00:3::, 帶有 PSP 和 USD flavor。我們在這一集後面說明 flavor。
- 在 CEF 裡, 它是一筆 /48 項目。它的動作是 SRv6 Endpoint uN shift。這就是程序的第一步, 由路由器顯示出來。
- 這裡沒有出口介面。移位之後, 路由器查詢新的位址。

Ep 4.2 · UA in the forwarding table

路由器 / 工具 : P3 · 來源 : lab-evidence/04-behaviors/P3-cef.log

```
RP/0/RP0/CPU0:P3#show segment-routing srv6 sid
fcbb:bb00:3:e004::          uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:0:P      isis-CORE      InUse  Y
fcbb:bb00:3:e005::          uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:0      isis-CORE      InUse  Y

RP/0/RP0/CPU0:P3#show isis neighbors
P5          Gi0/0/0/1      *PtoP*          Up      29      L2      Capable

RP/0/RP0/CPU0:P3#show cef ipv6 fcbb:bb00:3:e005::/64 detail
fcbb:bb00:3:e005::/64, version 114, SRv6 Endpoint uA (shift)
  via fe80::5454:ff:febl:be7a/128, GigabitEthernet0/0/0/1, 8 dependencies
```

- 現在我們看同一台路由器上的 adjacency SID。
- P3 在 GigabitEthernet 0/0/0/1 介面上有兩個 uA SID。一個標記為 P,代表 protected。另一個沒有標記。
- 路由器可以在故障時繞道修復受保護的鄰接。當你只要那一條鏈路時,就用不受保護的那個。第 9 集會用到這個差別。
- 這個介面通往 P5。
- 在 CEF 裡,uA 是一筆 /64。Block、節點和 function 加起來是 64 位元。
- uA 有一個固定的下一跳,從 GigabitEthernet 0/0/0/1 送往 P5。uN 沒有固定的下一跳。

Ep 4.3 · Ping the uA address

來源：lab-evidence/04-behaviors/ua-PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcb::bb00:3:e005:6:: source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to fcb::bb00:3:e005:6::, timeout is 2 seconds:
..
Success rate is 0 percent (0/2)

RP/0/RP0/CPU0:PE1#ping fcb::bb00:6:: source 2001:db8:100::1 count 2
Type escape sequence to abort.
Sending 2, 100-byte ICMP Echos to fcb::bb00:6::, timeout is 2 seconds:
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms
```

- 現在我們使用這個 uA。你可以在自己的實驗裡做同樣的事。首先,在 CML 裡,在路徑的每一條鏈路上啟動封包擷取。然後從 PE1 執行 ping。
- 目的位址是 fcb::bb00:3:e005:6::。在 block 之後,它有三個部分:3 是節點 P3,e005 是 P3 通往 P5 的 uA,6 是節點 PE6。
- 這個 ping 失敗了:兩個點,成功率 0%。這不代表封包遺失了。我們在這一集後面說明原因。
- 作為比較,我們用同樣的方式 ping PE6 的 node SID。
- 這個 ping 成功了:兩個回應,成功率 100%。一般位址有回應,第一個位址沒有。擷取結果會顯示差別。

Ep 4.4 · The capture, link by link

路由器 / 工具 : tcpdump · 來源 : lab-evidence/04-behaviors/ua-PE1-P2, ua-P2-P3, ua-P3-P5, ua-P4-P5, ua-P4-PE6

```
$ tcpdump -nn -t -v -r <capture file> icmp6
# PE1 to P2
IP6 (hlim 60) 2001:db8:100::1 > fcbb:bb00:3:e005:6::: [icmp6 sum ok] ICMP6, echo request, seq 0
# P2 to P3
IP6 (hlim 59) 2001:db8:100::1 > fcbb:bb00:3:e005:6::: [icmp6 sum ok] ICMP6, echo request, seq 0
# P3 to P5
IP6 (hlim 58) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0x0316 -> 0xe31e!] ICMP6, echo request, seq 0
# P5 to P4
IP6 (hlim 57) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0x0316 -> 0xe31e!] ICMP6, echo request, seq 0
# P4 to PE6
IP6 (hlim 56) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0x0316 -> 0xe31e!] ICMP6, echo request, seq 0
```

- 現在我們來讀擷取檔。我們從 CML 儲存每一個擷取檔,並在自己的電腦上用 tcpdump 讀取。我們顯示每個檔案裡的第一個 echo request。
- 在 PE1 到 P2 的鏈路上,目的位址包含全部三個指令。hop limit 是 60。tcpdump 說校驗和正確。
- 在 P2 到 P3 的鏈路上,目的位址相同。P2 沒有相符的 SID。它照路由轉送封包,並把 hop limit 減一。
- 經過 P3 之後,目的位址是 fcbb:bb00:6::。P3 移除了它的節點指令和 uA,並把 PE6 移到前面。現在 tcpdump 回報校驗和錯誤。請記住這一點。
- P5 和 P4 像轉送一般位址一樣轉送這個封包。封包經過四台路由器後,以 hop limit 56 到達 PE6。每一次改變都和預測一致。

Ep 4.5 · The shifted ping from PE1

來源：lab-evidence/05-usid-shift/checksum (direct-PE1.log, shifted-PE1.log)

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:6:: source 2001:db8:100::1 count 3
Sending 3, 100-byte ICMP Echos to fcbb:bb00:6::, timeout is 2 seconds:
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/3/3 ms

RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:4:6:: source 2001:db8:100::1 count 3
Sending 3, 100-byte ICMP Echos to fcbb:bb00:2:4:6::, timeout is 2 seconds:
...
Success rate is 0 percent (0/3)
```

- 擷取顯示了一個奇怪的結果。移位後的封包到了 PE6,但 PE6 沒有回應。我們比較從 PE1 發出的兩個 ping。
- 第一個,直接 ping PE6 的 node SID。三個封包都有回應。
- 第二個,ping fcbb:bb00:2:4:6::。這個位址包含三個指令:P2、P4、再到 PE6。它沒有得到回應。原因在 PE6 上。

Ep 4.6 · PE6 counters after each ping

路由器 / 工具 : PE6 · 來源 : lab-evidence/05-usid-shift/checksum

```
RP/0/RP0/CPU0:PE6#show ipv6 traffic      (ICMP statistics, three times)
before the pings
  Rcvd: 23 input, 6 checksum errors, 0 too short
        13 echo request, 0 echo reply
after 3 direct pings
  Rcvd: 26 input, 6 checksum errors, 0 too short
        16 echo request, 0 echo reply
after 3 shifted pings
  Rcvd: 29 input, 9 checksum errors, 0 too short
        16 echo request, 0 echo reply
```

- 在 PE6 上,我們在 ping 之前以及每個 ping 之後,讀取 ICMP 統計。指令是 show ipv6 traffic。
- 這是起點:6 個校驗和錯誤,13 個 echo request。
- 三個直接的 ping 之後,echo request 計數增加三,變成 16。校驗和錯誤計數沒有改變。
- 三個移位的 ping 之後,校驗和錯誤計數增加三,變成 9。echo request 計數沒有改變。PE6 把這些封包當作損壞的封包丟棄了。

Ep 4.7 · Why PE6 discards the packet

路由器 / 工具 : analysis · 來源 : lab-evidence/05-usid-shift/checksum/checksum-analysis.txt

```
# ICMPv6 checksum, calculated again from the capture on P4 to PE6
direct: arriving DA fcbb:bb00:6::, checksum in packet 0xfd85
    computed with the arriving DA                0xfd85  match
    computed with the sender's DA fcbb:bb00:6::    0xfd85  match
shifted: arriving DA fcbb:bb00:6::, checksum in packet 0xfd34
    computed with the arriving DA                0xfd3a  MISMATCH
    computed with the sender's DA fcbb:bb00:2:4:6:: 0xfd34  match
```

- 我們從 P4 到 PE6 這條鏈路的擷取結果,重新計算了 ICMPv6 校驗和。
- 對於直接的 ping,封包到達時的位址和 PE1 送出時相同。校驗和相符。
- 對於移位的 ping,用到達時的目的地位址計算得到 0xfd3a。封包裡帶的是 0xfd34。兩者不相符。
- 用 PE1 送出時的目的地位址 fcbb:bb00:2:4:6:: 計算,得到 0xfd34。這個相符。PE1 是用它自己的目的地位址計算校驗和的。
- RFC 9800 第 6.5 節規定,來源必須用最終目的地位址來計算校驗和:也就是封包到達時的位址。我們的 ping 把容器當成了一般位址。
- 這不影響客戶流量,因為客戶流量的校驗和在內層封包裡。這個教訓是針對測試的:檢查移位路徑時,要用擷取結果,不要用 ping 的回應。

Ep 4.8 · PSP: the policy that PE1 uses

來源：lab-evidence/06-srh/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf BLUE ipv6 2001:db8:c:106::/64 detail
2001:db8:c:106::/64, version 22, SRv6 Headend
  via fcbb:bb00:1:e009::/128, 3 dependencies, recursive
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e005::}
      SRv6 H.Insert.Red SID-list {fcbb:bb01:3:5::}
Hash  OK  Interface          Address
0     Y   GigabitEthernet0/0/0/1  fe80::5454:ff:fecl:58eb
```

- 現在在實驗裡看 PSP。通往 BLUE 客戶的 IPv6 路由使用一條 SR policy。第 6 集會說明這條 policy。這裡我們只讀轉送項目。
- 第一個 SID list 是 PE6 的 service SID,fcbb:bb00:6:e005::。PE1 把它放進路由標頭。
- 第二個 SID list 是 policy。它是一個容器 fcbb:bb01:3:5::,有兩個指令:P3,然後 P5。PE1 把它放進目的位址。
- 封包需要這兩個清單,所以它需要路由標頭。現在我們從 CE-A 執行 ping,並在鏈路上擷取。

Ep 4.9 · PSP on the wire

路由器 / 工具 : tcpdump · 來源 : lab-evidence/06-srh/v3/c300-PE1-P3, c300-P3-P5, c300-P4-P5, c300-P4-PE6

```
# CE-A: sudo ping6 -c 2 2001:db8:c:106::2    (2 received, 0% packet loss)
# PE1 to P3, 182 bytes
IP6 (hlim 255, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:3:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P3 to P5, 182 bytes
IP6 (hlim 254, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P5 to P4, 158 bytes
IP6 (hlim 253, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
# P4 to PE6, 158 bytes
IP6 (hlim 252, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
```

- 在實驗裡,我們在路徑的鏈路上啟動擷取。然後在 CE-A 上,ping 遠端的客戶位址。兩個回應都收到了。
- 在 PE1 到 P3 的鏈路上,目的位址是容器 fcbb:bb01:3:5::。路由標頭有一個項目:PE6 的 service SID。Segments Left 是 1。訊框有 182 位元組。
- 經過 P3 之後,目的位址是 fcbb:bb01:5::。P3 把它自己的指令移出去了。路由標頭還在,有一個剩餘 segment。
- P5 是倒數第二個端點。它把最後一個 SID 複製到目的位址,並移除路由標頭。這就是 PSP。訊框變小了 24 位元組,成為 158 位元組。
- 在 P4 到 PE6 的鏈路上,沒有路由標頭。PE6 收到一個一般的封裝封包,其餘由 service SID e005 完成。

Ep 4.10 · uDT4: a service decapsulates by itself

路由器 / 工具 : PE6 · 來源 : lab-evidence/03-faults/v3wire/red-PE6.log

```
RP/0/RP0/CPU0:PE6#show cef ipv6 fcbb:bb00:6:e006::/64
fcbb:bb00:6:e006::/64, version 179, SRv6 Endpoint uDT4
  next hop VRF - 'RED', table - 0xe0000001
    0      Y      recursive          Lookup in table
```

- 不要把 USD 和服務解封裝混淆。
- uDT4 是另一種 behavior。它不需要 flavor 來移除外層標頭。移除外層標頭就是它的功能。
- 然後它在指定的 VRF 裡查詢內層封包,這裡是 RED。所以它是服務,而不是中途點。

Ep 5.1 · Test 1: ping and capture

來源：lab-evidence/05-usid-shift/v3/PE1.log and the first echo request in shift-*.pcap

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:4:6:: source 2001:db8:100::1 count 1
Type escape sequence to abort.
Sending 1, 100-byte ICMP Echos to fcbb:bb00:2:4:6::, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)

$ tcpdump -nn -t -v -r shift-PE1-P2.pcap icmp6
IP6 (hlim 60) 2001:db8:100::1 > fcbb:bb00:2:4:6::: [icmp6 sum ok] ICMP6, echo request
$ tcpdump -nn -t -v -r shift-P2-P4.pcap icmp6
IP6 (hlim 59) 2001:db8:100::1 > fcbb:bb00:4:6::: [bad icmp6 cksum 0xde68 -> 0xde6a!] ICMP6, echo request
$ tcpdump -nn -t -v -r shift-P4-PE6.pcap icmp6
IP6 (hlim 58) 2001:db8:100::1 > fcbb:bb00:6::: [bad icmp6 cksum 0xde68 -> 0xde6e!] ICMP6, echo request
```

- 現在我們執行第一個測試。在 CML 裡,在每一條核心鏈路上啟動封包擷取。然後從 PE1 ping 這個測試位址。
- 目的位址是 fcbb:bb00:2:4:6::。它包含三個指令:P2、P4,然後 PE6。
- 這個 ping 沒有回應。第 4 集已說明原因:校驗和。所以我們從擷取結果來讀路徑。
- 在 PE1 到 P2 的鏈路上,目的位址是完整的位址。hop limit 是 60。
- 在 P2 到 P4 的鏈路上,P2 已經移位了位址。目的位址是 fcbb:bb00:4:6::,hop limit 是 59。tcpdump 現在回報校驗和錯誤。
- 在 P4 到 PE6 的鏈路上,目的位址是 fcbb:bb00:6::,hop limit 是 58。八條鏈路都和這張表一致,其他鏈路都沒有這個封包。

Ep 5.2 · Why normal VPN traffic looks simpler

來源：lab-evidence/03-faults/v3wire/healthy2-PE1-P2.pcap

```
Healthy GREEN customer ping, captured on PE1 to P2:  
 154 bytes   outer destination fcbb:bb00:6:e008::   inner 10.30.1.1 to 10.30.6.6  
  
Shortest path PE1 to PE6:  PE1, P2, P4, PE6   (cost 30)
```

- 為什麼第 3 集的 VPN 流量沒有出現移位？
- 客戶封包只有一個指令:PE6 的 service SID。P2 和 P4 沒有事情要做。
- 最短路徑本來就經過 P2 和 P4,所以不需要指定它們。Headend 只有在要不同的路徑時才加入中途點。所以這一集,我們手動製作測試位址。

Ep 5.3 · Test 2: ping the waypoints

來源：lab-evidence/05-usid-shift/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:3:5:6:: source 2001:db8:100::1 count 1
Type escape sequence to abort.
Sending 1, 100-byte ICMP Echos to fcbb:bb00:3:5:6::, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)
```

- 測試二是 ping fcbb:bb00:3:5:6::。請先在八條核心鏈路上啟動擷取。
- 目的位址包含三個指令:P3、P5,然後 PE6。
- 又是沒有回應。我們從擷取結果讀路徑,而不是從 ping。

Ep 5.4 · The capture for fcbb:bb00:3:5:6::

來源：lab-evidence/05-usid-shift/v3/shift-*.pcap, one ping, read in time order

```
$ tcpdump -nn -t -v -r shift-<link>.pcap icmp6      (eight files, merged in time order)
link      destination      hop limit
PE1 - P2   fcbb:bb00:3:5:6::         60
P2 - P3     fcbb:bb00:3:5:6::         59      toward P3
P3 - P2     fcbb:bb00:5:6::          58      P3 shifted, sends it back
P2 - P4     fcbb:bb00:5:6::          57
P4 - P5     fcbb:bb00:5:6::          56      toward P5
P5 - P4     fcbb:bb00:6::           55      P5 shifted, sends it back
P4 - PE6    fcbb:bb00:6::           54

PE1 - P3, P3 - P5, P5 - PE6:  nothing
```

- 這是一次 ping。我們用 tcpdump 讀取每一條核心鏈路的擷取檔,並依時間順序排列結果。
- PE1 到 P2,再從 P2 到 P3,位址沒有改變。這裡 P2 沒有指令,所以只是照路由轉送。
- P3 移位,把封包送回 P2,再送往 P4。
- P4 把封包轉送給 P5。P5 移位,再把它送回 P4。
- P4 把它送到 PE6,hop limit 是 54。封包用了七條鏈路。最短路徑只有三條鏈路。
- 三條下方鏈路沒有任何封包。預測在每一條鏈路上都正確。

Ep 5.5 · A full container

來源：lab-evidence/05-usid-shift/v3/shift-*.pcap, one ping, read in time order

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:3:5:4:6:: source 2001:db8:100::1 count 1
Sending 1, 100-byte ICMP Echos to fcbb:bb00:2:3:5:4:6:0, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)

fcbb:bb00:0002:0003:0005:0004:0006:0000      five instructions, one zero group

link      destination          hop limit
PE1 - P2   fcbb:bb00:2:3:5:4:6:0  60
P2 - P3     fcbb:bb00:3:5:4:6::    59      P2 executed its own instruction
P3 - P2     fcbb:bb00:5:4:6::      58
P2 - P4     fcbb:bb00:5:4:6::      57
P4 - P5     fcbb:bb00:5:4:6::      56
P5 - P4     fcbb:bb00:4:6::        55
P4 - PE6    fcbb:bb00:6::          54
```

- 一個位址能包含多少個指令?我們來填滿一個位址。
- 測試三是 ping fcbb:bb00:2:3:5:4:6::。路由器顯示這個位址時,會帶上最後一組零,也就是 :0。這個 ping 和之前一樣沒有回應。
- 32 位元的 block 之後有六個 16 位元的位置。我們放入五個 node SID:P2、P3、P5、P4 和 PE6。一組零表示結尾。
- 這次 P2 是第一個。所以 P2 先執行它的指令並移位位址,再把封包送往 P3。
- 然後封包走和之前一樣的路徑,經過 P3、P2、P4、P5,再回到 P4。
- PE6 收到 fcbb:bb00:6::,hop limit 54。五個指令在同一個位址裡,而且沒有路由標頭。

Ep 5.6 · Test 4: ping with a service SID

來源：lab-evidence/05-usid-shift/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#ping fcbb:bb00:2:4:6:e008:: source 2001:db8:100::1 count 1
Type escape sequence to abort.
Sending 1, 100-byte ICMP Echos to fcbb:bb00:2:4:6:e008::, timeout is 2 seconds:
.
Success rate is 0 percent (0/1)
```

- 測試四把一個服務 function 放在清單的結尾。ping 的位址是 fcbb:bb00:2:4:6:e008::。
- 這個位址有四個指令:P2、P4、PE6,以及 PE6 的服務 function e008。
- 和其他測試一樣沒有回應。擷取結果會顯示移位的過程。

Ep 6.1 · Run the pings

路由器 / 工具 : PE1 and CE-A · 來源 : lab-evidence/06-srh/v3/green.log, CE-A.log

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

CE-A:~$ sudo ping6 -c 2 2001:db8:c:106::2
64 bytes from 2001:db8:c:106::2: seq=0 ttl=62 time=7.698 ms
2 packets transmitted, 2 packets received, 0% packet loss

CE-A:~$ sudo ping6 -c 2 -s 1000 2001:db8:c:106::2
1008 bytes from 2001:db8:c:106::2: seq=0 ttl=62 time=2.140 ms
2 packets transmitted, 2 packets received, 0% packet loss
```

- 這一集的訊框來自三個 ping。在 CML 裡,先在每一條核心鏈路上,以及 CE-A 和 PE1 之間的鏈路上啟動擷取。
- 第一個,從 PE1 ping GREEN。來源是 PE1 的 GREEN 介面。兩個封包都成功。
- 第二個,從主機 CE-A 對 BLUE 客戶的遠端 ping IPv6。預設的 ping 封包有 56 位元組的資料。
- 第三個,同樣的 ping,但資料是 1000 位元組。我們用這兩種大小來量測訊框大小如何改變。兩個 ping 都成功。

Ep 6.2 · Configure the policy on PE1

```
! PE1 (trimmed)
segment-routing
traffic-eng
segment-lists
  srv6
    sid-format usid-f3216
    segment-list TWO-BLOCKS
    srv6
      index 10 sid fcbb:bb01:3::
      index 20 sid fcbb:bb01:5::
      index 30 sid fcbb:bb00:6::
policy BLUE6-C300
  srv6
    binding-sid dynamic behavior ub6-insert-reduced locator MAIN
    color 300 end-point ipv6 2001:db8:100::6
  candidate-paths
  preference 100
  explicit segment-list TWO-BLOCKS
```

- 為了刻意製造情況二,我們在 PE1 上設定一條顯式 policy。第 15 集會說明每一個指令。
- 首先是一個 segment list,使用 micro-SID 格式 f3216。它的名稱是 TWO-BLOCKS。
- 前兩個 SID,P3 和 P5,來自 fcbb:bb01 這個 block。它們是 Flex-Algo SID。
- 最後一個 SID,PE6,來自 fcbb:bb00 這個 block。它是一般的 SID。
- policy BLUE6-C300 的 color 是 300,終點是 PE6。它的 Binding SID 是動態分配的,來自 locator MAIN。
- 唯一的 candidate path 使用 segment list TWO-BLOCKS。

Ep 6.3 · Color the BLUE IPv6 routes on PE6

```
! PE6 (trimmed)
extcommunity-set opaque COLOR-300
  300
end-set
route-policy BLUE6-EXPORT-C300
  set extcommunity color COLOR-300
end-policy
vrf BLUE
  address-family ipv6 unicast
    export route-policy BLUE6-EXPORT-C300
```

- policy 需要流量。PE6 對它通告的 BLUE IPv6 路由加上 color 300。PE1 再用 color 和終點去比對 policy。
- 首先是一個值為 300 的 extended community。
- 第二是一個設定這個 color 的 route policy。
- 第三,PE6 在匯出 BLUE IPv6 路由時套用這個 route policy。

Ep 6.4 · A policy that mixes two blocks

來源：lab-evidence/06-srh/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 300 detail
Color: 300, End-point: 2001:db8:100::6
  Admin: up   Operational: up
    Explicit: segment-list TWO-BLOCKS (valid)
      SID[0]: fcbb:bb01:3::/48
      SID[1]: fcbb:bb01:5::/48
      SID[2]: fcbb:bb00:6::/48
    Binding SID: fcbb:bb00:1:e009::
```

- 現在我們在 PE1 上檢查這條 policy。送往 PE6 的 BLUE IPv6 流量會進入這條 policy。
- 前兩個中途點 P3 和 P5,是來自 fcbb:bb01 block 的 Flex-Algo SID。
- 最後一個中途點 PE6,是來自 fcbb:bb00 的一般 SID。
- 這條 policy 是 up,並有一個 Binding SID。有兩個 block,所以至少要有兩個容器。

Ep 6.5 · What PE1 programs

來源：lab-evidence/06-srh/v3/PE1.log

```
RP/0/RP0/CPU0:PE1#show cef vrf BLUE ipv6 2001:db8:c:106::/64 detail
  SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e005::}
  SRv6 H.Insert.Red SID-list {fcbb:bb01:3:5::}
```

- 現在看 BLUE 目的位址的轉送項目。它有兩行重要的內容。
- 第一行:H.Encaps.Red,目標是 PE6 的 service SID e005。這是 VPN 的部分。PE1 封裝客戶封包,並送往 PE6 上的 BLUE 服務。
- 第二行:H.Insert.Red,帶著路徑容器 fcbb:bb01:3:5::。這是 policy 的部分。PE1 把路徑插在 service SID 前面。
- policy 的第三個 SID,也就是 PE6 的 node SID,沒有出現。service SID 本來就在 PE6 結束,所以路由器不使用它。

Ep 6.6 · The SRH, field by field

路由器 / 工具 : capture · 來源 : lab-evidence/06-srh/v3/srh-decode.txt, from c300-PE1-P3.pcap

```
$ tshark -r c300-PE1-P3.pcap -Y "ipv6.nxt == 43" -V      (first packet, trimmed)
Internet Protocol Version 6, Src: 2001:db8:100::1, Dst: fcbb:bb01:3:5::
  Payload Length: 128
  Next Header: Routing Header for IPv6 (43)
  Hop Limit: 255
  Routing Header for IPv6 (Segment Routing)
    Next Header: IPv6 (41)
    Length: 2
    [Length: 24 bytes]
    Type: Segment Routing (4)
    Segments Left: 1
    Last Entry: 0
    Address[0]: fcbb:bb00:6:e005::
Internet Protocol Version 6, Src: 2001:db8:c:101:5054:ff:fe0b:8f87, Dst: 2001:db8:c:106::2
```

- 現在我們解碼 PE1 到 P3 鏈路的擷取檔裡,其中一個 182 位元組的封包。我們使用 tshark 的 verbose 選項。
- 外層目的位址是路徑容器:Flex-Algo block 裡的 P3,然後 P5。Next header 43 表示後面跟著一個路由標頭。
- 這就是 SRH。Type 4 表示 Segment Routing。它的 next header 是 41,一個 IPv6 封包:客戶的封包。它的長度是 24 位元組。
- 它有一個項目:service SID。Segments Left 是 1。在目前的目的位址之後,清單裡還有一個 SID。
- 接著是客戶的 IPv6 標頭,沒有任何改變。

Ep 6.7 · Segments Left along the path

路由器 / 工具 : tcpdump · 來源 : lab-evidence/06-srh/v3/c300-PE1-P3, c300-P3-P5, c300-P4-P5, c300-P4-PE6

```
$ tcpdump -nn -t -v -r <capture file> 'ip6[6] == 43 or ip6[6] == 41'    (first packet, trimmed)
# PE1 to P3, 182 bytes
IP6 (hlim 255, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:3:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P3 to P5, 182 bytes
IP6 (hlim 254, next-header Routing (43)) 2001:db8:100::1 > fcbb:bb01:5::: RT6 (segleft=1, [0]fcbb:bb00:6:e005::)
# P5 to P4, 158 bytes
IP6 (hlim 253, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
# P4 to PE6, 158 bytes
IP6 (hlim 252, next-header IPv6 (41)) 2001:db8:100::1 > fcbb:bb00:6:e005:::
```

- 沿著路徑追蹤 Segments Left 欄位。我們用 tcpdump 讀取每一條鏈路的擷取檔。RT6 的部分就是路由標頭。
- 封包離開 PE1 時,目的位址是容器 fcbb:bb01:3:5::。Segments Left 是 1,項目是 PE6 的 service SID。
- 經過 P3 之後,目的位址已經移位成 fcbb:bb01:5::,Segments Left 仍然是 1。容器內的移位不會改變 SRH。
- 在 P5,容器裡已沒有 micro-SID。P5 把 Segments Left 減到零,並把下一個 SID 複製到目的位址。P5 是倒數第二個端點,所以它也移除了 SRH。
- next header 現在是 41,訊框小了 24 位元組。這就是第 4 集那個程序的第二步,出現在線路上。

Ep 6.8 · The BLUE packet, measured

路由器 / 工具 : tshark · 來源 : lab-evidence/06-srh/v3/c300-PE1-CE-A, c300-PE1-P3, c300-P3-P5, c300-P4-P5, c300-P4-PE6 (summary)

```
$ tshark -r <capture file> -Y "icmpv6.type == 128 and icmpv6.echo.sequence_number == 0" -T fields -e frame.len
link          small  large  what is added
CE-A to PE1   118    1062   nothing: the customer packet
PE1 to P3, P3 to P5  182    1126   + 40 outer IPv6 + 24 SRH = 64
P5 to P4, P4 to PE6  158    1102   + 40 outer IPv6 only
```

- 現在我們計算每一個位元組。對每一個擷取檔,tshark 會印出 echo request 的訊框長度。小 ping 和大 ping 各給一個值。
- 在接入鏈路上,訊框是 118 和 1062 位元組。這是客戶的封包。
- 從 PE1 到 P5,每個訊框都大了 64 位元組:外層 IPv6 標頭 40,SRH 24。
- 經過 P5 之後,每個訊框只大 40 位元組。P5 是倒數第二個端點,PSP 移除了 SRH,如第 4 集所示。
- 所以同一條流量的額外開銷沿路會改變:前段 64 位元組,後段 40 位元組。

Ep 6.9 · Why our lab never shows 1596

來源：lab-evidence/01-baseline/PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 manager
Headend behaviors:
  T
  H.Insert.Red
  H.Encaps.Red
  H.Encaps.L2.Red
Signaled parameters:
  Max-H-Insert      : 3 sids
  Max-H-Encap       : 3 sids
```

- 我們的擷取裡沒有 1596 位元組的封包。原因如下。
- 這個 headend 只有 reduced 形式。沒有一般的 H.Encaps,所以第一個 SID 從不會同時出現在 SRH 裡。在這個系列裡,1596 的情況只是計算。
- 它最多能插入或封裝 3 個 SID。每個 SID 都可以是裝滿 micro-SID 的完整容器。所以 3 限制的是容器數,不是指令數。

Ep 7.1 · Case 1: what PE1 programs

來源 : lab-evidence/07-mtu/v3/A-PE1-cef.log

```
RP/0/RP0/CPU0:PE1#show cef vrf RED 10.2.6.0/24 detail | include SID-list
SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e006::}
SRv6 H.Insert.Red SID-list {fcbb:bb00:e004::}
```

- 情況一使用 VRF RED 裡的 IPv4。首先,我們檢查 PE1 為 RED 的 prefix 10.2.6.0/24 設定了什麼。
- service SID 是 fcbb:bb00:6:e006::。它是 PE6 上 RED 的 uDT4。
- 路徑是一個容器 fcbb:bb00:e004::。PE1 把它插在 service SID 前面。
- 每個清單都只有一個 SID,所以封包沒有路由標頭。額外開銷是 40 位元組,上限是 1460。

Ep 7.2 · Case 1: the pings

路由器 / 工具 : CE-A · 來源 : lab-evidence/07-mtu/v3/A-CE-A.log, A2-CE-A.log

```
CE-A:~$ sudo ping -c 1 -I 10.2.1.2 -s 1432 10.2.6.2
1440 bytes from 10.2.6.2: seq=0 ttl=62 time=1.495 ms
1 packets transmitted, 1 packets received, 0% packet loss

CE-A:~$ sudo ping -c 3 -I 10.2.1.2 -s 1433 10.2.6.2
3 packets transmitted, 0 packets received, 100% packet loss

CE-A:~$ sudo cat /proc/sys/net/ipv4/ip_no_pmtu_disc
0
```

- 現在 CE-A 對 RED 的遠端送出 IPv4 ping。選項 -s 是資料大小,-I 設定來源位址。
- 資料大小 1432 會產生一個 1460 位元組的 IPv4 封包。這正好是上限,回覆有回來。
- 資料大小 1433 會產生一個 1461 位元組的封包。三個 ping 全部遺失。封包遺失率是 100%。
- Linux 主機會設定 don't-fragment 位元,因為這個核心設定是 0。

Ep 7.3 · Case 1: the captures

路由器 / 工具 : tshark · 來源 : lab-evidence/07-mtu/v3/A-PE1-Gi0_0_0_3-CE-A, A-PE1-P3, A2-*

```
$ tshark -r <capture file> -Y icmp
# PE1 to CE-A, the 1460-byte ping
1474 bytes  10.2.1.2 > 10.2.6.2  echo request
1474 bytes  10.2.6.2 > 10.2.1.2  echo reply
# PE1 to CE-A, the 1461-byte ping
1475 bytes  10.2.1.2 > 10.2.6.2  echo request    (no reply follows)
# PE1 to P3, the 1460-byte ping
1514 bytes  10.2.1.2 > 10.2.6.2  echo request, inside the outer IPv6 header
# PE1 to P2, PE1 to P3, P4 to PE6, P5 to PE6, the 1461-byte ping
no echo request on any core link
```

- 我們在接入鏈路和核心鏈路上擷取,並用 tshark 讀取每個檔案。訊框長度包含 14 位元組的乙太網路標頭。
- 在接入鏈路上,1460 位元組的封包是 1474 位元組的訊框。回覆有回來。
- 1461 位元組的封包是 1475 位元組的訊框。它到達 PE1,之後沒有回覆。
- 1460 位元組的封包用 1514 位元組的訊框穿越核心。這是鏈路能承載的最大訊框。
- 對於 1461 位元組的封包,沒有任何核心鏈路承載 echo request。PE1 在封裝之前就把它丟棄了。

Ep 7.4 · Case 2: the pings

路由器 / 工具 : CE-A · 來源 : lab-evidence/07-mtu/v3/B-CE-A.log

```
CE-A:~$ sudo ping6 -c 1 -s 1388 2001:db8:c:106::2
1396 bytes from 2001:db8:c:106::2: seq=0 ttl=62 time=1.545 ms
1 packets transmitted, 1 packets received, 0% packet loss

CE-A:~$ sudo ping6 -c 2 -s 1389 2001:db8:c:106::2
1397 bytes from 2001:db8:c:106::2: seq=1 ttl=62 time=1.566 ms
2 packets transmitted, 1 packets received, 50% packet loss
```

- 情況二使用 VRF BLUE 裡的 IPv6,經過 colour 300。上限是 1436。Linux 的 ping6 會在資料大小之外加 48 位元組。
- 資料大小 1388 會產生一個 1436 位元組的封包。回覆有回來。
- 資料大小 1389 會產生一個 1437 位元組的封包。第一個封包遺失,第二個封包有回覆。遺失率是 50%。

Ep 7.5 · Case 2: the captures

路由器 / 工具 : tshark · 來源 : lab-evidence/07-mtu/v3/B-PE1-CE-A.pcap

```
$ tshark -r B-PE1-CE-A.pcap -Y icmpv6          (CE-A is 2001:db8:c:101:5054:ff:fe0b:8f87)
1450 bytes  CE-A > 2001:db8:c:106::2  echo request, seq 0
1450 bytes  2001:db8:c:106::2 > CE-A  echo reply, seq 0
1451 bytes  CE-A > 2001:db8:c:106::2  echo request, the 1437-byte packet
1294 bytes  2001:db8:100::1 > CE-A    Packet Too Big, MTU 1436
1446 and 75 bytes  CE-A > 2001:db8:c:106::2  the second request, in two fragments
1451 bytes  2001:db8:c:106::2 > CE-A  echo reply, seq 1
```

- 在接入鏈路上,我們用 tshark 讀取封包。CE-A 就是第一行顯示的位址。
- 1436 位元組的封包是 1450 位元組的訊框。請求和回覆都通過。
- 1437 位元組的封包是 1451 位元組的訊框。這是第一次嘗試。
- PE1 回應一個 ICMPv6 Packet Too Big。來源是 PE1 的 loopback。MTU 是 1436,正好就是我們的預算。
- CE-A 把第二個請求分成兩個分片送出,回覆有回來。IPv6 的路徑 MTU 探索成功了。

Ep 7.6 · Case 2: the long way home

路由器 / 工具 : tshark · 來源 : lab-evidence/07-mtu/v3/B-PE1-P2.pcap, B-P4-PE6.pcap, B-PE1-CE-A.pcap

```
$ tshark -r <capture file> -Y icmpv6.type==2      (outer header, trimmed)
# PE1 to P2, 1358 bytes
2001:db8:100::1 > fcb:b00:6:e005::      hop limit 255      Packet Too Big, toward PE6
# P4 to PE6, 1358 bytes
2001:db8:100::1 > fcb:b00:6:e005::      hop limit 253      the same message
# P4 to PE6, 1334 bytes
2001:db8:100::6 > fcb:b00:1:e006::      hop limit 255      the message, back toward PE1
# PE1 to P2, 1334 bytes
2001:db8:100::6 > fcb:b00:1:e006::      hop limit 253      the same message
# PE1 to CE-A, 1294 bytes
2001:db8:100::1 > CE-A      Packet Too Big, delivered
```

- 現在看看這個 Packet Too Big 走的路徑。PE1 是為自己接入鏈路上的一台主機產生它的。你會預期它直接送到主機。結果不是。
- PE1 把它封裝並送往 PE6 的 BLUE service SID,它從 PE1 往 P2 離開。
- 同一個訊息穿越核心,到達 P4 到 PE6 的鏈路。hop limit 是 253。
- PE6 解封裝,在 BLUE 裡查詢 CE-A 的位址,再把它送回 PE1 的 service SID。
- 它再次穿越核心,到達 PE1 到 P2 的鏈路。
- 然後 PE1 才把它送給 CE-A。它成功了,但穿越了核心兩次。我們沒有驗證入口為什麼這樣產生訊息。這可能和平台有關。

Ep 7.7 · Case 3: a smaller link in the middle

```
! P4, the link toward P2
interface GigabitEthernet0/0/0/0
  ipv6 mtu 1300
!
! RED traffic from CE-B to CE-A crosses P4 to P2
! inner 1260 + outer 40 = 1300    fits
! inner 1261 + outer 40 = 1301    does not fit
```

- 情況三。入口沒有問題,但中間有一條鏈路太小。我們在 P4 上把通往 P2 的 IPv6 MTU 設為 1300。
- 這次流量走相反方向,在 RED 裡從 CE-B 到 CE-A。這個方向走經過 P4 和 P2 的上方路徑。
- 1260 位元組的內層封包變成 1300 位元組。放得下。
- 在 1261 位元組時,它放不下了,P4 必須決定怎麼處理。P4 是 P 路由器。它沒有 VRF,也不知道 CE-B。

Ep 7.8 · Case 3: the pings

路由器 / 工具 : CE-B · 來源 : lab-evidence/07-mtu/v3/C-CE-B.log

```
CE-B:~$ sudo ping -c 1 -I 10.2.6.2 -s 1232 10.2.1.2
1240 bytes from 10.2.1.2: seq=0 ttl=62 time=1.796 ms
1 packets transmitted, 1 packets received, 0% packet loss

CE-B:~$ sudo ping -c 2 -I 10.2.6.2 -s 1233 10.2.1.2
1241 bytes from 10.2.1.2: seq=1 ttl=62 time=1.558 ms
2 packets transmitted, 1 packets received, 50% packet loss
```

- 現在 CE-B 在 RED 裡對 CE-A 送出 IPv4 ping。這個方向會經過 P4 和 P2。
- 資料大小 1232 會產生一個 1260 位元組的封包。放得下,回覆有回來。
- 資料大小 1233 會產生一個 1261 位元組的封包。第一個封包遺失,第二個封包有回覆。遺失率是 50%。

Ep 7.9 · Case 3: the captures

路由器 / 工具 : tshark · 來源 : lab-evidence/07-mtu/v3/C-PE6-Gi0_0_0_3-CE-B, C-P2-P4, C-PE1-P2, C-P4-PE6

```
$ tshark -r C-PE6-Gi0_0_0_3-CE-B.pcap -Y icmp          (PE6 to CE-B)
1275 bytes  10.2.6.2 > 10.2.1.2  echo request, the 1261-byte packet
110 bytes   10.0.0.4 > 10.2.6.2  destination unreachable, fragmentation needed, MTU 1236
1250 and 59 bytes  10.2.6.2 > 10.2.1.2  the second request, in two fragments
1275 bytes  10.2.1.2 > 10.2.6.2  echo reply

$ tshark -r <capture file> -Y icmp.type==3          (the error in the core, outer header)
P2 to P4    190 bytes  2001:db8:100::6 > fcbb:bb00:1:e005::  hop limit 254
PE1 to P2   190 bytes  2001:db8:100::6 > fcbb:bb00:1:e005::  hop limit 253
P4 to PE6   150 bytes  2001:db8:100::1 > fcbb:bb00:6:e006::  hop limit 252
```

- 我們在五條鏈路上擷取,並用 tshark 讀取這些檔案。首先是 PE6 上,CE-B 的接入鏈路。
- 1261 位元組的請求從 CE-B 送出。這是第一次嘗試。
- 一個 ICMPv4 錯誤回到 CE-B:fragmentation needed,MTU 是 1236。來源是 10.0.0.4,也就是 P4 的 IPv4 loopback。
- 現在看核心。P4 把錯誤訊息放在和被丟棄的封包相同的外層標頭裡送出。它從 P4 經過 P2,往 PE1 的 service SID 前進。
- PE1 解封裝,在 RED 裡找到 CE-B,再把它穿越核心送往 PE6 的 service SID。
- CE-B 學到這個 MTU,把第二個請求分成 1250 和 59 位元組的兩個分片送出。回覆有回來。

Ep 8.1 · Where the LSP content comes from

```
! PE6 (trimmed)
router isis CORE
 address-family ipv6 unicast
  metric-style wide
  segment-routing srv6
   locator MAIN
   locator FA128
 flex-algo 128
  metric-type delay
  advertise-definition
interface GigabitEthernet0/0/0/0
 point-to-point
 address-family ipv6 unicast
  fast-reroute per-prefix ti-lfa
  metric 10
performance-measurement
interface GigabitEthernet0/0/0/0
 delay-measurement
  advertise-delay 5000
```

- 在讀 LSP 之前,先看產生它的設定。這是 PE6,只保留重要的幾行。你可以在每台路由器上複製同樣的結構。
- Wide metric 是 Segment Routing 所必需的。IPv6 address family 承載所有內容。
- 兩個 locator 掛到 IS-IS 上:MAIN 屬於 algorithm 0,FA128 屬於 Flex-Algo 128。每一個都會成為 LSP 裡的一個 locator。
- flex-algo 128 區塊指定 metric 類型為 delay,並通告它的定義。我們在第 11 集使用它。
- 每個核心介面都是點對點,並帶有 metric。這個 metric 就是我們在 LSP 裡讀到的鏈路 metric。
- 5000 微秒的鏈路延遲來自 performance-measurement 區塊。它是我們設定的靜態值,不是量測值。

Ep 8.2 · Layer 1: the adjacency

來源：lab-evidence/08-isis/rerecord/PE1-healthy.log

```
RP/0/RP0/CPU0:PE1#show isis adjacency detail
P2                Gi0/0/0/0                *PtoP*                Up      25    06:50:33 Yes None Up
Neighbor IPv6 Address: fe80::5454:ff:fe66:75e1*
END.X SID: fcb::bb00:1:e000::/64 uA (PSP/USD), Algorithm 0
Non-FRR END.X SID: fcb::bb00:1:e001::/64 uA (PSP/USD), Algorithm 0
END.X SID: fcb::bb01:1:e000::/64 uA (PSP/USD), Algorithm 128
Non-FRR END.X SID: fcb::bb01:1:e001::/64 uA (PSP/USD), Algorithm 128
Topology: IPv6 Unicast
Usable by Flex-algo: 128
BFD Status: Neighbor Useable (BFD Required)
```

- 第一層,在 PE1 上,看鄰居 P2。
- 鄰接是 up。它使用 IPv6 link-local 位址。這裡沒有 IPv4。
- IS-IS 在 algorithm 0 裡為這條鏈路分配了兩個 adjacency SID。快速重路由可以保護其中一個。帶有 Non-FRR 標記的 SID 永遠不受保護。
- algorithm 128 還有兩個,也就是第 11 集的 Flex-Algo。一條鏈路有四個 adjacency SID。
- 拓撲是 IPv6 unicast, Flex-Algo 128 可以使用它。BFD 是必要的,而且是 up。這裡的每一行,都可能是某條路徑不見的原因。

Ep 8.3 · Layer 2: the LSP, capabilities

來源：lab-evidence/08-isis/rerecord/PE1-healthy.log

```
RP/0/RP0/CPU0:PE1#show isis database verbose PE6.00-00
PE6.00-00          0x00000034   0x359c          1145 /1200          0/0/0
  IPv6 Router ID: 2001:db8:100::6
  Router Cap:      0.0.0.0 D:0 S:0
    SR Algorithm:
      Algorithm: 0
      Algorithm: 1
      Algorithm: 128
    SRv6: O:0
  Node Maximum SID Depth:
    SRH Max SL:      3
    SRH Max End Pop:  3
    SRH Max T.insert: 3
    SRH Max T.encaps: 3
    SRH Max End D:    4
```

- 第二層是 PE1 上 PE6 的 LSP。我們分三段讀。請使用 verbose。detail 選項不會顯示大部分的資訊。
- 這是 PE6 的 LSP,帶有序號、剩餘存活時間和一個 IPv6 router ID。
- PE6 列出它使用的演算法:預設的 0 和 1,以及我們設定的 128。
- RFC 9352 定義了這個 SRv6 能力旗標。它表示 PE6 能處理 SRv6。
- 這些是最大 SID 深度。它們和第 2 集 manager 顯示的數值相同。每台路由器都從這裡得知其他路由器的限制。

Ep 8.4 · Layer 2: the LSP, locators

來源 : lab-evidence/08-isis/rerecord/PE1-healthy.log

```
SRv6 Locator:  MT (IPv6 Unicast) fcbb:bb00:6::/48 D:0 Metric: 1 Algorithm: 0
  END SID: fcbb:bb00:6:: uN (PSP/USD)
  SID Structure:
    Block Length: 32, Node-ID Length: 16, Func-Length: 0, Args-Length: 80
SRv6 Locator:  MT (IPv6 Unicast) fcbb:bb01:6::/48 D:0 Metric: 1 Algorithm: 128
  END SID: fcbb:bb01:6:: uN (PSP/USD)

Metric: 1          MT (IPv6 Unicast) IPv6 fcbb:bb00:6::/48
```

- 第二段:locator。
- SRv6 locator TLV 包含 fcbb:bb00:6::/48,屬於 algorithm 0,metric 是 1。這就是第 2 集 metric 31 裡多出來的 1。
- 它下面是 End SID,也就是 PE6 的 uN,以及它的結構:block 32、node 16、function 0、argument 80。這 80 位元的 argument,就是留給後面 micro-SID 的空間。
- 這是第二個 locator,屬於 algorithm 128,用於 Flex-Algo。
- 同一個 locator 又以一般 IPv6 prefix 的形式出現一次。RFC 9352 允許這樣做。這樣,沒有 SRv6 的路由器也能路由到這個 locator。

Ep 8.5 · Loopback and locator, side by side

來源：lab-evidence/08-isis/rerecord/PE1-healthy.log

```
Metric: 0          MT (IPv6 Unicast) IPv6 2001:db8:100::6/128
  Prefix Attribute Flags: X:0 R:0 N:1 E:0 A:0
  Source Router ID: 2001:db8:100::6
Metric: 1          MT (IPv6 Unicast) IPv6 fcbb:bb00:6::/48
  Prefix Attribute Flags: X:0 R:0 N:0 E:0 A:0
```

- 在 LSP 裡,比較 PE6 最重要的兩個 prefix。第 3 集的第一個故障就在這裡。
- Loopback 的 metric 是 0,並有 N 旗標。N 旗標表示這個 prefix 代表這個節點。
- Locator 的 metric 是 1,它是另一個 prefix。這是同一個 LSP 裡的兩行,來自設定中不同的部分。其中一個可能消失,而另一個仍然在。

Ep 8.6 · Layer 2: the LSP, links

來源：lab-evidence/08-isis/rerecord/PE1-healthy.log

```
Metric: 10          MT (IPv6 Unicast) IS-Extended P4.00
Interface IPv6 Address: 2001:db8:200:46::1
Neighbor IPv6 Address: 2001:db8:200:46::
Link Average Delay: 5000 us
END.X SID: fcb:b01:6:e000:: B:1 S:0 P:0 uA (PSP/USD) Alg:128
END.X SID: fcb:b01:6:e001:: B:0 S:0 P:0 uA (PSP/USD) Alg:128
END.X SID: fcb:b00:6:e000:: B:1 S:0 P:0 uA (PSP/USD) Alg:0
END.X SID: fcb:b00:6:e001:: B:0 S:0 P:0 uA (PSP/USD) Alg:0
SID Structure:
  Block Length: 32, Node-ID Length: 16, Func-Length: 16, Args-Length: 64
```

- 第三段:每一條鏈路。這是 PE6 通往 P4 的鏈路。
- metric 是 10,附上兩端的 IPv6 位址。
- 鏈路延遲是 5000 微秒。這是我們為 Flex-Algo 設定的靜態值。它不是量測值。第 11 集會用到它。
- 有四個 End.X SID,每個演算法兩個。B 旗標標示可以受保護的 SID。
- 它們的結構是 block 32、node 16、function 16、argument 64。我們最初的規劃預期 argument 是零。路由器顯示 64,而路由器是對的。

Ep 8.7 · Layers 3, 4 and 5

來源 : lab-evidence/08-isis/rerecord/PE1-healthy.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [31/115]
   via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48 detail
   Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
   fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:6::/48
   via fe80::5454:ff:fe66:75e1/128, GigabitEthernet0/0/0/0
```

- 現在看同一個 locator 在 PE1 上的最後三層。
- 第三層:IS-IS 算出了這條路由,metric 31,經過 P2。
- 第四層:RIB 安裝了它。它把這條路由顯示為 SRv6 locator、受保護。第 9 集會說明受保護這個詞。
- 第五層:CEF 從 GigabitEthernet 0/0/0/0 轉送出去。所有的層都顯示同一個下一跳。當各層不一致時,你就找到故障了。

Ep 8.8 · Fault 1: the interface leaves the IPv6 topology

```
! PE1
router isis CORE
 interface GigabitEthernet0/0/0/0
  no address-family ipv6 unicast
```

- 現在我們弄壞網路,一次一層。故障一:在 PE1 上,我們把通往 P2 的 IS-IS 介面的 IPv6 address family 移除。
- 介面仍然是 up,位址也都還在。只是 IS-IS 不再在它上面使用 IPv6。在下一頁之前,先預測結果。

Ep 8.9 · Fault 1: what changed

來源：lab-evidence/08-isis/rerecord/F1-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis neighbors
P3                Gi0/0/0/1          *PtoP*          Up      22      L2      Capable

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 71, SRv6-locator, type level-2

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:6:: source 2001:db8:100::1
 1  2001:db8:200:13::1
 2  2001:db8:200:23::
 3  2001:db8:200:24::1
 4  2001:db8:100::6

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms
```

- 這是二十秒後的 PE1。
- 通往 P2 的鄰接消失了。當 IS-IS 只執行 IPv6 時,IPv6 拓撲是保留鄰居的唯一理由。
- PE6 的 locator 現在的 metric 是 71,而不是 31。
- traceroute 顯示了新路徑:走昂貴的鏈路到 P3,折回 P2,然後經過 P4 到 PE6。每個封包現在多走兩跳。
- 客戶的 ping 仍然是通的。沒有人抱怨。這種故障很安靜,所以很危險。我們把 address family 加回去之後,P2 回來了,metric 也恢復成 31。

Ep 8.10 · Fault 2: what PE1 shows

來源 : lab-evidence/03-faults/v3/A-PE1.log

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    SID Locator: fcbb:bb00:6::/48, RIB metric: inaccessible

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
    unresolved
    0      Y      recursive          drop

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 0 percent (0/3)
```

- 你在第 3 集看過故障二:我們在 PE6 上把 locator 從 IS-IS 移除。這裡是 PE1 上的輸出,放在分層模型裡看。
- 第四層:RIB 裡沒有到 locator 的路由。
- BGP 仍然有客戶路徑,但 locator 是 inaccessible。
- 第五層:CEF 是 unresolved,最後一欄顯示 drop。
- 客戶的 ping 失敗了。

Ep 8.11 · Fault 2: layers 1 to 5 on PE1

來源：lab-evidence/08-isis/fault2/before-PE1.log, after-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis adjacency
P2          Gi0/0/0/0      *PtoP*    Up
P3          Gi0/0/0/1      *PtoP*    Up

RP/0/RP0/CPU0:PE1#show isis database verbose PE6.00-00 | include Locator
  SRv6 Locator:   MT (IPv6 Unicast) fcbb:bb01:6::/48 D:0 Metric: 1 Algorithm: 128
! before the fault: a second line, fcbb:bb00:6::/48, Algorithm: 0

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48
The requested route fcbb:bb00:6::/48 was not found in the IS-IS CORE IPv6 Unicast Routing Table

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table
```

- 現在我們讀故障二的全部五層。在 PE6 上,locator MAIN 從 IS-IS 移除。
- 第一層:鄰接是 up。PE1 仍然有鄰居 P2 和 P3。
- 第二層:PE6 的 LSP 失去了 locator。只剩下 Flex-Algo 的 locator。故障之前,LSP 裡還有 algorithm 0 的 fcbb:bb00:6::/48。
- 第三層:SPF 沒有 prefix 可以計算。IS-IS 路由表裡沒有這條路由。
- 第四層:RIB 裡沒有路由。接著第五層,CEF 丟棄服務,就像上一頁顯示的那樣。
- 故障從第二層開始,發生在遠端路由器上。把 locator 加回去之後,路由以 metric 31 回來,ping 也成功了。

Ep 8.12 · Fault 3: a filter on the way into the RIB

```
! PE1
ipv6 prefix-list BLOCK-PE6-LOCATOR
 10 deny fcbb:bb00:6::/48
 20 permit ::/0 le 128
!
router isis CORE
 address-family ipv6 unicast
  distribute-list prefix-list BLOCK-PE6-LOCATOR in
```

- 故障三最難發現。我們在 PE1 上,於 IS-IS 和 RIB 之間加一個過濾器。
- 它是一個 prefix list。它只拒絕 PE6 的 locator,允許其他所有 prefix。
- 我們用 distribute-list in 把它套用在 IPv6 address family 下。它過濾 IS-IS 要安裝的路由。它不會阻止 LSP 的泛洪。

Ep 8.13 · Fault 3: the layers disagree

來源 : lab-evidence/08-isis/rerecord/F3-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis database verbose PE6.00-00 | include Locator
  SRv6 Locator:   MT (IPv6 Unicast) fcbb:bb00:6::/48 D:0 Metric: 1 Algorithm: 0

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48
L2 fcbb:bb00:6::/48 [31/115]

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
% Network not in table

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
  valid, internal, best, group-best
  SID Locator: fcbb:bb00:6::/48, RIB metric: inaccessible

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 0 percent (0/2)
```

- 現在在 PE1 上,由上往下讀這五層。
- 第二層:PE6 的 locator 在 LSP 裡。過濾器沒有改變泛洪。
- 第三層:IS-IS 仍然算出這條路由,metric 31。
- 第四層:RIB 裡沒有它。Network not in table。
- BGP 仍把客戶路徑保持為 best,locator 則是 inaccessible,和第 3 集一樣。
- 客戶沒有服務。第二、三層說有,第四層說沒有。這種差異表示有一個過濾器。

Ep 9.1 · Link P2 to P4, repaired by P2

路由器 / 工具 : P2 · 來源 : lab-evidence/09-tilfa/v3/link-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
  Backup path: TI-LFA (link), via ..., GigabitEthernet0/0/0/2 P3, Metric: 71
    P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
    Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)

RP/0/RP0/CPU0:P2#show cef ipv6 fcbb:bb00:6::/48 detail
  next hop ..., Repair Node(s): 2001:db8:100::5
  SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}
```

- 現在看路由器。首先,在沒有節點保護的情況下,看我們畫的那個例子。
- P2 有一條經過 P3 的 TI-LFA 備援。修復後的 metric 是 71。
- P node 是 P3,Q node 是 P5,adjacency SID e005 從 P3 通往 P5。這和我們的預測一致。
- 在 CEF 裡,修復是 H.Insert.Red,使用一個容器 fcbb:bb00:3:e005::。P3 的 node SID 和它通往 P5 的鄰接放在同一個位址裡。
- 在這個平台上,修復是把一個路由標頭插入封包。它不會加第二個外層標頭。

Ep 9.2 · Link P4 to PE6, and link PE1 to P2

路由器 / 工具 : P4 and PE1 · 來源 : lab-evidence/09-tilfa/v3/link-P4.log, link-PE1.log

```
RP/0/RP0/CPU0:P4#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
  Backup path: TI-LFA (link), via ..., GigabitEthernet0/0/0/2 P5, Metric: 51
    P node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:5:: uN (PSP/USD)
    Q node: PE6.00 [2001:db8:100::6], SRv6 SID: fcbb:bb00:5:e001:: uA (PSP/USD)
  SRv6 H.Insert.Red SID-list {fcbb:bb00:5:e001::}

RP/0/RP0/CPU0:PE1#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
  Backup path: LFA, via ..., GigabitEthernet0/0/0/1, P3, Metric: 71
```

- 再看兩個鏈路的例子。
- P4 保護它通往 PE6 的鏈路:先到 P5,再使用 P5 到 PE6 的鄰接,放在一個容器裡。metric 是 51。這和預測一致。
- PE1 保護它通往 P2 的鏈路。備援是經過 P3 的一般 LFA,沒有任何 SID。不是每個故障都需要 segment list。這個例子就證明了這一點。

Ep 9.3 · Asking for node protection

```
router isis CORE
 address-family ipv6 unicast
  fast-reroute per-prefix tiebreaker node-protecting index 100
```

- 鏈路保護假設只有鏈路故障。如果整台鄰居路由器故障,鏈路保護的備援可能會把流量送回那台路由器。
- 在這個版本上,我們必須用 tiebreaker 要求節點保護。我們在六台路由器上都加了它。

Ep 9.4 · Node P2, repaired by PE1

來源：lab-evidence/09-tilfa/v3/node-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 fast-reroute fcb::bb00:6::/48 detail
  Backup path: TI-LFA (node), via ..., GigabitEthernet0/0/0/1 P3, Metric: 101
    P node: P3.00 [2001:db8:100::3], SRv6 SID: fcb::bb00:3:: uN (PSP/USD)
    Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcb::bb00:3:e005:: uA (PSP/USD)
  SRv6 H.Insert.Red SID-list {fcb::bb00:3:e005::}
```

- 現在看節點的例子。首先,看 PE1 眼中 P2 故障的情況。
- PE1 現在有一條經過 P3 的節點保護備援,metric 101。
- 清單是一個容器:先到 P3,再使用通往 P5 的鄰接。它比我們手算的三段還短。
- 為什麼?P3 是 PE1 的直接鄰居,所以第一跳不需要 adjacency SID;出口介面就夠了。從 P5 開始,到 PE6 的正常路徑經過 P4,而不經過 P2。所以 PE6 不需要一段。
- 路由器找到了最短的清單。我們的手算是對的,但不是最短的。核對你的預測時,請記住這個差別。

Ep 9.5 · Node P4, repaired by P2

路由器 / 工具 : P2 · 來源 : lab-evidence/09-tilfa/v3/node-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 fast-reroute fcbb:bb00:6::/48 detail
Backup path: TI-LFA (node), via ..., GigabitEthernet0/0/0/2 P3, Metric: 91
  P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
  Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)
  Q node: PE6.00 [2001:db8:100::6], SRv6 SID: fcbb:bb00:5:e001:: uA (PSP/USD)
SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005:e001::}
```

- 現在看 P2 眼中 P4 故障的情況。
- 有一條經過 P3 的節點保護備援,metric 91。
- 修復有三個部分:到 P3、使用 P3 到 P5 的鄰接,再使用 P5 到 PE6 的鄰接。這和手算一致。
- 看看編碼:fcbb:bb00:3:e005:e001::。第二個鄰接 e001 是 P5 的鄰接。它前面沒有 P5 的 node SID。P3 移位位址之後,P5 就把 e001 認作自己的本地 SID。
- 三個指令放在一個容器裡,而且沒有路由標頭。

Ep 9.6 · When node protection is impossible

路由器 / 工具 : P4 · 來源 : lab-evidence/09-tilfa/v3/node-P4.log

```
RP/0/RP0/CPU0:P4#show isis ipv6 fast-reroute fcb::bb00:6::/48 detail
  Backup path: TI-LFA (link), via ..., GigabitEthernet0/0/0/2 P5, Metric: 51
  TI-LFA Path Fallback Information:
    TI-LFA link: Path valid
    TI-LFA node: Path not found
```

- 還有一個預測裡沒有的例子。在要求節點保護的情況下,P4 保護送往 PE6 的流量。
- P4 仍然使用鏈路保護的備援。
- P4 給出了原因:它找不到節點保護的路徑。目的地 PE6 就是那個鄰居。你無法在某台路由器故障時,保護送往那台路由器的流量。

Ep 9.7 · How much is protected

路由器 / 工具 : P2 · 來源 : lab-evidence/09-tilfa/v3/node-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 fast-reroute summary
IS-IS CORE IPv6 Unicast FRR summary
```

	Critical Priority	High Priority	Medium Priority	Low Priority	Total
Prefixes reachable in L2					
All paths protected	0	0	5	10	15
Some paths protected	0	0	0	0	0
Unprotected	0	0	0	0	0
Protection coverage	0.00%	0.00%	100.00%	100.00%	100.00%

- 有一個指令可以顯示是否有 prefix 沒有受到保護。每次變更之後,都在每台路由器上使用它。
- 在 P2 上,15 個 IPv6 prefix 全都有受保護的路徑。
- 沒有任何 prefix 未受保護。
- 覆蓋率是 100%。之前,當兩條鏈路在沒有警告的情況下變成單向時,P4 在這裡只顯示 87%。當這個數字下降時,拓撲往往和你想的不一樣。

Ep 9.8 · Failure test: the ping and the shutdown

路由器 / 工具 : PE1 and P4 · 來源 : lab-evidence/09-tilfa/v3/fail-ping.log, fail-P4-shut.log

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 4000 timeout 1
Sending 4000, 100-byte ICMP Echos to 10.30.6.6 timeout is 1 seconds:

# while the ping runs, on P4:
RP/0/RP0/CPU0:P4(config)#interface GigabitEthernet0/0/0/0 shutdown
RP/0/RP0/CPU0:P4(config)#commit

# when the ping ends, on PE1:
Success rate is 100 percent (4000/4000), round-trip min/avg/max = 1/2/13 ms
```

- 現在是主要的測試。我們在流量進行時製造一個故障。首先,在 CML 裡,我們在核心鏈路上啟動擷取。
- 然後在 PE1 上,我們開始一個很長的 GREEN ping:4000 個封包,逾時 1 秒。
- ping 進行時,我們把 P4 通往 P2 的介面關掉。這就是故障。P2 是本地修復點。
- ping 結束時,4000 個封包全都有回應。接下來看路由器記錄了什麼。

Ep 9.9 · Failure test: what the routers saw

路由器 / 工具 : P2 and PE1 · 來源 : lab-evidence/09-tilfa/v3/fail-P2-log.log, fail-PE1-after.log

```
RP/0/RP0/CPU0:P2#show logging last 12
%L2-BFD-6-SESSION_STATE_DOWN : BFD session to neighbor ... has gone down. Reason: Nbor signalled down
%ROUTING-ISIS-5-ADJCHANGE : ISIS (CORE): Adjacency to P4 (GigabitEthernet0/0/0/1) (L2) Down, BFD session DOWN

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [81/115] Label: None, low priority
    via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2, SRGB Base: 16000, Weight: 0

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1
 2  2001:db8:200:23::1
 3  2001:db8:200:35::1
 4  2001:db8:200:45::
 5  2001:db8:100::6
```

- 路由器記錄了什麼?首先看 P2 的日誌。
- P2 上的 BFD 回報到 P4 的 session 已經 down。然後 IS-IS 移除了到 P4 的鄰接。
- 幾秒之後,PE1 有了新的 IS-IS 路由。metric 是 81,不是 31。下一跳仍然是 P2。
- traceroute 顯示新的路徑:P2、P3、P5、P4,然後 PE6。共有五跳,不是三跳。

Ep 9.10 · A repair, caught on the wire

路由器 / 工具 : tshark · 來源 : lab-evidence/09-tilfa/v3/fail-P2-P3.pcap (columns aligned)

```
$ tshark -r fail-P2-P3.pcap -Y "icmp.type == 8" -T fields -e ipv6.dst -e ipv6.routing.segleft -e ipv6.nxt | sort | uniq -c
  44  fcbb:bb00:3:e005:e001::      1   43
  15  fcbb:bb00:3:e005::          1   43
1481  fcbb:bb00:3:e005:6:e008::      4

# time of the first packet of each group, in seconds
9.645  fcbb:bb00:3:e005:e001::
9.749  fcbb:bb00:3:e005::
9.787  fcbb:bb00:3:e005:6:e008::
```

- 現在是最重要的部分:P2 到 P3 這條鏈路上的封包。我們依目的位址計算 echo request 的數量。最後兩欄是 Segments Left 和 next header。
- 階段一:44 個封包。P2 插入了它的節點保護修復路徑。路由標頭的 Segments Left 是 1,next header 是 43。
- 階段二:15 個封包,帶著較短的清單:P3 和通往 P5 的鄰接,仍然有路由標頭。我們沒有驗證是哪個路由器狀態造成這個短暫階段。
- 階段三:1481 個封包,沒有路由標頭。next header 是 4。PE1 使用它的新路徑,P2 停止了修復。第 10 集會說明這個階段。
- 時間顯示修復有多短。階段一從 9.645 秒開始,階段三從 9.787 秒開始。修復路徑承載流量大約 0.14 秒。

Ep 9.11 · Following phase 1 to PE6

路由器 / 工具 : tshark · 來源 : lab-evidence/09-tilfa/v3/fail-P2-P3, fail-P3-P5, fail-P5-PE6 (columns aligned)

```
$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e ipv6.dst -e ipv6.routing.segleft -e ipv6.nxt | sort | uniq -c
# P2 to P3
  44  fcbb:bb00:3:e005:e001::      1    43
# P3 to P5
  44  fcbb:bb00:e001::              1    43
 1496 fcbb:bb00:6:e008::              4
# P5 to PE6
  44  fcbb:bb00:6:e008::              4
```

- 跟著那 44 個修復封包一路到 PE6。我們用同一個指令讀取每條鏈路的擷取檔。
- 在 P2 到 P3 的鏈路上,P2 把修復路徑插在 service SID 前面。
- P3 執行了它的鄰接指令,並移位位址。剩下的指令是 e001,也就是 P5 通往 PE6 的鄰接。
- P5 執行 e001,把封包從直連鏈路送到 PE6。P5 是倒數第二個端點,所以它移除了路由標頭。PE6 只收到 service SID。
- P3 到 P5 這條鏈路上另外的 1496 個封包,是階段二和階段三之後的流量。它們的目的位址相同,而且沒有路由標頭。
- 第 4、5、6 集的功能,在不到四分之一秒內一起運作。

Ep 9.12 · Run it twice

來源 : lab-evidence/09-tilfa/v3/fail-ping.log, fail2-ping.log

```
# run 1
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 4000 timeout 1
Success rate is 100 percent (4000/4000), round-trip min/avg/max = 1/2/13 ms

# run 2, the same failure
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 6000 timeout 1
Success rate is 99 percent (5992/6000), round-trip min/avg/max = 1/2/17 ms
```

- 我們再做了一次同樣的故障,卻得到不同的結果。你也必須看到這一點。
- 在第一次測試中,修復路徑承載了大約 0.14 秒的流量,沒有任何 ping 遺失。
- 在第二次測試中,沒有出現任何修復封包,遺失了 8 個 ping。線路上大約有一秒沒有封包。然後 PE1 的新路徑才開始運作。
- 我們沒有驗證兩次測試為什麼不同。這是虛擬實驗裡的軟體轉送器。它能證明修復路徑存在,並顯示它的格式。它無法告訴你保護的速度。請在硬體上量測速度。

Ep 10.1 · The configuration and the delay

來源：lab-evidence/10-microloop/PE1-config.log

```
RP/0/RP0/CPU0:PE1#show running-config router isis CORE address-family ipv6 unicast
    microloop avoidance segment-routing

RP/0/RP0/CPU0:PE1#show isis protocol
    Microloop avoidance:      Enabled
    Configuration:           Type: Segment Routing, RIB update delay: 5000 msec
```

- 這是設定,以及路由器的輸出。
- 每台路由器的 IPv6 address family 底下有一行。
- 路由器顯示 RIB update delay 是 5000 毫秒。這是 Cisco 的預設值。沒有任何 RFC 規定這個數值。

Ep 10.2 · The failure test

路由器 / 工具 : P4 and P2 · 來源 : lab-evidence/10-microloop/P4-shut.log, P2-log.log

```
RP/0/RP0/CPU0:P4(config)#interface GigabitEthernet0/0/0/0 shutdown
RP/0/RP0/CPU0:P4(config)#commit

RP/0/RP0/CPU0:P2#show logging last 12
03:46:34.253 %L2-BFD-6-SESSION_STATE_DOWN : BFD session to neighbor ... has gone down. Reason: Nbor signalled down
03:46:34.253 %ROUTING-ISIS-5-ADJCHANGE : ISIS (CORE): Adjacency to P4 (GigabitEthernet0/0/0/1) (L2) Down, BFD session DOWN
```

- 現在是測試。和第 9 集一樣,我們在 PE1 的 GREEN ping 進行時,讓 P2 和 P4 之間的鏈路故障。我們先在 CML 裡對核心鏈路啟動擷取。
- 在 P4 上,我們把通往 P2 的介面關掉,並提交。
- 在 P2 上,日誌顯示到 P4 的 BFD 會話中斷,原因是鄰居通知它 down。
- IS-IS 在同一毫秒移除鄰接。這是接下來所有事件的時間零點。

Ep 10.3 · P2, a fraction of a second later

路由器 / 工具 : P2 · 來源 : lab-evidence/10-microloop/t1-P2.log

```
RP/0/RP0/CPU0:P2#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [71/115]
  Installed Oct 03 03:46:34.354
    via fe80::5454:ff:fea8:d13a, GigabitEthernet0/0/0/2, P3
    exp fe80::5454:ff:fea8:d13a, GigabitEthernet0/0/0/2, P3
      P node: P3.00 [2001:db8:100::3], SRv6 SID: fcbb:bb00:3:: uN (PSP/USD)
      Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcbb:bb00:3:e005:: uA (PSP/USD)
    SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}
```

- 我們再次讓 P2 到 P4 的鏈路故障,並在之後立刻讀取路由器。
- P2 在該分鐘的 34.354 秒,安裝了到 PE6 的新路由,metric 71。
- 新的下一跳是 P3。還有第二行,帶有 exp 標記:一條顯式路徑。
- 顯式路徑是先到 P3,再走通往 P5 的鄰接。這些 segment 和修復路徑相同。所以封包不需要 P3 的舊路由表就能到達 P5。

Ep 10.4 · PE1 does the same

來源 : lab-evidence/10-microloop/t1-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 route fcb::bb00:6::/48 detail
L2 fcb::bb00:6::/48 [81/115]
  Installed Oct 03 03:46:34.719
    via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2
    exp fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2
      P node: P3.00 [2001:db8:100::3], SRv6 SID: fcb::bb00:3:: uN (PSP/USD)
      Q node: P5.00 [2001:db8:100::5], SRv6 SID: fcb::bb00:3:e005:: uA (PSP/USD)
```

- PE1 距離故障兩跳,它也做出改變。
- PE1 在 34.719 秒安裝了新路由,metric 81。這比 P2 晚了三分之一秒多一點。
- PE1 也維持一條經過 P3 和通往 P5 鄰接的顯式路徑。它的一般下一跳仍然是 P2。

Ep 10.5 · PE1, a few seconds later

來源：lab-evidence/10-microloop/t3-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [81/115]
  Installed Oct 03 03:46:40.219
    via fe80::5454:ff:fe66:75e1, GigabitEthernet0/0/0/0, P2

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:6::/48 detail
  via fe80::5454:ff:fe66:75e1/128, GigabitEthernet0/0/0/0, protected
  via fe80::5454:ff:fecl:58eb/128, GigabitEthernet0/0/0/1, backup (Local-LFA)
```

- 這是幾秒之後的 PE1。
- 在 40.219 秒有一次新的安裝。顯式路徑不見了。只剩下經過 P2 的一般路徑。
- CEF 再次把這條路由顯示為受保護,並針對新拓撲有一條新的備援。網路已經穩定。

Ep 10.6 · The explicit path on the wire

路由器 / 工具 : tshark · 來源 : lab-evidence/09-tilfa/v3/fail2-P2-P3.pcap (second test)

```
$ tshark -r fail2-P2-P3.pcap -Y "icmp.type == 8" -T fields -e ipv6.dst | sort | uniq -c
  979  fcbb:bb00:3:e005:6:e008::
 2621  fcbb:bb00:6:e008::

# first and last packet of the explicit path, in seconds
9.897  first  fcbb:bb00:3:e005:6:e008::
14.087 last   fcbb:bb00:3:e005:6:e008::
```

- 現在我們可以說明第 9 集的第三階段。這段擷取來自那次故障的第二次測試。我們依目的位址,計算 P2 到 P3 鏈路上的 echo request。
- 故障之後,GREEN 流量立刻放進一個容器:P3、通往 P5 的鄰接,然後是 PE6 的 service SID。這就是顯式路徑,和服務放在同一個位址裡。共有 979 個封包。
- 之後,封包又只使用 service SID。它們沿著經過 P2、P3、P5、P4 的新最短路徑走。共有 2621 個封包。
- 顯式路徑從 9.897 秒持續到 14.087 秒,大約 4.2 秒。路由器狀態顯示的是 5.5 秒。請把兩個數字都當成大約的值,而不是量測值。

Ep 10.7 · Did anything loop?

路由器 / 工具 : tshark · 來源 : lab-evidence/09-tilfa/v3/fail-P2-P3.pcap, fail2-P2-P3.pcap

```
$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e eth.src -e ipv6.hlim | sort | uniq -c
# link P2 to P3, run 1
 1540  56:54:00:1c:70:f9  254
# link P2 to P3, run 2
 3600  56:54:00:1c:70:f9  254
```

- 微環路會在線路上留下證據。所以我們在 P2 和 P3 之間的鏈路上尋找它,也就是手算例子裡的那兩台路由器。我們依來源 MAC 位址和 hop limit 計算封包。
- 在第一次測試中,同一個來源送出全部 1540 個封包,hop limit 是 254。這個來源是 P2。
- 第二次測試得到相同的結果:3600 個封包,一個來源,hop limit 254。
- 沒有任何封包來自 P3。如果有封包在打轉,它會出現在兩個方向上,而且 hop limit 會一直下降。在啟用微環路防護時,我們一個也沒看到。
- 我們沒有驗證關掉防護時是否會出現環路。這是一個很適合在你的實驗裡做的測試。

Ep 10.8 · BFD

來源：lab-evidence/10-microloop/PE1-config.log

```
RP/0/RP0/CPU0:PE1#show running-config router isis CORE interface GigabitEthernet0/0/0/0
  bfd minimum-interval 100
  bfd multiplier 3
  bfd fast-detect ipv6

RP/0/RP0/CPU0:PE1#show bfd ipv6 session
Gi0/0/0/0          fe80::5454:ff:fe66:75e1
No                 n/a                 0s (0s*0)      300ms (100ms*3)  UP
```

- 偵測是第一步。這是我們讓它變快的方法。
- BFD 在每個 IS-IS 介面上執行,每 100 毫秒一次。連續 3 個封包沒收到,BFD 就判定會話中斷。
- 所以偵測時間是 300 毫秒,會話是 up。沒有 BFD 的話,IS-IS 會等 hello 的 holdtime,那要好幾秒。
- 在我們的測試中,P4 關掉自己的介面,所以 P2 是從 BFD 得知故障。所以 BFD 事件和鄰接事件發生在同一毫秒。

Ep 10.9 · Restoration is an event too

路由器 / 工具 : P4, P2 and PE1 · 來源 : lab-evidence/10-microloop/restore

```
RP/0/RP0/CPU0:P4(config)#no interface GigabitEthernet0/0/0/0 shutdown
RP/0/RP0/CPU0:P4(config)#commit

RP/0/RP0/CPU0:P2#show logging last 12
03:52:07.575  %L2-BFD-6-SESSION_STATE_UP : BFD session to neighbor ... on interface GigabitEthernet0/0/0/1 is up
03:52:14.102  %ROUTING-ISIS-5-ADJCHANGE : ISIS (CORE): Adjacency to P4 (GigabitEthernet0/0/0/1) (L2) Up, New adjacency

RP/0/RP0/CPU0:PE1#show isis ipv6 route fcbb:bb00:6::/48 detail
L2 fcbb:bb00:6::/48 [31/115] Label: None, low priority
    Installed Oct 03 03:52:20.122 for 00:00:04

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 6000 timeout 1
Success rate is 100 percent (6000/6000), round-trip min/avg/max = 1/2/12 ms
```

- 接著我們在 ping 持續進行時恢復了鏈路。恢復也是一次事件。
- 在 P4 上,我們移除 shutdown,並提交。
- BFD 最先恢復,在 03:52 之後的 07.575 秒。
- IS-IS 在 14.102 秒才把鄰接建立起來,晚了六秒半。
- PE1 在 20.122 秒把路由改回 metric 31,又晚了六秒。這和同樣的 5000 毫秒延遲套用在恢復事件上一致。
- 沒有任何 ping 遺失:6000 個全部成功。在整個測試期間,流量都留在繞道路徑上。修好的網路是在十幾秒之後,一步一步改回來的。

Ep 11.1 · The configuration

```
! PE1: defines and advertises algorithm 128
router isis CORE
 flex-algo 128
  priority 200
  metric-type delay
  advertise-definition
!
! P3: takes part, does not advertise a definition
router isis CORE
 flex-algo 128
  metric-type delay
!
! every router: a locator in algorithm 128
segment-routing srv6 locators locator FA128
 prefix fcbb:bb01:1::/48
 algorithm 128
```

- 這是我們的設定,分成三部分。
- PE1 以延遲 metric 定義 algorithm 128,優先順序 200,並通告這個定義。PE6 也做同樣的事,優先順序 100。
- P3 參與,使用相同的 metric 類型,但不通告定義。P2、P4 和 P5 的設定也一樣。
- 每台路由器都有第二個 locator,位於 fcbb:bb01 block,用於 algorithm 128。在這個系列裡,我們為 Flex- Algo 使用獨立的 block。這是我們的選擇,不是 RFC 的規定。

Ep 11.2 · Who defines algorithm 128?

路由器 / 工具 : P3 · 來源 : lab-evidence/11-flexalgo/P3.log

```
RP/0/RP0/CPU0:P3#show isis flex-algo 128
Flex-Algo 128:
  Level-2:
    Definition Priority: 200
    Definition Source: PE1.00
    Definition Equal to Local: Yes
    Definition Calculation Type: SPF
    Definition Metric Type: Delay
  Topologies supported:
    IPv6 Unicast
  Local Priority: 128
```

- 有兩台路由器通告定義。網路會使用哪一個定義?我們問 P3,它不通告任何定義。
- 被選中的定義來自 PE1,優先順序 200。
- 它和 P3 的本地設定相同。如果不同,P3 仍然會使用被選中的定義,而這一行會提醒你比較兩者。
- metric 類型是延遲。
- P3 的本地優先順序是預設值 128,因為 P3 不參與競爭。RFC 9350 選擇最高的優先順序。如果兩者相同,就選擇最高的 system ID。

Ep 11.3 · The delays we advertise

```
! PE1
performance-measurement
interface GigabitEthernet0/0/0/0
  delay-measurement
    advertise-delay 5000
interface GigabitEthernet0/0/0/1
  delay-measurement
    advertise-delay 1000
```

- 延遲值必須有來源。在真實網路裡,路由器會量測它們。在這個系列裡,我們通告固定值。這樣你就能建出同樣的實驗。
- PE1 為它通往 P2 的上方鏈路通告 5000 微秒。
- 它為通往 P3 的下方鏈路通告 1000。每條下方鏈路都是 1000。每條上方鏈路和交叉鏈路都是 5000。這和 cost 正好相反,是刻意的。

Ep 11.4 · Two routes, two paths

來源：lab-evidence/11-flexalgo/PE1.log

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  Known via "isis CORE", distance 115, metric 3001, SRv6-locator (algo 128), type level-2

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1
 2  2001:db8:200:24::1
 3  2001:db8:100::6
RP/0/RP0/CPU0:PE1#traceroute fcbb:bb01:6:: source 2001:db8:100::1
 1  2001:db8:200:13::1
 2  2001:db8:200:35::1
 3  2001:db8:100::6
```

- 路由器給出相同的結果。
- 到 PE6 有兩條路由,每個 locator 一條。Algorithm 0 的 metric 是 31。Algorithm 128 的 metric 是 3001:3000 微秒加上 locator 的 1。
- traceroute 到 PE6 的 algorithm 0 SID,經過 P2 和 P4。
- traceroute 到它的 algorithm 128 SID,經過 P3 和 P5。路由器和鏈路都一樣,但有兩套拓撲。目的位址決定使用哪一套拓撲。

Ep 11.5 · Every algorithm has its own SIDs

來源：lab-evidence/11-flexalgo/PE1-frr128.log (FA128 locator, node and adjacency SIDs)

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
*** Locator: 'FA128' ***
fcbb:bb01:1::          uN (PSP/USD)      'default':1          sidmgr
fcbb:bb01:1:e000::      uA (PSP/USD)      [Gi0/0/0/0, Link-Local]:128:P    isis-CORE
fcbb:bb01:1:e001::      uA (PSP/USD)      [Gi0/0/0/0, Link-Local]:128      isis-CORE
fcbb:bb01:1:e002::      uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:128:P    isis-CORE
fcbb:bb01:1:e003::      uA (PSP/USD)      [Gi0/0/0/1, Link-Local]:128      isis-CORE
```

- 每個演算法都有自己的 SID。第 8 集在 LSP 裡顯示過這一點。
- PE1 有第二個 node SID fcbb:bb01:1::,屬於 algorithm 128。
- 它也為每條鏈路有第二對 adjacency SID,帶有 128 標記。使用這些 SID 的封包,在每一跳都依循 algorithm 128 的路徑,而不只是第一跳。

Ep 11.6 · Protection stays inside the algorithm

來源：lab-evidence/11-flexalgo/PE1-frr128.log

```
RP/0/RP0/CPU0:PE1#show isis ipv6 fast-reroute flex-algo 128 fcbb:bb01:6::/48 detail
  via ..., GigabitEthernet0/0/0/1, P3
    Backup path: TI-LFA (node), via ..., GigabitEthernet0/0/0/0 P2, Metric: 15001
      P node: P4.00 [2001:db8:100::4], SRv6 SID: fcbb:bb01:4:: uN (PSP/USD)

RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb01:6::/48 detail
  SRv6 H.Insert.Red SID-list {fcbb:bb01:4::}
```

- TI-LFA 也在每個演算法內運作。
- 到 PE6 algorithm 128 locator 的主路徑經過 P3。它的節點保護備援經過 P2, 延遲 metric 是 15001。
- 修復用的 SID 是 P4 的 algorithm 128 node SID fcbb:bb01:4::。修復路徑和它保護的流量, 留在同一套拓撲裡。

Ep 11.7 · Putting a customer on algorithm 128

路由器 / 工具 : PE6 · 來源 : lab-evidence/11-flexalgo/PE6.log

```
RP/0/RP0/CPU0:PE6#show running-config router bgp 65000 vrf RED
  address-family ipv4 unicast
    segment-routing srv6
      locator MAIN
  address-family ipv6 unicast
    segment-routing srv6
      locator FA128

RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid | include RED
fcbb:bb01:6:e004::          uDT6          'RED'          bgp-65000
fcbb:bb00:6:e006::          uDT4          'RED'          bgp-65000
```

- 現在看結果。如何把客戶放到低延遲的拓撲上?在 SRv6 裡,最簡單的方法是在出口 PE 上加一行。
- 對 RED IPv4,PE6 從 MAIN,也就是 algorithm 0,分配它的 service SID。
- 對 RED IPv6,PE6 從 FA128,也就是 algorithm 128,分配它的 service SID。
- 所以 RED IPv6 的 service SID 位於 fcbb:bb01 block。PE1 送往它的每個封包,都會自動依循 algorithm 128。不需要 policy,也不需要 colour。

Ep 11.8 · RED IPv6, both directions

路由器 / 工具：CE-A and tshark · 來源：lab-evidence/11-flexalgo/red6-before-CE-A.log, red6-before-PE1-P3.pcap, red6-before-PE1-P2.pcap

```
CE-A:~$ sudo ping6 -c 3 2001:db8:c:206::2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmpv6.type == 128 or icmpv6.type == 129" -T fields -e ipv6.dst
# PE1 to P3, CE-A to CE-B
fcbb:bb01:6:e004::      (outer destination, algorithm 128)
# PE1 to P2, CE-B to CE-A
fcbb:bb00:1:e007::      (outer destination, algorithm 0)
```

- 現在看線路上的結果。我們在 RED IPv6 裡從 CE-A ping CE-B。我們擷取 PE1 的兩條核心鏈路,並用 tshark 讀取每個檔案。
- 三個 ping 全部成功。
- 往 CE-B 的方向,封包從下方鏈路送出,目的位址是 algorithm 128 的 service SID。
- 回覆從上方鏈路回來。PE1 自己的 RED IPv6 SID 仍在 MAIN,所以回程使用 algorithm 0。兩個方向不同,因為我們只設定了一端。
- 如果你要兩個方向都是低延遲,就在兩台 PE 上都從 algorithm 128 的 locator 分配。

Ep 11.9 · Experiment: make one link slow

```
! P3 and P5, the link between them
performance-measurement
interface GigabitEthernet0/0/0/1      ! on P5: GigabitEthernet0/0/0/0
delay-measurement
advertise-delay 20000
```

- 現在做一個測試。我們把 P3 到 P5 鏈路兩端的通告延遲,從 1000 提高到 20000 微秒。cost 不變。
- 在下一頁之前先預測結果。每個演算法會怎樣?RED IPv6 又會怎樣?

Ep 11.10 · One number changed

來源：lab-evidence/11-flexalgo/after-delay-PE1.log, red6-after-CE-A.log, red6-after-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  Known via "isis CORE", distance 115, metric 15001, SRv6-locator (algo 128), type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected

RP/0/RP0/CPU0:PE1#traceroute fcbb:bb01:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1
 2  2001:db8:200:24::1
 3  2001:db8:100::6

CE-A:~$ sudo ping6 -c 3 2001:db8:c:206::2
3 packets transmitted, 3 packets received, 0% packet loss

# captures after the change, from the same ping
PE1 to P2: fcbb:bb01:6:e004:: and fcbb:bb00:1:e007::
PE1 to P3: no echo request
```

- 這是 P3 到 P5 鏈路的延遲變成 20000 之後的結果。我們從 PE1 上的路由開始。
- Algorithm 0 沒有改變。它的 metric 仍是 31,路徑也一樣。它不使用延遲。
- Algorithm 128 現在的 metric 是 15001,下一跳是 P2,也就是上方鏈路。下方路徑總共是 22000 微秒,上方路徑的 15000 比較好。
- 對 algorithm 128 SID 的 traceroute 證實了這一點:P2、P4,然後 PE6。
- 客戶的 ping 仍然是通的。服務本身沒有改變。
- 在線路上,RED IPv6 的流量也移動了。兩個方向現在都在 PE1 到 P2 的鏈路上,PE1 到 P3 的鏈路上沒有任何流量。當我們把延遲改回來之後,流量又移回原路。

Ep 12.1 · The VRF

```
vrf BLUE
 address-family ipv4 unicast
  import route-target
    65000:100
  export route-target
    65000:100
 address-family ipv6 unicast
  import route-target
    65000:100
  export route-target
    65000:100
```

- 這是 PE1 上的 VRF BLUE。
- 有一個名稱為 BLUE 的 VRF。
- IPv4 匯入和匯出 route target 65000:100。
- IPv6 也一樣。RED 使用 65000:200。在這個系列裡,這些數值只是範例。請和你自己的編號規劃比較。

Ep 12.2 · The attachment

```
interface GigabitEthernet0/0/0/2
  description TO-CE BLUE
  vrf BLUE
  ipv4 address 10.1.1.1 255.255.255.0
  ipv6 address 2001:db8:c:101::1/64
```

- 這是通往 CE-A 的接入。
- 首先,介面放進 VRF BLUE。然後它才設定位址。
- 然後它設定 IPv4 和 IPv6 位址。RED 有自己的埠和自己的位址。

Ep 12.3 · BGP

```
router bgp 65000
 address-family vpnv4 unicast
 address-family vpnv6 unicast
 neighbor 2001:db8:100::6
  remote-as 65000
 update-source Loopback0
 address-family vpnv4 unicast
 address-family vpnv6 unicast
vrf BLUE
 rd 65000:101
 address-family ipv4 unicast
  segment-routing srv6
   locator MAIN
   alloc mode per-vrf
 redistribute connected
```

- BGP 把所有部分連接起來。
- PE loopback 之間的一條 IPv6 會話,承載 VPNv4 和 VPNv6。這個核心裡沒有 IPv4。
- 在 VRF 底下有一個 route distinguisher:PE1 是 65000:101,PE6 是 65000:106。
- 這是 SRv6 的部分。BGP 從 locator MAIN 分配 service SID,每個 VRF 一個。
- redistribute connected 會通告 CE 子網路和 PE 上的租戶 loopback。在真實網路裡,你還要加上通往 CE 的靜態路由或 eBGP。

Ep 12.4 · The service SIDs

路由器 / 工具：PE1 and PE6 · 來源：lab-evidence/12-l3vpn/PE1-build.log, PE6-sids.log (service SIDs only)

```
RP/0/RP0/CPU0:PE1#show segment-routing srv6 sid
fcbb:bb00:1:e004::          uDT4          'BLUE'          bgp-65000
fcbb:bb00:1:e005::          uDT6          'BLUE'          bgp-65000
fcbb:bb00:1:e006::          uDT6          'RED'           bgp-65000
fcbb:bb00:1:e007::          uDT4          'RED'           bgp-65000

RP/0/RP0/CPU0:PE6#show segment-routing srv6 sid
fcbb:bb00:6:e004::          uDT4          'BLUE'          bgp-65000
fcbb:bb00:6:e005::          uDT6          'BLUE'          bgp-65000
fcbb:bb00:6:e006::          uDT4          'RED'           bgp-65000
fcbb:bb01:6:e004::          uDT6          'RED'           bgp-65000
```

- 這是兩台 PE 上的 service SID。
- 在 PE1 上,BLUE 有一個 uDT4 和一個 uDT6。兩個都屬於 BGP。
- RED 有自己的一對。function 值 e004 到 e007 來自動態範圍,依分配的順序排列。
- PE6 有自己的 SID,數值也不同。在 PE6 上,RED IPv6 位於 fcbb:bb01 block。這是第 11 集 algorithm 128 的設定。不要把這些值寫進你的設定裡。要去讀它們。

Ep 12.5 · BGP and the VRF table

來源：lab-evidence/12-l3vpn/PE1-build.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast summary
Neighbor          Spk      AS MsgRcvd MsgSent      TblVer  InQ OutQ  Up/Down  St/PfxRcd
2001:db8:100::6    0 65000      31      21          70    0   0 00:02:51      5

RP/0/RP0/CPU0:PE1#show route vrf BLUE
C   10.1.1.0/24 is directly connected, 00:05:38, GigabitEthernet0/0/0/2
L   10.1.1.1/32 is directly connected, 00:05:38, GigabitEthernet0/0/0/2
B   10.1.6.0/24 [200/0] via 2001:db8:100::6 (nexthop in vrf default), 00:01:13
L   10.100.1.1/32 is directly connected, 00:05:43, Loopback101
B   10.100.6.6/32 [200/0] via 2001:db8:100::6 (nexthop in vrf default), 00:01:13
```

- 現在看控制平面。
- 通往 PE6 的 VPNv4 會話是 up。它收到五個 prefix。VPNv6 會話顯示四個。
- PE1 的 VRF BLUE 裡有本地 CE 子網路和租戶 loopback。
- 它也有兩條來自 PE6 的 BGP 路由。它們的下一跳在 default 表裡:PE6 的 loopback。路由器透過 PE6 的 locator 和 service SID 解析這個下一跳。第 13 集會一步一步追蹤這條鏈。

Ep 12.6 · Tests from CE-A

路由器 / 工具 : CE-A · 來源 : lab-evidence/12-l3vpn/CE-A-tests.log

```
CE-A:~$ sudo ping -c 2 -I 10.1.1.2 10.1.6.2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -I 10.2.1.2 10.2.6.2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping6 -c 2 2001:db8:c:106::2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping6 -c 2 2001:db8:c:206::2
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -W 1 -I 10.1.1.2 10.2.6.2
2 packets transmitted, 0 packets received, 100% packet loss
```

- 現在從客戶端做重要的測試。在 CE-A 上,選項 -I 設定來源位址,而來源位址決定使用 BLUE 或 RED 的埠。
- BLUE IPv4,站點對站點:兩個封包都有回應。
- RED IPv4:兩個封包都有回應。
- BLUE 和 RED IPv6:ping 成功。
- 現在是反向測試:從 BLUE 的來源到 RED 的位址,沒有回應。只測試可達性只做了一半。一定也要證明隔離。

Ep 12.7 · Experiment: the same address in both customers

```
! PE6
interface Loopback199
  description BLUE overlap test
  vrf BLUE
  ipv4 address 10.99.0.6 255.255.255.255
!
interface Loopback299
  description RED overlap test
  vrf RED
  ipv4 address 10.99.0.6 255.255.255.255
```

- 客戶不會一起規劃它們的位址。所以我們故意在 PE6 上給兩個客戶相同的位址。
- BLUE 裡的 10.99.0.6。
- RED 裡的 10.99.0.6。同一台路由器有兩次相同的位址。

Ep 12.8 · Two routes for one address

來源 : lab-evidence/12-l3vpn/PE1-overlap.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.99.0.6/32 detail
  Received Label 0xe0040
  Extended community: RT:65000:100
  Source Route Distinguisher: 65000:106

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf RED 10.99.0.6/32 detail
  Received Label 0xe0060
  Extended community: RT:65000:200
  Source Route Distinguisher: 65000:206
```

- BGP 如何把它們分開？
- BLUE 的路由帶著 PE6 BLUE 的 route distinguisher 到達。RED 的路由帶著 RED 的到達。所以 BGP 有兩條不同的路由。
- 每條路由都有自己的 route target。PE1 上的每個 VRF 只匯入自己的路由。
- 每條路由都有不同的 service SID function:BLUE 是 e004,RED 是 e006。它們放在 label 欄位裡,如第 3 集所示。

Ep 12.9 · One address, two destinations

路由器 / 工具：CE-A and tshark · 來源：lab-evidence/12-l3vpn/CE-A-tests.log, build-PE1-P2.pcap

```
CE-A:~$ sudo ping -c 2 -I 10.1.1.2 10.99.0.6
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -I 10.2.1.2 10.99.0.6
2 packets transmitted, 2 packets received, 0% packet loss

# outer destination on PE1 to P2, from the capture
from 10.1.1.2, BLUE   fcbb:bb00:6:e004::
from 10.2.1.2, RED    fcbb:bb00:6:e006::
```

- 現在看線路。CE-A 用兩個不同的來源位址,ping 同一個位址 10.99.0.6。
- 從它的 BLUE 位址出發,ping 由 BLUE 的 loopback 回應。
- 從它的 RED 位址出發,ping 由 RED 的 loopback 回應。
- 在擷取結果裡,PE1 把 BLUE 的封包送往 PE6 的 BLUE service SID e004。它把 RED 的封包送往 RED 的 service SID e006。同一個位址、兩個客戶、兩個 SID:路由器不會把它們混淆。

Ep 12.10 · Experiment: forget one import

```
! PE1
vrf BLUE
 address-family ipv4 unicast
  no import route-target 65000:100
```

- 再做一個測試,看看 route target 的功能。我們在 PE1 上只移除 BLUE 的 IPv4 匯入 route target。匯出和 IPv6 都不變。
- 在下一頁之前,先預測什麼會失敗,以及是哪個方向。

Ep 12.11 · Half a VPN

路由器 / 工具 : PE1, PE6 and CE-A · 來源 : lab-evidence/12-l3vpn/rt/

```
RP/0/RP0/CPU0:PE1#show route vrf BLUE
C    10.1.1.0/24 is directly connected, 00:11:03, GigabitEthernet0/0/0/2
L    10.1.1.1/32 is directly connected, 00:11:03, GigabitEthernet0/0/0/2
L    10.100.1.1/32 is directly connected, 00:11:07, Loopback101

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24
Paths: (0 available, no best path)

RP/0/RP0/CPU0:PE6#show route vrf BLUE
B    10.1.1.0/24 [200/0] via 2001:db8:100::1 (nexthop in vrf default), 00:08:19
B    10.100.1.1/32 [200/0] via 2001:db8:100::1 (nexthop in vrf default), 00:08:19

CE-A:~$ sudo ping -c 2 -W 1 -I 10.1.1.2 10.1.6.2
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 2001:db8:c:106::2
2 packets transmitted, 2 packets received, 0% packet loss
```

- 這是在 PE1 上移除 BLUE IPv4 的 65000:100 匯入之後,兩端的結果。
- PE1 的 BLUE 表失去了所有來自 PE6 的 IPv4 路由。只剩下本地的路由。
- BGP 沒有 CE-B 子網路的路徑。PE1 沒有保留 PE6 的路由,因為 PE1 上沒有任何 VRF 匯入那個 route target。
- PE6 不知道這個問題。它仍然有 PE1 的路由,因為 PE1 仍然匯出它們。這個 VPN 只有一個方向壞了。
- BLUE IPv4 的 ping 失敗。
- 但 BLUE IPv6 仍然正常,因為我們沒有改變 IPv6 的匯入。我們把那一行加回去之後,所有路由在幾秒內就回來了。
- 當某個客戶的某個位址族失效時,先檢查兩端該位址族的 route target。

Ep 13.1 · Step 1: PE6 allocates and advertises

路由器 / 工具 : PE6 · 來源 : lab-evidence/13-chain/PE6-egress.log

```
RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
BGP routing table entry for 10.1.6.0/24, Route Distinguisher: 65000:106
  SRv6-VPN SID: fcbb:bb00:6:e004::/64
  Format: f3216
  Alloc Mode/Locator ID: per-vrf/2
  Advertised to peers (in unique update groups):
    2001:db8:100::1
    Origin incomplete, metric 0, localpref 100, weight 32768, valid, redistributed, best
    Extended community: RT:65000:100
```

- 第一步,在 PE6 上,也就是這個 prefix 開始的地方。
- 這條路由在 BLUE 裡,帶著 PE6 BLUE 的 route distinguisher。
- PE6 加上它 BLUE 的 service SID fcbb:bb00:6:e004::。這個 SID 來自 locator MAIN,使用 per-VRF 分配。
- PE6 把這條路由通告給 PE1。
- Route target 是 65000:100。現在我們在線路上擷取這個通告。

Ep 13.2 · Step 2: make PE6 send an update

路由器 / 工具 : PE6 and tshark · 來源 : lab-evidence/13-chain/flap-shut.log, flap-noshut.log

```
RP/0/RP0/CPU0:PE6(config)#interface Loopback199 shutdown
RP/0/RP0/CPU0:PE6(config)#commit
RP/0/RP0/CPU0:PE6(config)#no interface Loopback199 shutdown
RP/0/RP0/CPU0:PE6(config)#commit

$ tshark -r bgp-update-P4-PE6.pcap -Y "bgp.type == 2" -V
```

- 第二步。為了讓 PE6 送出更新,我們讓它的一個 BLUE loopback 閃斷一次。我們先在 CML 裡對 PE6 到 P4 的鏈路啟動擷取。
- 我們把 loopback 關掉,PE6 就撤回它的路由。
- 然後我們移除 shutdown,PE6 再次通告這些路由。這就是我們想看的更新。
- 我們用 tshark 的 verbose 模式讀取擷取檔,並以 BGP update 作為篩選條件。接下來是解碼後的更新。

Ep 13.3 · Step 2: the update on the wire, part 1

路由器 / 工具 : capture · 來源 : lab-evidence/13-chain/bgp-update-decode.txt, from bgp-update-P4-PE6.pcap

```
Border Gateway Protocol - UPDATE Message
  Path Attribute - MP_REACH_NLRI
    Address family identifier (AFI): IPv4 (1)
    Subsequent address family identifier (SAFI): Labeled VPN Unicast (128)
    Next hop: RD=0:0 IPv6=2001:db8:100::6
    Network Layer Reachability Information (NLRI)
      Label Stack: 917568 (bottom)
      Route Distinguisher: 65000:106
      MP Reach NLRI IPv4 prefix: 10.99.0.6
  Path Attribute - EXTENDED_COMMUNITIES
    Route Target: 65000:100
```

- 這個更新攜帶第 12 集的重疊位址 10.99.0.6。CE-B 的子網路使用相同的格式。
- 位址族是 IPv4、labeled VPN unicast。MPLS VPN 也使用同一個位址族。
- 下一跳是 PE6 的 IPv6 loopback。它不是 SID。
- NLRI 裡有一個 label、route distinguisher 和 prefix。label 是 917568。請記住這個數字。
- Route target 是一個 extended community。

Ep 13.4 · Step 2: the update on the wire, part 2

路由器 / 工具 : capture · 來源 : lab-evidence/13-chain/bgp-update-decode.txt

```
Path Attribute - BGP Prefix-SID
  Type Code: BGP Prefix-SID (40)
  SRv6 L3 Service
    Type: SRv6 L3 Service (5)
    SRv6 Service Sub-TLV - SRv6 SID Information
      SRv6 SID Value: fcbb:bb00:6::
      SRv6 Endpoint Behavior: End.DT4 with NEXT-CSID (0x003f)
      SRv6 SID Structure
        Locator Block Length: 32
        Locator Node Length: 16
        Function Length: 16
        Argument Length: 0
        Transposition Length: 16
        Transposition Offset: 48
```

- 這是同一個更新的第二部分:SRv6 資訊。
- 它在 BGP Prefix-SID 屬性裡,type 40,放在 SRv6 L3 Service TLV 裡,type 5。RFC 9252 定義了它們。
- SID 值只有 fcbb:bb00:6::,也就是 PE6 的 locator。行為碼是 End.DT4 with NEXT-CSID。這就是 IOS XR 上的 uDT4。
- 結構是 block 32、node 16、function 16、argument 0。
- 還有兩個數字:轉置長度 16 和偏移 48。function 不在 SID 值裡。這兩個數字說明了它在哪裡。

Ep 13.5 · Step 3: PE1's BGP path

來源 : lab-evidence/13-chain/PE1-ingress.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
BGP routing table entry for 10.1.6.0/24, Route Distinguisher: 65000:101
  Received Label 0xe0040
  Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best, imported
  Extended community: RT:65000:100
  SID Locator: fcb:b00:6::/48, RIB metric: 31
  T:1(Sid information), Sid:fcb:b00:6::(Transposed), Behavior:63
    Length [Loc-blk,Loc-node,Func,Arg]:[32,16,16,0], Tpose-len:16, Tpose-offset:48
  Source AFI: VPNv4 Unicast, Source VRF: default, Source Route Distinguisher: 65000:106
```

- 第三步,在 PE1 上。
- PE1 把這條路由匯入 BLUE,從 PE6 的 route distinguisher 匯入到 PE1 自己的 route distinguisher。
- 收到的 label 是 e0040。這和線路上的數字相同,只是以十六進位顯示。
- BGP 檢查了 locator:它可達,metric 31。在第 3 集,這一行顯示的是 inaccessible。
- 這是收到的轉置 SID 和它的結構。PE1 現在有建立完整 SID 所需的全部資訊。

Ep 13.6 · Step 4: the VRF route

來源 : lab-evidence/13-chain/PE1-ingress.log

```
RP/0/RP0/CPU0:PE1#show route vrf BLUE 10.1.6.0/24 detail
Routing entry for 10.1.6.0/24
  Known via "bgp 65000", distance 200, metric 0, type internal
    2001:db8:100::6, from 2001:db8:100::6
      Nexthop in Vrf: "default", Table: "default", IPv6 Unicast
      Source RD attributes: 0x0000:65000:106
      SRv6 Headend: H.Encaps.Red [f3216], SID-list {fcbb:bb00:6:e004::}
```

- 第四步,VRF 路由表。
- 這條路由在 BLUE 裡。PE1 從 BGP 學到它。
- 它的下一跳是 default 表裡的 PE6 loopback。BGP 的下一跳是 loopback,和 MPLS VPN 一樣。
- 這是完整的 SID fcbb:bb00:6:e004::,以及封裝的指令:reduced 形式、f3216。這一行就是這條路由裡全部的 SRv6 資訊。

Ep 13.7 · Step 5: CEF resolves through the locator

來源 : lab-evidence/13-chain/PE1-ingress.log

```
RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
10.1.6.0/24, version 31, SRv6 Headend
  via fcbb:bb00:6::/128, 9 dependencies, recursive
    next hop fcbb:bb00:6::/128 via fcbb:bb00:6::/48
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
      0      Y      GigabitEthernet0/0/0/0      fe80::5454:ff:fe66:75e1

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
```

- 第五步,CEF。
- 這筆項目不是透過 PE6 的 loopback 解析。它是透過 PE6 的 locator fcbb:bb00:6::/48 解析。所以必須可達的是 locator,而不是 loopback。
- 這是封裝的指令,目標是 service SID。
- 這是出口介面,通往 P2 的 GigabitEthernet 0/0/0/0。它來自到 locator 的 IS-IS 路由。PE1 上的鏈到這裡結束:VPN 路由、locator 路由、介面。

Ep 13.8 · Step 6: the wire

路由器 / 工具 : CE-A and tshark · 來源 : lab-evidence/13-chain/CE-A.log, dp-*.pcap

```
CE-A:~$ sudo ping -c 2 -I 10.1.1.2 10.1.6.2
2 packets transmitted, 2 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e frame.len -e ipv6.dst -e ipv6.hlim
# CE-A to PE1          98
# PE1 to P2           138    fcb:b00:6:e004::    255
# P4 to PE6           138    fcb:b00:6:e004::    253
# PE6 to CE-B          98
```

- 第六步,封包。CE-A 送出一個 ping,我們在四個位置擷取它。
- 兩個 ping 都有回應。
- 從 CE-A 送出的,是一個一般的 98 位元組 IPv4 ping。
- 離開 PE1 時,它大了 40 位元組,目的位址是 service SID,外層 hop limit 是 255。
- 到達 PE6 時,它完全一樣,只是兩台路由器減少了外層 hop limit。
- 在通往 CE-B 的鏈路上,它又是一個一般的封包。客戶封包在隧道裡沒有改變。

Ep 13.9 · Step 7: PE6 receives

路由器 / 工具 : PE6 · 來源 : lab-evidence/13-chain/PE6-egress.log

```
RP/0/RP0/CPU0:PE6#show cef ipv6 fcb::bb00:6:e004::/64
fcb::bb00:6:e004::/64, version 136, SRv6 Endpoint uDT4
  next hop VRF - 'BLUE', table - 0xe0000002
  0      Y      recursive          Lookup in table

RP/0/RP0/CPU0:PE6#show route vrf BLUE 10.1.6.2
Routing entry for 10.1.6.0/24
  Known via "connected", distance 0, metric 0 (connected)
    directly connected, via GigabitEthernet0/0/0/2
```

- 第七步,再回到 PE6。
- 目的位址符合 PE6 的 uDT4 項目。它的動作是在 BLUE 裡查詢內層封包。
- 在 BLUE 裡,CE-B 是直接連接的。整條鏈完成了:它從 PE6 的分配開始,在 PE6 的一個埠結束。

Ep 13.10 · TTL, hop limit and DSCP

路由器 / 工具：CE-A and tshark · 來源：lab-evidence/13-chain/CE-A.log, dp-PE1-CE-A.pcap, dp-PE1-P2.pcap, dp-P4-PE6.pcap

```
CE-A:~$ sudo traceroute -n -t 184 -s 10.1.1.2 10.1.6.2 -w 1 -q 1 -m 5
 1  10.1.1.1
 2  10.46.0.1
 3  10.1.6.2

$ tshark -r <capture file> -Y "ip.dst == 10.1.6.2 and udp" -T fields \
    -e frame.len -e ipv6.dst -e ipv6.hlim -e ipv6.tclass -e ip.dsfield.dscp -e ip.ttl
# CE-A to PE1      60                                DSCP 46      TTL 1, 2, 3
# PE1 to P2       100      fcb:b00:6:e004::      hop limit 255   tclass 0x0    DSCP 46      TTL 1, 2
# P4 to PE6       100      fcb:b00:6:e004::      hop limit 253   tclass 0x0    DSCP 46      TTL 1, 2
```

- 有兩個標頭欄位很重要:hop limit 和 QoS 標記。我們從 CE-A 送出 traceroute 探測封包,標記為 EF,也就是 DSCP 46。
- traceroute 顯示 PE1、PE6,然後是 CE-B。客戶看不到 P2 和 P4。RFC 2983 稱之為 pipe 模型。
- 在接入鏈路上,探測封包的 DSCP 是 46,TTL 是 1、2、3。
- 在核心裡,PE1 把外層 hop limit 設為 255。它不複製客戶的 TTL。P2 和 P4 只減少外層的 hop limit:到 PE6 時是 253。
- 內層 TTL 沒有改變。內層 DSCP 仍然是 46,但外層 traffic class 是 0。依外層標頭排隊的核心路由器,會把這個流量當成盡力而為來傳送。
- 在這個平台上,traffic class 的傳遞是可以設定的,如第 2 集 manager 所顯示。如果你的核心依外層標頭排隊,就要設定它,並用擷取來驗證。

Ep 14.1 · The configuration on PE1

來源：lab-evidence/14-vpws/PE1-commit.log, PE1-state.log

```
interface GigabitEthernet0/0/0/4
  description TO-CE L2 attachment circuit
  l2transport
!
l2vpn
  xconnect group SRV6-VPWS
  p2p ELINE-300
    interface GigabitEthernet0/0/0/4
      neighbor evpn evi 300 service 300 segment-routing srv6
!
evpn
  segment-routing srv6
  locator MAIN
```

- 這是 PE1 上的全部設定。PE6 上的設定相同,只是方向相反。BGP 會話已經承載 Layer 2 VPN EVPN 位址族。
- 客戶埠變成二層傳輸埠。它沒有 IP 位址。PE1 不在這個埠上做路由。
- 一個點對點的 cross-connect 把這個埠連到一個遠端服務。
- 這是 EVPN 實例 300、服務 300,走 SRv6。兩台 PE 上的服務編號必須相同。
- EVPN 從 locator MAIN 分配它的 SID。我們沒有設定 route target。EVPN 用 AS 號碼和實例編號產生了 65000:300。

Ep 14.2 · Route type 1 on the wire, part 1

路由器 / 工具 : capture · 來源 : lab-evidence/14-vpws/bgp-update-decode.txt, frame 179 of build-PE1-P2.pcap

```
Border Gateway Protocol - UPDATE Message
  Path Attribute - MP_REACH_NLRI
    Address family identifier (AFI): Layer-2 VPN (25)
    Subsequent address family identifier (SAFI): EVPN (70)
    Next hop: 2001:db8:100::1
    EVPN NLRI: Ethernet AD Route
      Route Type: Ethernet AD Route (1)
      Route Distinguisher: 10.0.0.1:300
      ESI: 00:00:00:00:00:00:00:00:00:00
      Ethernet Tag ID: 300
      MPLS Label 1: 917680
  Path Attribute - EXTENDED_COMMUNITIES
    EVPN Layer 2 Attributes: flags: 0x0002, L2 MTU: 0
    Route Target: 65000:300
```

- 我們在 commit 期間擷取 PE1 的核心鏈路。不到兩秒,PE1 就送出了這個更新。
- 位址族是 Layer 2 VPN EVPN。下一跳是 PE1 的 loopback。
- 這是 type 1 路由,Ethernet auto-discovery。PE1 用它的 router ID 和實例編號產生了 route distinguisher。
- Ethernet segment identifier 是零,因為 CE 只連到一台 PE。Ethernet tag 是 300,也就是服務編號。遠端 PE 用它來配對兩端。
- 又有一個 label 欄位:917680。
- 有兩個 extended community。第一個是 Layer 2 attributes,MTU 為零。零的意思是:不檢查。第二個是 route target。

Ep 14.3 · Route type 1 on the wire, part 2

路由器 / 工具 : capture · 來源 : lab-evidence/14-vpws/bgp-update-decode.txt

```
Path Attribute - BGP Prefix-SID
  SRv6 L2 Service
    Type: SRv6 L2 Service (6)
  SRv6 Service Sub-TLV - SRv6 SID Information
    SRv6 SID Value: fcb:bb00:1::
    SRv6 Endpoint Behavior: End.DX2 with NEXT-CSID (0x0041)
  SRv6 SID Structure
    Locator Block Length: 32
    Locator Node Length: 16
    Function Length: 16
    Argument Length: 0
    Transposition Length: 16
    Transposition Offset: 48
```

- 第二部分有和第 13 集相同的 Prefix-SID 屬性,只有一個不同。
- TLV 是 SRv6 L2 Service,type 6。在第 13 集,它是 L3 Service,type 5。
- SID 值是 PE1 的 locator。行為是 End.DX2 with NEXT-CSID,代號 65。在 IOS XR 上,這就是 uDX2:移除外層標頭,把訊框從某一個埠送出去。
- 轉置也相同:16 位元,偏移 48。所以 function 又放在 label 欄位裡。

Ep 14.4 · The cross-connect is up

來源：lab-evidence/14-vpws/PE1-verify.log

```
RP/0/RP0/CPU0:PE1#show l2vpn xconnect detail
Group SRV6-VPWS, XC ELINE-300, state is up; Interworking none
AC: GigabitEthernet0/0/0/4, state is up
MTU 1500; XC ID 0x2; interworking none
EVPN: PW ID: evi 300, ac-id 300, state is up ( established )
Encapsulation SRv6
  SRv6          Local          Remote
  uDX2          fcb:b00:1:e00b:: fcb:b00:6:e007::
  AC ID         300            300
  MTU           1514          0
  Locator Resolved Yes        N/A
  SRv6 Headend  H.Encaps.L2.Red N/A
```

- 現在看 PE1 上的狀態。
- cross-connect 是 up。客戶埠也是 up。
- EVPN 這一側是 up。本地 SID 是 e00b,遠端 SID 是 e007。這正是我們從擷取中重組出來的 SID。
- 兩端都是服務 300。PE6 通告的 MTU 是零,所以不做 MTU 檢查。請記住這一點。稍後它很重要。
- headend 行為是 H.Encaps.L2.Red:封裝一個二層訊框,reduced 形式。

Ep 14.5 · BGP and CEF on PE1

來源 : lab-evidence/14-vpws/PE1-bgp.log

```
RP/0/RP0/CPU0:PE1#show bgp l2vpn evpn rd 10.0.0.6:300 [1][0000.0000.0000.0000.0000][300]/120 detail
  2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
    Received Label 0xe00700
    Extended community: EVPN L2 ATTRS:0x02:0 RT:65000:300
    SID Locator: fcb:b00:6::/48, RIB metric: 31
    T:1(Sid information), Sid:fcb:b00:6:: (Transposed), Behavior:65

RP/0/RP0/CPU0:PE1#show cef ipv6 fcb:b00:1:e00b::/64 detail
fcb:b00:1:e00b::/64, version 243, SRv6 Endpoint uDX2
  via ::/128, GigabitEthernet0/0/0/4, 2 dependencies, weight 0, class 0
  AC Interface: Gi0/0/0/4
  XConnect ID: 0x2
```

- 這和三層服務的解析鏈相同,只是最後一步不同。
- PE1 收到了 PE6 的 type 1 路由。label 以三個位元組的形式顯示。
- 這是 locator 檢查,metric 31,行為代號 65。如果 PE6 的 locator 不可達,這個服務就會失效。這和第 3 集的故障 A 相同。
- 這是 CEF 裡 PE1 的 SID。uDT4 的動作是在 VRF 裡查表。uDX2 沒有查表。動作就是一個埠,Gi0/0/0/4。

Ep 14.6 · From CE-A

路由器 / 工具 : CE-A · 來源 : lab-evidence/14-vpws/CE-A-ping.log

```
CE-A:~$ sudo ip neigh flush dev eth2
CE-A:~$ sudo ping -c 3 172.16.30.6
64 bytes from 172.16.30.6: seq=0 ttl=64 time=2.686 ms
64 bytes from 172.16.30.6: seq=1 ttl=64 time=1.449 ms
64 bytes from 172.16.30.6: seq=2 ttl=64 time=1.362 ms
CE-A:~$ sudo ping6 -c 3 2001:db8:30::6
3 packets transmitted, 3 packets received, 0% packet loss
CE-A:~$ sudo ip neigh show dev eth2
172.16.30.6 lladdr 52:54:00:34:24:24 ref 1 used 0/0/0 probes 4 REACHABLE
2001:db8:30::6 lladdr 52:54:00:34:24:24 ref 1 used 0/0/0 probes 4 REACHABLE
```

- 現在送流量。首先,我們清空 CE-A 上的鄰居表。這樣 ARP 也必須穿越這個服務。
- 鄰居表是空的。
- IPv4 ping CE-B 收到三個回應,TTL 64。沒有路由器減少它。對客戶來說,兩台 CE 之間沒有路由器。
- IPv6 也能運作。PE 上沒有為這個服務設定任何 IPv6。
- 這證明這個服務是二層的。CE-A 學到 CE-B 的 MAC 位址,結尾是 24:24。在三層 VPN 裡,CE-A 只會學到 PE1 的 MAC 位址。

Ep 14.7 · The frames, captured at three points

路由器 / 工具 : tshark · 來源 : lab-evidence/14-vpws/dp-PE1-Gi0_0_0_4-CE-A.pcap, dp-PE1-P2.pcap, dp-P4-PE6.pcap

```
$ tshark -r <capture file> -Y "arp or icmp or ipv6.nxt == 143" -T fields \
  -e frame.len -e ipv6.dst -e ipv6.nxt -e ipv6.hlim
# CE-A to PE1
  42                                ARP request
  60                                ARP reply
  98                                echo request
# PE1 to P2
 114 fcb:b00:6:e007:: 143 255    ARP request
 114 fcb:b00:1:e00b:: 143 253    ARP reply
 152 fcb:b00:6:e007:: 143 255    echo request
# P4 to PE6
 114 fcb:b00:6:e007:: 143 253    ARP request
 152 fcb:b00:6:e007:: 143 253    echo request
```

- CE-A 在 ping 的時候,我們在三個位置擷取,並用 tshark 讀取每個檔案。
- 在接入鏈路上,有一個 42 位元組的 ARP request、一個 60 位元組的 ARP reply,以及 98 位元組的 ping。
- 在核心裡,ARP 廣播變成單播,送往 PE6 的 SID,外層 next header 是 143。點對點服務的廣播只有一個目的地。
- CE-B 的回應回到 PE1 的 SID,e00b。
- 98 位元組的 ping 在核心裡是 152 位元組。next header 143 代表乙太網路。RFC 8986 定義了這個值。
- 在 P4 到 PE6 的鏈路上,訊框相同,外層 hop limit 少了二,和三層服務一樣。

Ep 14.8 · Testing the edge

路由器 / 工具 : CE-A · 來源 : lab-evidence/14-vpws/CE-A-mtu.log, CE-A-ping2 in lab-evidence/13-evpn-vpws

```
CE-A:~$ sudo ping -c 2 -s 1418 172.16.30.6
PING 172.16.30.6 (172.16.30.6): 1418 data bytes
1426 bytes from 172.16.30.6: seq=0 ttl=64 time=1.296 ms
1426 bytes from 172.16.30.6: seq=1 ttl=64 time=1.639 ms
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -s 1419 172.16.30.6
PING 172.16.30.6 (172.16.30.6): 1419 data bytes
2 packets transmitted, 0 packets received, 100% packet loss
```

- 1418 位元組資料的 ping,是一個 1446 位元組的 IP 封包:1418,加上 ICMP 的 8,再加上 IPv4 的 20。
- 1446 位元組的封包通過。
- 多一個位元組,所有的 ping 都遺失了。CE-A 沒有收到任何錯誤訊息。它只能等待。

Ep 14.9 · Where the 1447-byte packet died

路由器 / 工具 : CE-A, tshark and routers · 來源 : lab-evidence/14-vpws/CE-A-mtu.log, mtu-PE1-P2.pcap, mtu-P4-PE6.pcap, PE1-mtu.log, PE6-mtu.log

```
CE-A:~$ sudo ping -c 2 -s 1419 172.16.30.6
2 packets transmitted, 0 packets received, 100% packet loss

$ tshark -r <capture file> -Y "ipv6.fraghdr" -T fields -e frame.len -e ipv6.src -e ipv6.dst -e ipv6.fraghdr.offset
# PE1 to P2
1510    2001:db8:100::1    fcb:b00:6:e007::    0
   75    2001:db8:100::1    fcb:b00:6:e007::    181
# P4 to PE6: the same two fragments

RP/0/RP0/CPU0:PE1#show ipv6 traffic
      2 fragmented into 4 fragments, 0 failed
RP/0/RP0/CPU0:PE6#show ipv6 traffic
      0 fragments, 0 total reassembled
```

- 封包去了哪裡?我們有四項證據。首先是資料 1419 位元組的 ping。
- 所有的 ping 都遺失了,CE-A 沒有收到任何錯誤。
- PE1 沒有丟棄這個訊框。它把外層 IPv6 封包切成兩個片段:線路上是 1510 和 75 位元組。
- PE1 的計數器也吻合:兩個封包被切成四個片段。這就是我們的兩個 ping。
- 兩個片段都穿越了核心,到達 PE6。
- 但 PE6 的 IPv6 計數器顯示零個片段。轉送平面比對 SID 後,在重組之前就把片段丟掉了。CE-B 什麼也沒收到,也沒有任何設備送出訊息。
- 這是最糟的一種故障:它無聲無息,而且只發生在大封包上。

Ep 15.1 · The policy on PE1

來源：configs/policy/PE1.cfg

```
segment-routing
traffic-eng
segment-lists
segment-list VIA-P3-P5-UN
  srv6
    index 10 sid fcbb:bb00:3::
    index 20 sid fcbb:bb00:5::
    index 30 sid fcbb:bb00:6::
policy BLUE-C100
  srv6
    locator MAIN binding-sid dynamic behavior ub6-insert-reduced
  color 100 end-point ipv6 2001:db8:100::6
  candidate-paths
    preference 100
    explicit segment-list VIA-P3-P5-UN
```

- 這是 PE1 上的 policy。我們的目標是讓 BLUE 送往 PE6 的流量走下面那排鏈路,經過 P3 和 P5。
- segment list 有三個 node SID:P3、P5 和 PE6。
- Binding SID 來自 locator MAIN。它是動態的,行為是 insert-reduced。第 4 集顯示這個平台只提供這種形式。
- color 是 100。endpoint 是 PE6 的 loopback。
- 有一條候選路徑,preference 100。它使用這個 list。

Ep 15.2 · The policy is up

來源：lab-evidence/15-policy/v3/PE1-steer.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 100 detail
Color: 100, End-point: 2001:db8:100::6
  Admin: up   Operational: up for 00:35:12 (since Oct  3 07:46:12.843)
  Preference: 100 (configuration) (active)
    Explicit: segment-list VIA-P3-P5-UN (valid)
      SID[0]: fcbb:bb00:3::/48
      SID[1]: fcbb:bb00:5::/48
      SID[2]: fcbb:bb00:6::/48
    Binding SID behavior: uB6 (PSP/USD Insert.Red)
    Install Final Node SID: No
  Binding SID: fcbb:bb00:1:e008::
  Steering BGP disabled: no
  Invalidation drop enabled: no
```

- 這是 PE1 上的狀態。
- policy 是 up。顯式路徑是 active, 它的 list 有效。
- 看這一行: install final node SID, no。PE6 的 node SID 在 list 裡, 但 PE1 不把它放進封包。service SID 本身就會到達 PE6。
- Binding SID 是 fcbb:bb00:1:e008::。它是 PE1 locator 上的一個 SID。
- 允許 BGP 導流。invalidation drop 是關閉的。如果 policy 失效, 流量會回到一般路徑。我們最後會測試這一點。

Ep 15.3 · Colour on the egress

路由器 / 工具 : PE6 · 來源 : configs/policy/PE6.cfg; commit in lab-evidence/15-policy/v3/PE6-commit.log

```
extcommunity-set opaque COLOR-100
  100
end-set
!
route-policy BLUE-EXPORT-C100
  set extcommunity color COLOR-100
end-policy
!
vrf BLUE
  address-family ipv4 unicast
    export route-policy BLUE-EXPORT-C100
```

- 接下來是出口這一側。PE6 選擇它的哪些路由使用 color 100。
- color 是一個 opaque extended community。這裡它的值是 100。
- 由一個 route policy 設定它。
- BLUE 的 IPv4 export 使用這個 policy。只有 BLUE IPv4 得到 color。BLUE IPv6、RED 和 GREEN 都不變。

Ep 15.4 · The coloured update on the wire

路由器 / 工具 : capture · 來源 : lab-evidence/15-policy/v3/colour-update-decode.txt, frame 157 of colour-P4-PE6.pcap (condensed)

```
Border Gateway Protocol - UPDATE Message
  Path Attribute - MP_REACH_NLRI
    Next hop:  RD=0:0 IPv6=2001:db8:100::6
      Label Stack: 917568 (bottom)  RD 65000:106  10.99.0.6
      Label Stack: 917568 (bottom)  RD 65000:106  10.1.6.0
      Label Stack: 917568 (bottom)  RD 65000:106  10.100.6.6
  Path Attribute - EXTENDED_COMMUNITIES
    Color: 0x0000 0x0000 0x0064 [Transitive Opaque]
    Route Target: 65000:100
  Path Attribute - BGP Prefix-SID
    SRv6 SID Value: fcbb:bb00:6::
    SRv6 Endpoint Behavior: End.DT4 with NEXT-CSID (0x003f)
```

- 我們在 commit 期間擷取 PE6 到 P4 的鏈路。PE6 立刻重新通告了 BLUE。
- 一個更新攜帶三個 BLUE prefix,label 都相同。第 13 集說過這一點:這些 prefix 共用同一組屬性。
- 這是新的屬性:color,值是 0x64。這是十進位的 100。
- service SID 沒有改變。color 不改變 PE6 對封包做什麼。它改變的是 PE1 到 PE6 所用的路徑。

Ep 15.5 · PE1 steers into the policy

來源 : lab-evidence/15-policy/v3/PE1-steer.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
 2001:db8:100::6 C:100 (bsid: fcbb:bb00:1:e008::) (metric 30) from 2001:db8:100::6
   Extended community: Color:100 RT:65000:100
   SR policy color 100, up, not-registered, bsid fcbb:bb00:1:e008::

RP/0/RP0/CPU0:PE1#show route vrf BLUE 10.1.6.0/24 detail
 2001:db8:100::6, from 2001:db8:100::6 via srv6-sid fcbb:bb00:1:e008:: (resolution len 64)
   SRv6 Headend: H.Encaps.Red [f3216], SID-list {fcbb:bb00:6:e004::}

RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
 via local-srv6-sid fcbb:bb00:1:e008::, 3 dependencies, recursive
   SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
   SRv6 H.Insert.Red SID-list {fcbb:bb00:3:5::}
 0      Y      GigabitEthernet0/0/0/0      fe80::5454:ff:fe66:75e1
```

- 在 PE1 上,我們看和第 13 集相同的三張表。每一張都顯示了新東西。
- 首先是 BGP。下一跳現在顯示 C 100 和 Binding SID。color 100 和下一跳 PE6 比對到我們的 policy,而這條 policy 是 up。
- 接著是 VRF 路由。service SID 相同。但這條路由現在透過 Binding SID 解析,而不是透過 PE6 的 locator。
- CEF 顯示兩個步驟。首先,用 service SID 封裝。然後,把 policy 的 list 插到它前面:fcbb:bb00:3:5::。這就是 Binding SID 的 insert 形式。
- 第一條鏈路是 GigabitEthernet 0/0/0/0,通往 P2。它不是通往 P3 的鏈路。請記住這一點。

Ep 15.6 · On the wire: every core link

路由器 / 工具 : CE-A and tshark · 來源 : lab-evidence/15-policy/v3/CE-A-c100.log, c100-*.pcap

```
CE-A:~$ sudo ping -c 3 -I 10.1.1.2 10.1.6.2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8" -T fields -e ipv6.dst -e ipv6.hlim | sort -u
# PE1 to P2      fcb:b00:3:5:6:e004:: 255
# P2 to P3      fcb:b00:3:5:6:e004:: 254
# P3 back to P2 fcb:b00:5:6:e004:: 253
# P2 to P4      fcb:b00:5:6:e004:: 252
# P4 to P5      fcb:b00:5:6:e004:: 251
# P5 back to P4 fcb:b00:6:e004:: 250
# P4 to PE6     fcb:b00:6:e004:: 249
```

- CE-A 在 BLUE 裡 ping CE-B 時,我們擷取了全部八條核心鏈路。三個 ping 都有回應。我們用 tshark 讀取每個檔案。這些是所有載送請求的鏈路。
- PE1 把封包送給 P2。它沒有送給 P3。
- P2 不做任何改變,把它送給 P3。
- P3 移除它的 micro-SID,封包又回到 P2。
- 接著封包到 P4 和 P5。P5 移除它的 micro-SID。
- 然後它又回到 P4,再到 PE6。這是七跳而不是三跳,還有兩次迴轉。封包沒有用到下面那排鏈路。
- hop limit 顯示了這一點:它從 255 降到 249,每台路由器減一。

Ep 15.7 · A strict path, with adjacency SIDs

來源：lab-evidence/15-policy/v3/pe1-strict.cfg, PE1-strict.log

```
segment-lists
segment-list VIA-P3-P5-UA
  srv6
    index 10 sid fcbb:bb00:3:e005::
    index 20 sid fcbb:bb00:5:e003::
policy BLUE-C100
candidate-paths
  preference 200
  explicit segment-list VIA-P3-P5-UA

RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e004::}
SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005:5:e003::}
```

- 要走下面那排鏈路,我們需要第 4 集的 adjacency SID。我們在這次測試之前才在 P3 和 P5 上讀取它們。動態的 uA 值在重新開機後,或中斷超過三分鐘後,可能會改變。
- P3 通往 P5 的 adjacency SID 是 e005。
- P5 通往 PE6 的 adjacency SID 是 e003。
- 我們把它們加成第二條候選路徑,preference 200。它變成 active,而我們不必刪除第一條路徑。
- CEF 現在插入 fcbb:bb00:3:e005:5:e003::。Binding SID 沒有改變。所以 BGP 和 VRF 路由也都沒有改變。

Ep 15.8 · Strict path on the wire

路由器 / 工具：CE-A and tshark · 來源：lab-evidence/15-policy/v3/CE-A-strict.log, strict-*.pcap

```
CE-A:~$ sudo ping -c 3 -I 10.1.1.2 10.1.6.2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8 or icmp.type == 0" \
  -T fields -e ipv6.dst -e ipv6.hlim -e icmp.type | sort -u
# PE1 to P2      fcb:b00:3:e005:5:e003:6:e004      255      request
# P2 to P3       fcb:b00:3:e005:5:e003:6:e004      254      request
# P3 to P5       fcb:b00:5:e003:6:e004::           253      request
# P5 to PE6      fcb:b00:6:e004::                 252      request
# P4 to PE6      fcb:b00:1:e004::                 255      reply
# P2 to P4       fcb:b00:1:e004::                 254      reply
# PE1 to P2      fcb:b00:1:e004::                 253      reply
```

- 這是同一個 ping 在線路上的位址。
- 一個位址裡有六個 micro-SID:P3、它的鄰接、P5、它的鄰接、PE6 和服務。位址已經滿了,但沒有 routing header。大小仍然是 138 位元組。
- P2 不做任何改變地轉送它。
- P3 移除兩個 micro-SID:它的 node SID 和它的 adjacency SID。然後它把封包從 P3 到 P5 的鏈路送出。
- P5 也同樣處理,送往 PE6。
- 回覆呢?它們走 PE6、P4、P2、PE1,依 IGP 路徑,送往 PE1 的 BLUE SID。只有 PE6 的匯出會加上 color。一條 policy 只導引一個方向。要導引兩個方向,另一端也要加上 color。

Ep 15.9 · When the policy goes down

來源 : lab-evidence/15-policy/v3/PE1-shut.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 100
  Admin: down Operational: down for 00:00:08 (since Oct  3 08:23:46.018)
  Preference: 200 (configuration) (inactive) (shutdown)
  Preference: 100 (configuration) (inactive) (shutdown)

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
  2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
    Extended community: Color:100 RT:65000:100

RP/0/RP0/CPU0:PE1#show cef vrf BLUE 10.1.6.0/24 detail
  via fcb:b00:6::/128, 11 dependencies, recursive
  next hop fcb:b00:6::/128 via fcb:b00:6::/48
  SRv6 H.Encaps.Red SID-list {fcb:b00:6:e004::}
```

- 最後一個測試:policy 失效時會發生什麼?我們把它 shutdown。
- policy 是 admin down。兩條候選路徑都是 down。PE1 釋放了 Binding SID。
- 路由仍然帶著 color 100。但 BGP 現在不再把它指向 policy。
- CEF 回到 PE6 的 locator 和單純的封裝,和第 13 集一樣。擷取顯示 PE1、P2、P4、PE6,所有 ping 都有回應。
- 在這個平台上,這種退回是預設行為,因為 invalidation drop 是關閉的。如果某個服務必須一直留在它的路徑上,例如為了隔離,就要開啟 invalidation drop。然後測試它確實會丟棄流量。
- no shutdown 之後,policy 恢復了,Binding SID 的值也相同。但這個值是動態的。不要做需要這個數字的設計。

Ep 16.1 · The template on PE1

來源：lab-evidence/16-odn/v3/A-fresh.cfg; configs/odn-srh/PE6.cfg

```
segment-routing
traffic-eng
on-demand color 200
  srv6
    locator FA128 binding-sid dynamic behavior ub6-insert-reduced
  !
dynamic
metric
  type latency

extcommunity-set opaque COLOR-200
200
end-set
vrf RED
address-family ipv4 unicast
export route-policy RED-EXPORT-C200
```

- 這是 template。首先,沒有 PCE。
- on-demand color 是 200。
- 它的 Binding SID 來自 Flex-Algo 的 locator,FA128。
- 意圖是一條最低延遲的動態路徑。
- 在 PE6 上,RED 的 IPv4 路由在匯出時帶有 color 200。所以現在每一條 RED IPv4 路由都要求 PE1 提供低延遲路徑。

Ep 16.2 · Test A: the headend computes

來源：lab-evidence/16-odn/v3/A4-PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 200 detail
Color: 200, End-point: 2001:db8:100::6
  Admin: up   Operational: down for 00:00:48 (since Oct  3 08:34:41.702)
  Preference: 200 (BGP ODN) (inactive)
    Last error: SRv6 SIDs failed verification
    Dynamic (inactive)
    Last error: HE computation not supported for SRV6 policy
      Metric Type: LATENCY (IANA PCEP/IGP: 12/1), Path Accumulated Metric: 0
  Preference: 100 (BGP ODN) (inactive)
    Last error: SRv6 SIDs failed verification
```

- 測試 A。沒有 PCE。PE1 必須自己計算路徑。
- PE1 為 PE6 建立了 policy,但 policy 是 down。
- 輸出給出了原因:Headend computation not supported for SRv6 policy。
- 第二條候選路徑,preference 100,也失敗了。我們稍後說明它的作用。

Ep 16.3 · Test A: what the traffic does

來源 : lab-evidence/16-odn/v3/A4-PE1.log, A-CE-A.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf RED 10.2.6.0/24 detail
  2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
    Extended community: Color:200 RT:65000:200

RP/0/RP0/CPU0:PE1#show cef vrf RED 10.2.6.0/24 detail
  via fcb:b00:6::/128, 7 dependencies, recursive
    next hop fcb:b00:6::/128 via fcb:b00:6::/48
    SRv6 H.Encaps.Red SID-list {fcb:b00:6:e006::}
    0      Y      GigabitEthernet0/0/0/0      fe80::5454:ff:fe66:75e1

CE-A:~$ sudo ping -c 3 -I 10.2.1.2 10.2.6.2
3 packets transmitted, 3 packets received, 0% packet loss
```

- RED 的流量會怎樣？
- 路由仍然帶著 color 200,但它沒有 policy。
- CEF 回到 PE6 的 locator 和單純的封裝,往 P2 送出。擷取顯示 PE1、P2、P4、PE6。這是成本最低的路徑。在這個實驗裡,它也是延遲最高的路徑。
- 所有 ping 都有回應。什麼都沒壞,所以沒有人發現問題。客戶付錢買低延遲,卻得到高延遲,而且沒有任何設備通知任何人。

Ep 16.4 · A minimal PCE

路由器 / 工具 : P4 · 來源 : lab-evidence/16-odn/v3/P4-state.log, PE1-state.log; configs/pce/P4.cfg

```
RP/0/RP0/CPU0:P4#show running-config pce
pce
  address ipv6 2001:db8:100::4

router isis CORE
  distribute link-state

RP/0/RP0/CPU0:PE1#show running-config segment-routing traffic-eng pcc
pcc
  source-address ipv6 2001:db8:100::1
  pce address ipv6 2001:db8:100::4

RP/0/RP0/CPU0:P4#show pce ipv6 peer
Peer address: 2001:db8:100::1
State: Up
Capabilities: Stateful, Segment-Routing, Update, Instantiation, SRv6, SR-P2MP
```

- 所以我們加上一個 PCE。在正式網路中,它是一台專用的伺服器或路由器。這裡由 P4 擔任 PCE,只要三行設定。
- P4 在它的 loopback 上成為 PCE。
- IS-IS 把它的拓撲資料庫交給 PCE,包括延遲和 SID。
- 在 PE1 上,使用者端這一側告訴 PE1 PCE 的位址。
- 會話是 up,兩端都支援 SRv6。

Ep 16.5 · Test B: the PCE computes

來源：lab-evidence/16-odn/v3/B-PE1.log

```
RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 200 detail
Admin: up Operational: up for 00:00:48 (since Oct  3 08:30:28.609)
Preference: 200 (BGP ODN) (inactive) (shutdown)
Preference: 100 (BGP ODN) (active)
Dynamic (pce 2001:db8:100::4) (valid)
Metric Type: LATENCY (IANA PCEP/IGP: 12/1), Path Accumulated Metric: 3000
  SID[0]: fcbb:bb00:1:e002::/64 Behavior: uA (PSP/USD) (57)
  SID[1]: fcbb:bb00:3:e004::/64 Behavior: uA (PSP/USD) (57)
  SID[2]: fcbb:bb00:5:e002::/64 Behavior: uA (PSP/USD) (57)
Binding SID: fcbb:bb01:1:e004::
```

- 測試 B。這是同一個 template,在 dynamic 底下加上 pcep。
- policy 是 up。
- 現在可以看出兩條候選路徑的作用。preference 200 是 headend 上的計算,它是關閉的。preference 100 委派給 PCE,它是 active。
- 總延遲是 3000 微秒。
- 路徑有三個 adjacency SID:PE1 到 P3、P3 到 P5、P5 到 PE6。三個都是受保護的類型。
- Binding SID 來自 FA128 locator,如 template 所指定的。

Ep 16.6 · Test B in CEF

來源 : lab-evidence/16-odn/v3/B-PE1.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf RED 10.2.6.0/24 detail
 2001:db8:100::6 C:200 (bsid: fcbb:bb01:1:e004::) (metric 30) from 2001:db8:100::6
  SR policy color 200, up, registered, bsid fcbb:bb01:1:e004::

RP/0/RP0/CPU0:PE1#show cef vrf RED 10.2.6.0/24 detail
 via local-srv6-sid fcbb:bb01:1:e004::, 3 dependencies, recursive
  SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e006::}
  SRv6 H.Insert.Red SID-list {fcbb:bb00:e004:e002::}
  0      Y      GigabitEthernet0/0/0/1      fe80::5454:ff:fecl:58eb
```

- 導流的方式和第 15 集一樣。
- RED 的路由現在指向 ODN policy 的 Binding SID。
- 這是 RED 的 service SID e006,以及插入的 list:fcbb:bb00:e004:e002::。三個鄰接只有兩個 micro-SID,而且沒有節點編號。我們現在來解讀它。
- 第一跳是 GigabitEthernet 0/0/0/1,直接送往 P3。

Ep 16.7 · Test B on the wire

路由器 / 工具 : CE-A and tshark · 來源 : lab-evidence/16-odn/v3/B-CE-A.log, B-*.pcap

```
CE-A:~$ sudo ping -c 3 -I 10.2.1.2 10.2.6.2
3 packets transmitted, 3 packets received, 0% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8 or icmp.type == 0" -T fields -e ipv6.dst -e ipv6.hlim -e icmp.type
# PE1 to P3      fcb:b00:e004:e002:6:e006::      255    request
# P3 to P5      fcb:b00:e002:6:e006::          254    request
# P5 to PE6     fcb:b00:6:e006::                253    request
# PE1 to P2     fcb:b00:1:e007::                253    reply
# P2 to P4      fcb:b00:1:e007::                254    reply
# P4 to PE6     fcb:b00:1:e007::                255    reply
```

- 現在看線路,CE-A 在 RED 裡 ping CE-B。我們用 tshark 讀取每個擷取檔。
- 三個 ping 都有回應。從 PE1 到 P3,位址是滿的。
- P3 移除 e004,把封包送上它到 P5 的鏈路。
- P5 移除 e002,把封包送往 PE6。這是三跳,走低延遲路徑。
- 上面那排鏈路只載送回應封包,走的是 IGP 路徑。和第 15 集一樣,color 只導引一個方向。

Ep 16.8 · The PCE's view

路由器 / 工具 : P4 · 來源 : lab-evidence/16-odn/v3/B-P4.log

```
RP/0/RP0/CPU0:P4#show pce lsp detail
PCC 2001:db8:100::1:
Tunnel Name: bgp_c_200_ep_2001:db8:100::6_discr_100
  State: Admin up, Operation active
  Setup type: SRv6
  Binding SID: fcb:b01:1:e004:: Behavior: uB6 (PSP/USD Insert.Red) (79)
  Prefix-SID algorithm: 0 (set: 0)
  PCEP information:
    PLSP-ID 0x2, flags: D:1 S:0 R:0 A:1 O:2 C:0
  Computed path: (Local PCE)
    Metric type: LATENCY (12), Accumulated Metric 3000
```

- 這是同一條 policy,從 PCE 的角度來看。
- P4 知道 PE1 這條 ODN policy 的名稱,而且 policy 是 active。
- 看這一行:prefix-SID algorithm 0。這條路徑用的是一般 algorithm 0 的 adjacency SID,依延遲挑選。它不是 Flex-Algo 路徑。
- D 旗標表示 PE1 把這條路徑的控制權委派給了 PCE。
- PCE 在本地計算出這條路徑,延遲 3000。

Ep 16.9 · Test C: add sid-algorithm 128

來源 : lab-evidence/16-odn/v3/C-algo.cfg, C-PE1.log, C-P4.log

```
on-demand color 200
dynamic
  pcep
  metric
    type latency
constraints
  segments
    sid-algorithm 128

RP/0/RP0/CPU0:PE1#show segment-routing traffic-eng policy color 200 detail
Admin: up Operational: down for 00:01:17 (since Oct  3 08:37:24.784)
Preference: 100 (BGP ODN) (inactive)
Last error: SRv6 SIDs failed verification
Prefix-SID Algorithm: 128

RP/0/RP0/CPU0:P4#show pce lsp detail
State: Admin up, Operation down
Prefix-SID algorithm: 128 (set: 1)
Computed path: (Local PCE)
```

- 測試 C。ODN policy 能不能使用 Flex-Algo 128 的 SID?這樣路徑會跟著這個演算法走,修復也跟著它。
- 我們在 template 裡多加一個區塊:segment 必須來自 algorithm 128。
- policy 變成 down,訊息是:SRv6 SIDs failed verification。
- 在 P4 上,請求帶著 algorithm 128 抵達,但計算出的路徑是空的。RED 的流量又一次退回最短路徑,沒有任何警告,和測試 A 一樣。

Ep 16.10 · The PCE has the data

路由器 / 工具 : P4 · 來源 : lab-evidence/16-odn/v3/C-P4-topo.log

```
RP/0/RP0/CPU0:P4#show pce ipv6 topology
Node 7
  Host name: P5
    SR Algo Participation:
      0, 1, 128
    SID: fcb:b01:5:: Behavior: uN (PSP/USD) (48)
    LBL:32 LNL:16 FL:0 AL:80 Algo:128
  Link[0]: local address 2001:db8:200:35::1, remote address 2001:db8:200:35::
    Metric: IGP 40, TE 40, Latency 1000 microseconds
    SID[2]: fcb:b01:5:e000:: (protected) Behavior: uA (PSP/USD) (57)
    LBL:32 LNL:16 FL:16 AL:64 Algo:128
  Flex-algo Metric: Min Latency 1000
```

- PCE 有全部的資料嗎?這是測試 C 期間它的拓撲資料庫。
- P5 參與 algorithm 128。
- 它的 algorithm 128 node SID 在資料庫裡。
- 它的 algorithm 128 adjacency SID 和鏈路延遲也都在裡面。每個節點都是如此。資料是完整的,但平台不支援這種計算。
- 我們也試了另一個使用 XRv9000 路由器的實驗。結果是同樣的失敗。

Ep 16.11 · Way one, re-checked today

來源 : lab-evidence/16-odn/v3/alt1-PE1.log, alt1-*.pcap (summarised)

```
RP/0/RP0/CPU0:PE1#show route vrf RED ipv6 2001:db8:c:206::/64 detail
      SRv6 Headend: H.Encaps.Red [f3216], SID-list {fcbb:bb01:6:e004::}

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  Known via "isis CORE", distance 115, metric 3001, SRv6-locator (algo 128), type level-2
    fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Protected

Captured, ping6 from CE-A to CE-B in RED:
  PE1 to P3    fcbb:bb01:6:e004::    hop limit 255
  P3 to P5     fcbb:bb01:6:e004::    hop limit 254
  P5 to PE6    fcbb:bb01:6:e004::    hop limit 253
```

- 我們今天在同一個實驗上重新檢查了第一種方法。
- RED IPv6 使用 PE6 在 FA128 block,也就是 fcbb:bb01 裡的 SID。
- IS-IS 在 algorithm 128 裡路由這個 locator,metric 3001,經過 P3。它有同一個演算法內的 TI-LFA 保護。
- 在線路上,目的位址不會改變。沒有路由器移除 micro-SID。每台路由器只是在 algorithm 128 裡路由 fcbb:bb01:6。這是最簡單的方法。

Ep 16.12 · A trap: up, but not on Algo 128

來源：configs/color-flexalgo/PE1.cfg; lab-evidence/16-odn-srh/color-flexalgo/PE1-verify.log, c400-*.pcap

```
segment-list PE6-ALGO128
  srv6
    index 10 sid fcbb:bb01:6::
policy GREEN-C400
  color 400 end-point ipv6 2001:db8:100::6

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
  SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
    0      Y      GigabitEthernet0/0/0/1      fe80::5454:ff:fecl:58eb

Captured: PE1 to P3, then P4 to PE6. Nothing on P3 to P5 or P5 to PE6.
```

- 再看一個案例。我們能不能寫一條只有一個 segment 的顯式 policy,也就是 PE6 的 algorithm 128 node SID?
- color 是 400,只有一個 segment:fcbb:bb01:6::。policy 成功 up 了。
- 但看看 CEF。它只顯示 service SID,而且來自一般的 block。PE1 不安裝最終 node SID,如第 15 集所示。所以封包裡沒有任何關於 algorithm 128 的資訊。
- 封包往 P3 送出。然後它依 algorithm 0 從 P4 到達 PE6。policy 是 up,但路徑不是它名字所說的那條。我們已經移除它。

Ep 17.1 · Layer 1: local, on the egress

路由器 / 工具 : PE6 · 來源 : lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 locator
Name          ID      Algo Prefix          Status  Flags
FA128         1       128 fcb::bb01::/48   Up      U
MAIN          2        0 fcb::bb00::/48   Up      U

RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
Block fcb::bb01::/32:
  Number of SIDs free: 7675
  Max SIDs: 7680
  Status: Resource Available
```

- 第一層,在 PE6 上。PE6 擁有這些 SID。
- 兩個 locator 都是 up:MAIN 屬於 algorithm 0,FA128 屬於 algorithm 128。健康的跡象是狀態為 Up。
- manager 顯示每個 block 的 SID 資源,還有很多可用。第 19 集會再用到這些數字。如果 locator 是 down 或 block 已滿,就停在這裡。上面的層次都無法運作。

Ep 17.2 · Layer 2: the IGP

來源：lab-evidence/17-oam/v3/L-PE6.log, L-PE1.log

```
RP/0/RP0/CPU0:PE6#show isis segment-routing srv6 locators
Name          ID      Algo  Prefix          Status
FA128         1       128   fcbb:bb01:6::/48 Active
MAIN          2        0   fcbb:bb00:6::/48 Active

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
    fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)
  Repair Node(s): 2001:db8:100::5
```

- 第二層是 IGP。我們在兩端檢查它。
- 在 PE6 上,IS-IS 顯示兩個 locator 都是 active。第 3 集的故障 A 就壞在這裡。locator 在本地是 up,但 IS-IS 沒有通告它。
- 在 PE1 上,locator 是一條來自 IS-IS 的路由,metric 31。
- 主要路徑顯示 protected,還有一條經由修復節點 P5 的 TI-LFA 備援。健康的跡象是入口上有這條路由。
- ping loopback 不能取代這個檢查。在第 3 集,ping loopback 正常,但服務卻無法運作。

Ep 17.3 · Layer 3: forwarding

來源：lab-evidence/17-oam/v3/L-PE1.log, L-PE6.log

```
RP/0/RP0/CPU0:PE1#show cef ipv6 fcbb:bb00:6::/48
fcbb:bb00:6::/48, version 130, SRv6 Headend
  via fe80::5454:ff:fe66:75e1/128, GigabitEthernet0/0/0/0, 8 dependencies, protected
  via fe80::5454:ff:fecl:58eb/128, GigabitEthernet0/0/0/1, 13 dependencies, backup (TI-LFA)
  next hop fe80::5454:ff:fecl:58eb/128, Repair Node(s): 2001:db8:100::5
  SRv6 H.Insert.Red SID-list {fcbb:bb00:3:e005::}

RP/0/RP0/CPU0:PE6#show cef ipv6 fcbb:bb00:6:e004::/64
fcbb:bb00:6:e004::/64, version 136, SRv6 Endpoint uDT4
  next hop VRF - 'BLUE', table - 0xe0000002
    0      Y      recursive          Lookup in table
```

- 第三層是轉送。RIB 顯示路由器想做什麼。CEF 顯示它實際做什麼。
- 在 PE1 上,主要路徑經過 P2。dependencies 是透過它解析的服務路由。
- 備援路徑也已經安裝,帶著它的修復 segment,和第 9 集一樣。
- 在 PE6 上,service SID 的項目是在 BLUE 裡查表。健康的跡象是入口有介面,出口指向正確的 VRF。在第 3 集,轉送表顯示的是 unresolved 和 drop。

Ep 17.4 · Layer 4: the service

來源：lab-evidence/15-policy/v3/PE1-steer.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.6.0/24 detail
2001:db8:100::6 C:100 (bsid: fcbb:bb00:1:e008::) (metric 30) from 2001:db8:100::6
Received Label 0xe0040
Extended community: Color:100 RT:65000:100
SR policy color 100, up, not-registered, bsid fcbb:bb00:1:e008::
SID Locator: fcbb:bb00:6::/48, RIB metric: 31
T:1(Sid information), Sid:fcbb:bb00:6::(Transposed), Behavior:63
```

- 第四層是服務。一個指令就能看到大部分內容。這是第 15 集的 BLUE。
- 這是 SID,連同它的 label 和行為。如果它不正確或不存在,就表示出口沒有分配它。這是第 3 集的故障 B。
- 這是 color 和 route target。route target 不正確時,VRF 什麼都不會匯入,和第 12 集一樣。
- 如果路由有 color,就看 policy。記住第 15 和 16 集:policy 可以是 up,卻做了你不想要的事。
- 這是 locator 檢查,metric 31。如果它顯示 inaccessible,就回到第二層。

Ep 17.5 · Layer 5: end to end

來源 : lab-evidence/17-oam/v3/PE1-oam.log

```
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.1.6.2 source 10.1.1.1 count 3
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms

RP/0/RP0/CPU0:PE1#traceroute vrf BLUE 10.1.6.2 source 10.1.1.1 probe 1 timeout 1 maxttl 5
 1  10.46.0.1 3 msec
 2  10.1.6.2 2 msec
```

- 第五層是端到端。
- 我們在 VRF 裡,從 PE1 的 BLUE 位址 ping CE-B。它有回應。
- 接著 VRF traceroute 顯示 PE6,然後是 CE-B。如第 13 集所示,在 VRF 裡看不到核心。
- 小 ping 通過,只表示小封包在一個方向上能通過。在第 7 和 14 集,只有大封包才顯現出故障。要用不同大小測試。如果不確定,就擷取封包。

Ep 17.6 · OAM: pinging SIDs

來源 : lab-evidence/17-oam/v3/PE1-oam.log

```
RP/0/RP0/CPU0:PE1#ping fcb::bb00:6:: source 2001:db8:100::1 count 3
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
RP/0/RP0/CPU0:PE1#ping fcb::bb00:6:e000:: source 2001:db8:100::1 count 3
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
RP/0/RP0/CPU0:PE1#ping fcb::bb00:6:e004:: source 2001:db8:100::1 count 3 timeout 1
!!!
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/4/5 ms
```

- 接下來是維運工具。首先,你能 ping 一個 SID 嗎?
- PE6 的 node SID 有回應。
- PE6 的一個 adjacency SID 有回應。
- BLUE 的 service SID,一個 uDT4,也有回應。

Ep 17.7 · OAM: traceroute

來源：lab-evidence/17-oam/v3/PE1-oam.log, PE1-trace-srv6.log

```
RP/0/RP0/CPU0:PE1#traceroute ipv6 fcb:b00:6:: source 2001:db8:100::1 probe 1 timeout 1 maxttl 6
Tracing the route to fcb:b00:6::
 1  2001:db8:200:12::1 3 msec
 2  2001:db8:200:24::1 2 msec
 3  2001:db8:100::6 2 msec

RP/0/RP0/CPU0:PE1#traceroute ipv6 fcb:b00:6:: source 2001:db8:100::1 probe 1 timeout 1 maxttl 6 srv6
% Invalid input detected at '^' marker.
RP/0/RP0/CPU0:PE1#traceroute ipv6 fcb:b00:6:: srv6
% Invalid input detected at '^' marker.

! In the help tree of this release (not run):
traceroute segment-routing srv6 policy name <name>
```

- 接下來是 traceroute。我們說明四點。
- 第一點:一般的 traceroute 可以使用。對 PE6 的 node SID,它顯示 P2、P4,然後是 PE6,走 IGP 路徑。
- 第二點:Cisco 記載了兩種 SRv6 形式的 traceroute。XRd 26.2.2 的 ipv6 說明樹裡都沒有。我們的說法只限於 ipv6 這種語法。
- 第三點:這個版本有 traceroute segment-routing srv6 policy name,後面接名稱。它用來追蹤 policy。我們沒有執行它。
- 第四點:文件不一定和你的版本相符。在你的路由器上使用問號,並測試語法。

Ep 17.8 · Traceroute into a container

來源：lab-evidence/17-oam/PE1-oam.log

```
RP/0/RP0/CPU0:PE1#traceroute fcbb:bb00:3:5:6:: source 2001:db8:100::1
 1  2001:db8:200:12::1 3 msec 1 msec 1 msec
 2  2001:db8:100::3 1 msec 1 msec 1 msec
 3  2001:db8:200:23:: 1 msec 1 msec 1 msec
 4  2001:db8:200:24::1 2 msec 1 msec 1 msec
 5  2001:db8:100::5 2 msec 4 msec 1 msec
 6  2001:db8:200:45:: 1 msec 1 msec 1 msec
 7  * * *
 8  * * *
```

- 現在對一個我們手動組成的位址做 traceroute:fcbb:bb00:3:5:6::。這是第 15 集的鬆散路徑。
- 第一跳是 P2。
- 第二跳是 P3。它從 loopback 回應,因為這個位址是它的 node SID。接著第三跳又是 P2。這是第 15 集的第一次迴轉,不用擷取就看得得到。
- 然後是 P4,接著是從 loopback 回應的 P5,然後又是 P4。這是第二次迴轉。
- PE6 沒有回應。這和第 4、5 集一致。探測封包的 checksum 使用最初的目的位址,所以最後一個節點把它丟掉。我們看到了路徑,但看不到最後一跳。

Ep 18.1 · Fault 1: capture the path

路由器 / 工具 : CE-A and tshark · 來源 : lab-evidence/18-faults/F1-CE-A.log, F1-PE1-P2.pcap, F1-P2-P4.pcap, F1-P4-PE6.pcap

```
CE-A:~$ sudo ping -c 2 -s 1300 172.16.30.6
2 packets transmitted, 2 packets received, 0% packet loss
CE-A:~$ sudo ping -c 2 -s 1400 172.16.30.6          (no reply)

$ tshark -r <capture file> -Y "ipv6.nxt == 143 or icmpv6.type == 2" -T fields -e frame.len -e ipv6.src -e ipv6.dst
# PE1 to P2
1396    2001:db8:100::1    fcb:b00:6:e007::    1300-byte ping, request
1396    2001:db8:100::6    fcb:b00:1:e00b::    1300-byte ping, reply
1496    2001:db8:100::1    fcb:b00:6:e007::    1400-byte ping, request
1294    2001:db8:100::1    (to PE1's loopback) Packet Too Big, MTU 1336
# P2 to P4, and P4 to PE6
1396    the 1300-byte request and reply only
```

- 第五層告訴我們該看哪裡:線路。CE-A 用三種大小 ping 的同時,我們擷取了三條核心鏈路。我們用 tshark 讀取每個檔案。
- 1300 位元組的 ping 有回應。1400 位元組的 ping 沒有回應。
- 1300 位元組的 ping 是 1396 位元組的訊框,而且在兩個方向都穿越核心。
- 1400 位元組的請求以 1496 位元組的訊框離開 PE1,並到達 P2。
- P2 送出了一個 ICMPv6 Packet Too Big,送往封裝來源,也就是 PE1 的 loopback。對二層訊框來說,PE1 無法使用這個訊息。所以客戶收不到任何訊息。
- 在 P2 到 P4 以及 P4 到 PE6 的鏈路上,只有小訊框。大訊框在 P2 就停了。所以故障在 P2,在它通往 P4 的鏈路上。

Ep 18.2 · Fault 1: the cause

路由器 / 工具 : P2 · 來源 : lab-evidence/18-faults/F1-P2.log, F1-PE1-P2.pcap, F1-fix.cfg

```
RP/0/RP0/CPU0:P2#show ipv6 interface GigabitEthernet0/0/0/1
  MTU is 1514 (1400 is available to IPv6)

Packet Too Big, captured on PE1 to P2:
  Type: Packet Too Big (2)
  MTU: 1336
    Source Address: 2001:db8:100::1
    Destination Address: fcbb:bb00:6:e007::

Fix on P2:
interface GigabitEthernet0/0/0/1
  no ipv6 mtu
```

- P2 上的第三層顯示了原因。
- 介面 MTU 仍然是 1514,但 IPv6 只能用 1400。有人在這條鏈路上設定了 IPv6 MTU。這就是我們埋下的故障。在真實網路中,它常常是某一條鏈路上的變更,沒有人把它和客戶聯想在一起。
- Packet Too Big 包含 PE1 的外層封包,目的是 PE6 的二層 SID。它的 MTU 欄位顯示 1336,而不是 1400。
- 我們之後在 XRd 和 Xrv9000 上測試過。當目的路由有 TI-LFA 修復路徑時,兩種平台都顯示 MTU 減 64。沒有修復路徑時,這個值是正確的。
- 轉送一律使用真正的 MTU。我們認為這 64 位元組是留給修復標頭的空間,但 Cisco 沒有文件說明。要相信擷取,而不只是回報的數值。
- 我們刪掉一行,1418 位元組的 ping 又通了。教訓:要用不同大小測試。某個大小失敗時,就逐條鏈路擷取,直到封包消失為止。

Ep 18.3 · Fault 1: the configuration change

路由器 / 工具 : P2 · 來源 : lab-evidence/18-faults/F1-break.cfg

```
! On P2, toward P4
interface GigabitEthernet0/0/0/1
  ipv6 mtu 1400

# Largest customer IP packet over E-Line with this limit:
# 1400 - 40 (outer IPv6 header) - 14 (inner Ethernet header) = 1346 bytes
```

- 這是 P2 上的變更。
- 一條指令把通往 P4 那條鏈路的 IPv6 MTU 設為 1400。介面 MTU 仍然是 1514。
- 算式:1400 減去外層 IPv6 標頭的 40,再減去內層 Ethernet 標頭的 14,等於 1346。大於 1346 位元組的客戶封包無法通過這條鏈路。這和 ping 的結果一致:1300 通過,1400 失敗。

Ep 18.4 · Fault 2: one direction

路由器 / 工具：CE-A and tshark · 來源：lab-evidence/18-faults/F2-CE-A.log, F2-*.pcap

```
CE-A:~$ sudo ping -c 3 -W 2 -I 10.1.1.2 10.1.6.2
3 packets transmitted, 0 packets received, 100% packet loss

$ tshark -r <capture file> -Y "icmp.type == 8 or icmp.type == 0" -T fields -e ipv6.dst -e icmp.type | sort | uniq -c
# CE-A to PE1      3 requests
# PE1 to P2        3 requests    fcbb:bb00:3:5:6:e004::
# P4 to PE6        3 requests    fcbb:bb00:6:e004::
# PE6 to CE-B      3 requests, and 3 replies from CE-B
```

- 故障 2。BLUE 無法運作:CE-A ping 不到 CE-B。在 PE1 上,CE-B prefix 的每一層看起來都正常。所以我們直接去看線路。
- 三個請求離開 CE-A,沒有任何回應回來。
- 請求穿越核心,送往 PE6 上 BLUE 的 service SID。
- CE-B 收到它們並回覆三次。回覆到達 PE6,就停在那裡。去程方向是正確的。回程方向壞了。ping 無法顯示是哪個方向壞了。擷取可以。

Ep 18.5 · Fault 2: layer 4, on the other PE

來源：lab-evidence/18-faults/F2-PE6.log, F2-PE1.log

```
RP/0/RP0/CPU0:PE6#show route vrf BLUE 10.1.1.0/24
% Network not in table
RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast rd 65000:101 10.1.1.0/24
Paths: (0 available, no best path)

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf BLUE 10.1.1.0/24 detail
  Advertised to peers (in unique update groups):
    2001:db8:100::6
      Extended community: RT:65000:110
```

- 回程流量由 PE6 送出。所以我們在 PE6 上檢查 CE-A prefix 的各個層次。
- PE6 的 BLUE 裡沒有通往 CE-A 子網路的路由。
- BGP 裡也沒有它的路徑。
- 但 PE1 顯示它已經把路由通告給 PE6, route target 是 65000:110。PE6 匯入的是 65000:100。差了一個數字。
- PE6 收到了路由, 找不到相符的 VRF, 於是把它丟掉。我們把 PE1 的 export 改回 65000:100, 故障就修好了。
- 教訓: ping 失敗時, 要檢查兩個方向。要檢查回應停住的那台 PE, 即使回報問題的客戶在另一邊。

Ep 18.6 · Fault 3: one VRF dead

來源：lab-evidence/18-faults/F3b-PE1.log, F3c-PE1.log

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 0 percent (0/3)
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.1.6.2 source 10.1.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/2 ms
RP/0/RP0/CPU0:PE1#ping vrf RED 10.2.6.2 source 10.2.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 3/5/7 ms

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    SID Locator: fcbb:bb00:66::/48, RIB metric: inaccessible
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32
    unresolved
    SRv6 H.Encaps.Red SID-list {fcbb:bb00:66:e008::}
    0      Y      recursive                drop
```

- 故障 3。只有 GREEN 無法運作。
- GREEN 失敗。
- 送往同一台 PE 的 BLUE 和 RED 都通過。所以 PE6 可達,它的主要 locator 也正常。
- GREEN 的第四層:SID 的 locator 是 fcbb:bb00:66::/48。它不是 PE6 的 MAIN locator,而且它是 inaccessible。
- CEF 顯示 unresolved 和 drop。這和第 3 集的故障 A 一樣,只是只影響一個 VRF。

Ep 18.7 · Fault 3: down to layers 1 and 2

路由器 / 工具 : PE6 · 來源 : lab-evidence/18-faults/F3-PE6.log, F3c-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 locator
FA128      1      128    fcbb:bb01:6::/48      Up      U
MAIN       2       0    fcbb:bb00:6::/48      Up      U
TEST       3       0    fcbb:bb00:66::/48     Up      U

RP/0/RP0/CPU0:PE6#show isis segment-routing srv6 locators
FA128      1      128    fcbb:bb01:6::/48      Active
MAIN       2       0    fcbb:bb00:6::/48      Active
TEST       3       0    fcbb:bb00:66::/48     Inactive

RP/0/RP0/CPU0:PE6#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
  SRv6-VPN SID: fcbb:bb00:66:e008::/64
  Alloc Mode/Locator ID: per-vrf/3
```

- inaccessible 指向第二層。所以我們去找擁有者 PE6。
- 第一層顯示第三個 locator,TEST。它在本地是 up。
- 第二層:IS-IS 知道這個 locator,但顯示為 inactive。沒有人把它加進 IS-IS。所以其他路由器都到不了。
- GREEN 的 BGP 設定從它分配 SID,locator ID 3。這是一個沒有做完的變更。有人建立了新的 locator,把 VRF 移過去,卻沒有做 IGP 那一步。
- 我們把 GREEN 移回 MAIN,故障就修好了。變更期間,大約有四十秒,PE1 還在使用舊的 SID,而 PE6 已經把它刪除了。
- 更換 SID 本身也是一次中斷。要事先規劃。

Ep 18.8 · Fault 3: the configuration changes

路由器 / 工具 : PE6 · 來源 : lab-evidence/18-faults/F3-break.cfg, F3b-break.cfg (the nesting is shortened)

```
! Step 1, on PE6: a new locator, and RED allocates from it
segment-routing srv6 locators locator TEST
  micro-segment behavior unode psp-usd
  prefix fcbb:bb00:66::/48
router bgp 65000 vrf RED address-family ipv4 unicast
  segment-routing srv6 locator TEST
! (the locator is NOT added under router isis: the missing step)

! Step 2, on PE6: RED goes back to MAIN, and GREEN moves to TEST
router bgp 65000 vrf RED address-family ipv4 unicast
  segment-routing srv6 locator MAIN
router bgp 65000 vrf GREEN address-family ipv4 unicast
  segment-routing srv6 locator TEST
```

- 這是故障 3 的設定變更,全部在 PE6 上。我們分兩步做。
- 第一步建立 locator TEST,並讓 RED IPv4 從它分配 service SID。
- 請看最後一行。這個 locator 沒有加到 IS-IS 底下。這就是漏掉的步驟。所以沒有任何其他路由器有通往它的路由。
- 第二步把 RED 放回 MAIN,並把 GREEN 移到 TEST。現在只有 GREEN 依賴壞掉的 locator。這一頁之後的插曲會說明為什麼 RED 在第一步時沒有失效。

Ep 18.9 · Fault 3: the repair

路由器 / 工具 : PE6 · 來源 : lab-evidence/18-faults/F3-fix1.cfg, F3-fix2.cfg (the nesting is shortened)

```
! Repair 1, on PE6: GREEN back to MAIN
router bgp 65000 vrf GREEN address-family ipv4 unicast
  segment-routing srv6 locator MAIN

! Repair 2, on PE6: remove the unused locator
segment-routing srv6 locators
  no locator TEST
```

- 現在是修復,同樣在 PE6 上,分兩步。
- 修復一把 GREEN 移回 MAIN locator。新的 SID 通告之後,流量就恢復了。
- 修復二刪除沒有人使用的 locator TEST。這要最後才做。要在沒有任何 VRF 從它分配之後,才刪除 locator。

Ep 18.10 · Fault 4: hidden until a link fails

來源：lab-evidence/18-faults/F4-before.log, F4-latent.log

Before the change:

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Backup (TI-LFA)
  fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Protected
```

After the change on P2 and P4, everything still passes:

```
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
  fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1
```

- 故障 4 是最難的一種。做變更的當下,什麼都沒壞。我們從 P2 和 P4 移除了 Flex- Algo 128。工程師可能為了簡化這兩台路由器而做這種變更。RED IPv6 使用 algorithm 128 locator,它照常運作。
- 變更之前,PE1 通往 PE6 algorithm 128 locator 的路由是受保護的,有一條經過 P2 的 TI-LFA 備援。
- 變更之後,只剩一條路徑,沒有保護。algorithm 128 的拓撲變成一條直線:PE1、P3、P5、PE6。所有 ping 仍然都通。
- 只有第二層顯示出問題,而且只有在和參考資料比對時才看得到。

Ep 18.11 · Fault 4: the link fails

來源 : lab-evidence/18-faults/F4-event-*.log

```
P3: interface GigabitEthernet0/0/0/1 shutdown    (the link to P5)

CE-A:~$ sudo ping6 -c 3 -W 2 -I 2001:db8:c:201::2 2001:db8:c:206::2
3 packets transmitted, 0 packets received, 100% packet loss

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb01:6::/48
% Network not in table
RP/0/RP0/CPU0:PE1#show cef vrf RED ipv6 2001:db8:c:206::/64
    unresolved
    0      Y    recursive          drop

BLUE: 2 of 2.  RED IPv4: 2 of 2 (the PCE recomputed: latency 15000, via PE6's node SID).
```

- 現在發生拓撲事件:P3 到 P5 的鏈路故障。
- 我們在 P3 上把它關閉。
- RED IPv6 無法運作了。
- 第二層:algorithm 128 的 locator 不見了。P2 和 P4 不在這個演算法裡,所以演算法裡沒有其他路徑。第三層顯示 unresolved 和 drop。
- 其他服務都繼續運作。BLUE 使用 algorithm 0。RED IPv4 使用 PCE 的 policy,PCE 在剩下的鏈路上計算了新的路徑。
- 付費購買特殊路徑的客戶,正是失去服務的客戶。我們把鏈路和演算法恢復,路由又重新受到保護。

Ep 18.12 · The four configuration changes

```
! fault 1, on P2: a smaller IPv6 MTU toward P4
interface GigabitEthernet0/0/0/1
  ipv6 mtu 1400
! fault 2, on PE1: a different export route target
vrf BLUE
  address-family ipv4 unicast
    export route-target
      no 65000:100
      65000:110
! fault 3, on PE6: a locator that IS-IS does not advertise
! (full configuration on the Fault 3 pages)
locator TEST prefix fcbb:bb00:66::/48
router bgp 65000 vrf RED address-family ipv4 unicast
  segment-routing srv6 locator TEST
! fault 4, on P2 and P4
router isis CORE
  no flex-algo 128
```

- 這是四個變更,也就是解答。你可以在自己的實驗裡重做每一個,先用五個層次把它找出來,再來看這一頁。
- 故障 1:在 P2 上,通往 P4 的鏈路,IPv6 MTU 是 1400。
- 故障 2:在 PE1 上,BLUE IPv4 的匯出 route target 從 65000:100 改成 65000:110。
- 故障 3:在 PE6 上,RED 從一個新的 locator 分配 service SID。IS-IS 沒有通告這個 locator。
- 故障 4:在 P2 和 P4 上,我們移除 Flex-Algo 128。在鏈路故障之前,服務一切正常。

Ep 19.1 · The untrusted port

來源：configs/boundary/PE1-outside.cfg; lab-evidence/19-boundary/v3/CE-A-setup.log

```
interface GigabitEthernet0/0/0/4
  description UNTRUSTED outside
  ipv6 address 2001:db8:ffff:1::1/64

Outside host, on CE-A's third port:
CE-A:~$ sudo ip -6 addr add 2001:db8:ffff:1::2/64 dev eth2
CE-A:~$ sudo ip -6 route add fcb:b00::/31 via 2001:db8:ffff:1::1
CE-A:~$ sudo ip -6 route add 2001:db8:100::/64 via 2001:db8:ffff:1::1
```

- 現在開始測試。第 14 集二層服務的那個埠,變成 PE1 全域路由表裡一個不受信任的埠。在真實網路中,它可能是網際網路互連、管理網路,或是一個會犯錯的客戶。
- 它有一個普通的 IPv6 位址,沒有 VRF。
- 外部主機在這條鏈路上拿到一個位址。
- 主機也拿到經由 PE1 通往兩個 SID block 和 loopback 的路由。真正的攻擊者不需要這些路由。一條預設路由就有同樣的效果。

Ep 19.2 · Before: the outside reaches the core

路由器 / 工具 : CE-A · 來源 : lab-evidence/19-boundary/v3/before-CE-A.log

```
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:6::  
64 bytes from 2001:db8:100::6: seq=0 ttl=61 time=3.693 ms  
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:4::  
64 bytes from 2001:db8:100::4: seq=0 ttl=62 time=1.757 ms  
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:6:e000::  
64 bytes from 2001:db8:100::6: seq=0 ttl=61 time=1.801 ms  
CE-A:~$ sudo ping6 -c 2 -W 2 fcbb:bb00:6:e004::  
64 bytes from 2001:db8:100::6: seq=0 ttl=61 time=1.656 ms
```

- 還沒有過濾。外部主機嘗試核心的 SID。
- PE6 的 node SID 有回應,由 PE6 的 loopback 回應。
- P4 這台轉送路由器的 node SID,有回應。
- PE6 的一個 adjacency SID 有回應。
- BLUE 的 service SID 也有回應。一個陌生人從網路外部,碰到了通往客戶 VRF 的那扇門。
- 如第 17 集所示,ping 不會觸發 VRF 查表。但它表示外部的封包能到達這個 SID。一台連內層封包也一起偽造的主機,不需要更多條件。我們沒有做那種攻擊。也沒有必要。

Ep 19.3 · Before, on the wire

路由器 / 工具 : tshark · 來源 : lab-evidence/19-boundary/v3/before-PE1-P2.pcap, before-P4-PE6.pcap

```
$ tshark -r <capture file> -Y "icmpv6.type == 128 or icmpv6.type == 129" \
-T fields -e ipv6.src -e ipv6.dst -e icmpv6.type | sort | uniq -c
# PE1 to P2: requests and replies
 2  2001:db8:ffff:1::2    fcb:b00:6::      128
 2  2001:db8:ffff:1::2    fcb:b00:4::      128
 2  2001:db8:100::6      2001:db8:ffff:1::2  129
 2  2001:db8:100::4      2001:db8:ffff:1::2  129
# P4 to PE6
 2  2001:db8:ffff:1::2    fcb:b00:6::      128
 2  2001:db8:100::6      2001:db8:ffff:1::2  129
```

- 這是那次測試的擷取結果。我們用 tshark 讀取每個檔案。type 128 是請求,type 129 是回應。
- 在 PE1 到 P2 的鏈路上,外部主機的請求原封不動地進入核心,送往那些 SID。路由器的 loopback 回應了。
- 同樣的請求也到達了 PE6。邊緣沒有任何東西擋住一個從 domain 外部送往 domain 內部指令的封包。

Ep 19.4 · What the platform lists

路由器 / 工具：PE6 · 來源：lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
Security rules:
SEC-1
SEC-2
SEC-3
```

- 路由器會保護自己嗎?SRv6 manager 列出了安全規則。
- 這是 SEC-1 到 SEC-3。這些名稱來自 RFC 8754 第 5.1 節。它描述在 domain 邊緣和 SR 節點上的過濾。
- 這份清單顯示軟體支援什麼。我們的測試顯示沒有自己的過濾時的結果:每一個 SID 都回應了外部主機。清單裡的規則不等於已設定的規則。第 14 集對能力清單也教過同樣的一課。

Ep 19.5 · The boundary ACL

來源：configs/boundary/PE1-acl.cfg

```
ipv6 access-list SRV6-BOUNDARY-IN
 10 deny ipv6 any fcbb:bb00::/31 log
 20 deny ipv6 any any routing log
 30 permit ipv6 any any
!
interface GigabitEthernet0/0/0/4
 ipv6 access-group SRV6-BOUNDARY-IN ingress
```

- 修正方法是在每一個面向 SR domain 外部的介面上套用存取清單。
- 第 10 行:任何外部封包都不能送往 SID space。一個 /31 prefix 就涵蓋兩個 block:algorithm 0 的 fcbb:bb00 和 algorithm 128 的 fcbb:bb01。這就是 block 相鄰配置的好處。
- 第 20 行:任何帶有 routing header 的封包都不能進來。在這個版本上,ACL 不接受 routing-type 4。它接受通用的 routing 關鍵字。
- 其他封包一律允許。
- 我們把 ACL 套用在不受信任埠的入方向。

Ep 19.6 · After: the door is shut

路由器 / 工具 : CE-A · 來源 : lab-evidence/19-boundary/v3/after-CE-A.log

```
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:6::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:4::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:6:e000::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 fcbb:bb00:6:e004::
2 packets transmitted, 0 packets received, 100% packet loss
CE-A:~$ sudo ping6 -c 2 -W 2 -I 2001:db8:ffff:1::2 2001:db8:100::6
2 packets transmitted, 2 packets received, 0% packet loss
```

- 套用 ACL 之後,我們做同樣的測試。
- 四個 SID 都沒有回應。
- PE6 的 loopback 仍然有回應。邊界只擋 SID space。一般的可達性沒有改變。
- 注意 -I 這個選項。沒有它的話,外部主機用了另一個位址當來源。PE6 沒有回到那個位址的路由,所以連 loopback ping 也失敗了。測試失敗時,在怪罪過濾器之前,先檢查來源位址。

Ep 19.7 · After, on the wire

路由器 / 工具 : tshark · 來源 : lab-evidence/19-boundary/v3/after-PE1-Gi0_0_0_4-CE-A.pcap, after-PE1-P2.pcap

```
$ tshark -r <capture file> -Y "icmpv6.type == 128" -T fields -e ipv6.src -e ipv6.dst | sort | uniq -c
# outside host to PE1, on the port that is not trusted
  2  2001:db8:ffff:1::2    fcb:b00:4::
  2  2001:db8:ffff:1::2    fcb:b00:6::
  1  2001:db8:ffff:1::2    fcb:b00:6:e000::
# PE1 to P2
(no echo request)
```

- ACL 真的把封包丟掉了嗎?我們用擷取來看,指令相同。
- 請求到達了 PE1 那個不受信任的埠。
- 它們沒有一個出現在 PE1 的核心鏈路上。所以 PE1 在邊緣把它們丟掉了。
- 在這個平台上,ACL 的 deny 計數器沒有顯示任何符合次數。要用來自外部的封包和內部的擷取來驗證過濾器。不要把計數器當成唯一的證據。

Ep 19.8 · Planning the SID space for the boundary

路由器 / 工具：PE6 · 來源：lab-evidence/17-oam/v3/L-PE6.log; configs/boundary/PE1-acl.cfg

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
SRv6 Operational Mode:
  Micro-segment:
    SID Base Block: fcbb:bb00::/24

Blocks in use:      fcbb:bb00::/32 (algorithm 0)    fcbb:bb01::/32 (algorithm 128)
Our ACL line:       deny ipv6 any fcbb:bb00::/31
Whole base block:   deny ipv6 any fcbb:bb00::/24
```

- 還有一個設計重點:deny 的 prefix 應該多大?
- manager 顯示了 micro-SID 的基礎 block:fcbb:bb00::/24。這台路由器能使用的所有 block 都在裡面。
- 我們今天用了兩個 block,我們的 ACL 那一行只涵蓋這兩個。
- 如果你封鎖整個基礎 block,也就涵蓋了你以後新增的每一個 block。它可能是另一個演算法,或第 6 模組裡的遷移 block。
- 小的 prefix 很精確。用大的 prefix,網路擴大時你不會漏掉某個 block。選擇其中一種,並記下理由。

Ep 19.9 · Scale: the numbers on PE6

路由器 / 工具 : PE6 · 來源 : lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
  uSID LIB Range:
    LIB Start   : 0xe000
    ELIB Start  : 0xfe00
Summary:
  Number of Locators: 2 (2 operational)
  Number of SIDs: 14 (0 stale)
  Max SID resources: 64000
  Number of free SID resources: 63986
  Block fcb::bb00::/32:
    Number of SIDs free: 7671
    Max SIDs: 7680
  Max Locators: 16
```

- 接著談規模。第 17 集第一層的那個指令顯示了這些數字。
- 本地 SID 範圍從 e000 開始,到 fe00 之前結束。這個系列裡所有的 function 都來自這個範圍:e004、e007、e00b。它們是本地的,每台路由器都重複使用相同的值,如第 16 集所示。
- PE6 在 64000 個資源中使用了 14 個 SID。
- 每個 block 有 7680 個 SID。這就是從 e000 到 fdff 這個範圍的大小。所以每個 locator 真正的上限是本地範圍,而不是總數。
- locator 最多 16 個。

Ep 20.1 · The legacy configuration on PE1

來源 : lab-evidence/20-ldp-migration/PE1-commit.log

```
interface GigabitEthernet0/0/0/0
  ipv4 address 10.12.0.0 255.255.255.254
router isis CORE
  address-family ipv4 unicast
    metric-style wide
  interface GigabitEthernet0/0/0/0
    address-family ipv4 unicast
      metric 10
router bgp 65000
  neighbor 10.0.0.6
    remote-as 65000
  update-source Loopback0
  address-family vpnv4 unicast
mpls ldp
  router-id 10.0.0.1
  interface GigabitEthernet0/0/0/0
```

- 這是 PE1 上的 MPLS 設定,省略了部分內容。其他路由器也套用同樣的模式。
- 核心鏈路上有 IPv4 位址。
- IPv4 在同一個 IS-IS instance 裡,metric 和 IPv6 相同。
- 有一個通往 PE6 IPv4 loopback 的 VPNv4 會話。
- 核心介面上啟用了 LDP。GREEN 在任何路由器上都沒有 SRv6 分配。

Ep 20.2 · GREEN on MPLS

來源：lab-evidence/20-ldp-migration/PE1-mpls.log

```
RP/0/RP0/CPU0:PE1#show mpls ldp neighbor brief
10.0.0.2:0      N   N   00:01:26   1   0   4   0   14   0
10.0.0.3:0      N   N   00:01:24   1   0   4   0   14   0

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
 10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6), if-handle 0x00000000
   Received Label 24000

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
 next hop 10.0.0.6/32 via 24011/0/21
 next hop 10.12.0.1/32 GigabitEthernet0/0/0/0 labels imposed {24009 24000}
```

- 這是 MPLS 上的 GREEN,就像傳統網路會顯示的那樣。
- 有通往 P2 和 P3 的 LDP 會話。
- 來自 PE6 的 GREEN 路由,下一跳是 10.0.0.6,帶著一個真正的 MPLS label 24000。
- CEF 推入兩個 label。24009 是通往 PE6 的 LDP label,24000 是 VPN label。和第 13 集比較一下:一個 IPv6 標頭加一個 SID。

Ep 20.3 · Both worlds on one wire

來源：lab-evidence/20-ldp-migration/PE1-mpls.log, mpls-PE1-P2.pcap, mpls-P2-P4.pcap, mpls-P4-PE6.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.100.6.6 source 10.100.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

$ tshark -r <capture file> -Y "mpls or ipv6.nxt == 4" -T fields -e frame.len -e mpls.label -e ipv6.dst -e icmp.type
# PE1 to P2      122    24009,24000    GREEN request, MPLS
#                154    fcbb:bb00:6:e004:: BLUE request, SRv6
# P2 to P4      122    24009,24000    GREEN request
#                154    fcbb:bb00:6:e004:: BLUE request
# P4 to PE6     118    24000         GREEN request, top label popped
#                154    fcbb:bb00:6:e004:: BLUE request
```

- PE1 用 100 位元組的封包 ping GREEN 和 BLUE 時,我們擷取了三條核心鏈路。我們用 tshark 讀取每個檔案。
- 兩個 ping 都正常。離開 PE1 時,GREEN 帶著兩個 label,122 位元組。BLUE 帶著一個 IPv6 標頭,154 位元組。
- P2 到 P4 的鏈路上也一樣。
- 在最後一條鏈路上,GREEN 少了頂層 label。P4 把它移除了:這就是倒數第二跳彈出。BLUE 從頭到尾都沒有改變。兩種技術使用每一台路由器和每一條鏈路,而且互不影響。
- 注意大小。SRv6 封包比兩個 label 的 MPLS 封包大了 32 位元組。GREEN 遷移時,會失去 32 位元組的 MTU。遷移之前,請用第 7 集的方法檢查這一點。

Ep 20.4 · Reading the labels hop by hop

路由器 / 工具 : tshark · 來源 : lab-evidence/20-ldp-migration/mpls-PE1-P2.pcap, mpls-P2-P4.pcap, mpls-P4-PE6.pcap

```
$ tshark -r <capture file> -Y "mpls" -T fields -e frame.len -e mpls.label -e icmp.type
# GREEN request, from PE1 to PE6
PE1 to P2      122    24009,24000
P2 to P4       122    24009,24000
P4 to PE6      118    24000          (the top label was popped by P4)
# GREEN reply, from PE6 to PE1
P4 to PE6      122    24001,24000
P2 to P4       122    24000,24000
PE1 to P2      118    24000          (the top label was already popped)
```

- 我們再看一次 MPLS,因為它顯示了 SRv6 拿掉了什麼。
- PE1 送出的頂層是 24009。這是 P2 為 PE6 分配的 label。底下是 24000,PE6 的 VPN label。
- P2 把頂層 label 換成 P4 為 PE6 分配的 label。這個 label 也是 24009。每台路由器都有自己的 label。數字相同只是巧合。
- P4 在 PE6 之前移除頂層 label。
- 回應以相反方向呈現同樣的模式,PE6 從 24001 開始。每台路由器都要為每台 PE 維護一張 label 表。而在 SRv6 裡,轉送路由器轉送的是一個不變的 IPv6 目的位址。它們用的是一般的路由表。

Ep 20.5 · Step 1: PE6 moves first

來源 : lab-evidence/20-ldp-migration/step1-PE6-commit.log, step1-PE1.log

```
router bgp 65000
vrf GREEN
  address-family ipv4 unicast
    segment-routing srv6
      locator MAIN
      alloc mode per-vrf

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
  10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6), if-handle 0x00000000
    Received Label 0xe0080
      SID Locator: fcbb:bb00:6::/48, RIB metric: 31
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
  SRv6 H.Encaps.Red SID-list {fcbb:bb00:6:e008::}
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/3/6 ms
```

- 第一步。只有 PE6 改變。
- PE6 為 GREEN 分配一個 SRv6 SID。這和第 12 集 BLUE 的那五行相同。
- 在 PE1 上, GREEN 的路由現在帶著 SID 到達。看看下一跳: 仍然是 10.0.0.6, 也就是 IPv4 會話。PE6 在兩個會話上都通告了 SID, 而 PE1 選了來自 IPv4 會話的路徑。
- 這不重要。決定封裝方式的是 SID。不是會話。
- CEF 現在用 PE6 的 GREEN SID 來封裝。
- GREEN 正常運作。

Ep 20.6 · Step 1: one route, two paths

來源：lab-evidence/20-ldp-migration/step1-PE1.log

```
RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast rd 65000:306 10.30.6.6/32
Paths: (2 available, best #1)
  Path #1: Received by speaker 0
    10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6)
      Received Label 0xe0080
      Origin incomplete, metric 0, localpref 100, valid, internal, best, group-best
  Path #2: Received by speaker 0
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6)
      Received Label 0xe0080
      Origin incomplete, metric 0, localpref 100, valid, internal
```

- 為什麼 PE1 用的是 IPv4 會話的路徑?這是 PE1 上來自 PE6 的路由,在匯入 GREEN 之前的樣子。
- 同一條路由有兩條路徑,每個會話各一條。
- 路徑 1 透過 IPv4,帶著 SID。它是最佳路徑。
- 路徑 2 透過 IPv6,帶著同一個 SID。輸出裡的每一個屬性都相同。所以由 BGP 最後的平手規則來選擇。
- 兩條路徑都通往同一個 SID,產生同樣的封包。遷移期間有兩個會話是正常的。要知道哪一條是最佳路徑,以及原因。

Ep 20.7 · Step 1 on the wire: half and half

來源：lab-evidence/20-ldp-migration/step1-PE1.log, step1-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/3/6 ms

$ tshark -r step1-PE1-P2.pcap -Y "mpls or ipv6.nxt == 4" \
  -T fields -e frame.len -e mpls.label -e ipv6.dst -e ip.src | sort | uniq -c
  3   154                fcbb:bb00:6:e008:: 10.30.1.1    PE1 to PE6, requests: SRv6
  3   118   24000                10.30.6.6    PE6 to PE1, replies: MPLS
```

- 線路顯示了第一步的結果,在 PE1 到 P2 的鏈路上。ping 是通的。
- 從 PE1 到 PE6 的請求是 SRv6,因為它們的出口 PE6 已經遷移了。
- 從 PE6 回到 PE1 的回應仍然是 MPLS,因為它們的出口 PE1 還沒有遷移。一個客戶、一個 ping 使用兩種技術,而這個 ping 能運作。一端可以先遷移。

Ep 20.8 · Step 2: PE1 follows

來源 : lab-evidence/20-ldp-migration/step2-PE1.log, step2-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 1/3/6 ms
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.100.6.6 source 10.100.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/3 ms

$ tshark -r step2-PE1-P2.pcap -Y "ipv6.nxt == 4" -T fields -e frame.len -e ipv6.dst -e ip.src | sort | uniq -c
  3   154   fcb:b00:6:e008::  10.30.1.1   GREEN request
  3   154   fcb:b00:1:e00a::  10.30.6.6   GREEN reply
  2   154   fcb:b00:6:e004::  10.100.1.1  BLUE request
  2   154   fcb:b00:1:e004::  10.100.6.6  BLUE reply
```

- 第二步:我們在 PE1 上設定同樣的五行。然後做同樣的兩個 ping。
- GREEN 的請求送往 PE6 的 GREEN SID,和之前一樣。
- 現在回應送往 PE1 新的 GREEN SID,e00a。兩個方向都是 SRv6 了。
- BLUE 沒有受到影響。這一步之後,GREEN 和 BLUE 的 ping 都通過。

Ep 20.9 · Rollback of step 1

來源：lab-evidence/20-ldp-migration/rollback-PE6-commit.log, rollback-PE6-PE1.log

```
router bgp 65000
vrf GREEN
address-family ipv4 unicast
no segment-routing srv6

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
10.0.0.6 (metric 30) from 10.0.0.6 (10.0.0.6), if-handle 0x00000000
Received Label 24000
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
next hop 10.12.0.1/32 GigabitEthernet0/0/0/0 labels imposed {24009 24000}
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/4/6 ms
```

- 接下來是每份變更計畫都需要的測試:能不能退回去?PE1 已經遷移了,我們在 PE6 上回退第一步。
- PE6 上的一行指令移除了 GREEN 的 SRv6 分配。
- PE1 再次收到 label 24000。
- CEF 又回到兩個 label。LDP 仍在運作,所以傳輸 label 早已就緒。
- GREEN 通過。在線路上,請求又變回 MPLS。回應仍然是 SRv6,因為我們沒有回退 PE1。兩個方向又各用一種技術,只是方向反過來。
- 我們重新套用第一步,兩個方向又都恢復成 SRv6。

Ep 20.10 · Rollback on the wire

來源：lab-evidence/20-ldp-migration/rollback-PE6-PE1.log, rollback-PE6-PE1-P2.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/4/6 ms

$ tshark -r rollback-PE6-PE1-P2.pcap -Y "mpls or ipv6.nxt == 4" \
  -T fields -e frame.len -e mpls.label -e ipv6.dst -e ip.src | sort | uniq -c
  3   122   24009,24000                               10.30.1.1   GREEN request: MPLS again
  3   154                               fcbb:bb00:1:e00a:: 10.30.6.6   GREEN reply: still SRv6
```

- 這是回退期間的擷取。ping 是通的。
- 送往 PE6 的請求又變回 MPLS,帶著兩個 label,因為 PE6 撤回了它的 SID。
- 回到 PE1 的回應仍然是 SRv6,送往 PE1 的 GREEN SID。這個規則反過來也成立:每個方向都跟著它的出口,不管出口是往前遷移還是退回去。

Ep 20.11 · Step 3: remove the legacy

來源：lab-evidence/20-ldp-migration/step3-*--commit.log, step3-PE1.log, step3-PE1-mpls.log

```
router bgp 65000
  no neighbor 10.0.0.6
no mpls ldp                                (on all six routers)

RP/0/RP0/CPU0:PE1#show bgp vpnv4 unicast vrf GREEN 10.30.6.6/32 detail
    2001:db8:100::6 (metric 30) from 2001:db8:100::6 (10.0.0.6), if-handle 0x00000000
    Received Label 0xe0080
RP/0/RP0/CPU0:PE1#show mpls forwarding
RP/0/RP0/CPU0:PE1#show mpls ldp neighbor brief
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
```

- 第三步:按順序移除 MPLS 設定。
- 首先,移除兩台 PE 上的 IPv4 BGP 會話。
- 接著移除六台路由器上的 LDP。
- GREEN 的路由現在來自 PE6 的 IPv6 loopback,帶著它的 SID。
- MPLS 轉送表是空的,也沒有任何 LDP 鄰居。
- GREEN 和 BLUE 都通過。在線路上,GREEN 的兩個方向都是 SRv6。遷移完成。

Ep 21.1 · Step 1: SR-MPLS configuration

來源：lab-evidence/21-srmpls-migration/P2-commit.log, start-PE1-commit.log

```
On every router, here P2:
router isis CORE
 address-family ipv4 unicast
  segment-routing mpls
interface Loopback0
 address-family ipv4 unicast
  prefix-sid index 2
```

```
On PE1, mirrored on PE6:
router bgp 65000
 neighbor 10.0.0.6
  remote-as 65000
 update-source Loopback0
 address-family vpnv4 unicast
vrf GREEN
 address-family ipv4 unicast
  no segment-routing srv6
```

- 第 20 集的 IPv4 核心仍然存在。這次沒有 LDP。
- 我們在 IS-IS 的 IPv4 位址族啟用 MPLS segment routing。
- 每個 loopback 都有 prefix SID index。P2 使用 2。PE6 使用 6。
- PE 使用 IPv4 BGP 會話傳送帶 label 的 VPNv4 路由。
- GREEN 沒有 SRv6 分配。所以它使用 MPLS label。

Ep 21.2 · Step 1: the result

來源：lab-evidence/21-srmpls-migration/srmpls-start-PE1.log

```
RP/0/RP0/CPU0:PE1#show isis segment-routing label table
Label          Prefix          Algorithm    Interface
16006          10.0.0.6/32      SPF

```

```
RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32
Local  Outgoing  Prefix          Outgoing  Next Hop        Bytes
16006  16006      SR Pfx (idx 6)   Gi0/0/0/0  10.12.0.1       872

```

```
RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
  next hop 10.0.0.6/32 via 16006/0/21
  next hop 10.12.0.1/32 GigabitEthernet0/0/0/0 labels imposed {16006 24000}

```

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/2 ms

```

- 這是 PE1 上的結果。
- PE6 的 loopback 使用 label 16006。預設 label block 從 16000 開始。再加上 PE6 的 index,也就是 6。
- PE1 把 label 16006 換成 label 16006。數值相同,因為這些路由器使用相同的 SRGB。通告的 index 在整個 domain 有一致的意義。
- GREEN 封包帶兩個 label。label 16006 用來到達 PE6。label 24000 識別 PE6 上的 VPN 服務。
- ping 收到回應。

Ep 21.3 · Step 1: on the wire

來源：lab-evidence/21-srmpls-migration/srmpls-start-PE1.log, srmpls-start-PE1-P2.pcap, srmpls-start-P2-P4.pcap

```
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/2 ms

$ tshark -r <capture file> -Y "mpls" -T fields -e frame.len -e mpls.label -e ip.src
# PE1 to P2
122    16006,24000    10.30.1.1    GREEN request
118    24000          10.30.6.6    GREEN reply
# P2 to P4
122    16006,24000    10.30.1.1    GREEN request
122    16001,24000    10.30.6.6    GREEN reply
```

- 我們檢查兩條核心鏈路,用 PE1 的 GREEN ping,並用 tshark 讀取每個檔案。
- 在 PE1,label 16006 位於 label 24000 上面。回應只帶 label 24000,因為 PE1 之前的路由器已經移除了頂層 label。
- 在下一條鏈路上,頂層 label 仍然是 16006。這些路由器使用相同的 SRGB,並把 index 6 對應到 PE6。在第 20 集,LDP 是各自獨立分配 label 的。回應帶著 PE1 的 label 16001。

Ep 21.4 · Step 2: move GREEN to SRv6

來源：lab-evidence/21-srmpls-migration/migrate-PE6-commit.log, migrate-PE1-commit.log, srmpls-migrated-PE1.log

```
On PE6 and PE1:
router bgp 65000
vrf GREEN
  address-family ipv4 unicast
    segment-routing srv6
      locator MAIN
      alloc mode per-vrf

RP/0/RP0/CPU0:PE1#show cef vrf GREEN 10.30.6.6/32 detail
10.30.6.6/32, version 19, SRv6 Headend
  SRv6 H.Encaps.Red SID-list {fcb:b00:6:e008::}
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 3
Success rate is 100 percent (3/3), round-trip min/avg/max = 2/2/3 ms
```

- 遷移使用和第 20 集相同的五行設定。
- 我們在每台 PE 為 GREEN 設定 SRv6 分配。第 20 集示範過逐台 PE 遷移。這裡也適用相同規則:出口決定封裝方式。
- CEF 現在使用 PE6 的 SID 封裝 GREEN。
- GREEN 收到 ping 回應。

Ep 21.5 · Step 2: on the wire

路由器 / 工具 : tshark · 來源 : lab-evidence/21-srmppls-migration/srmppls-migrated-PE1-P2.pcap, srmppls-migrated-P2-P4.pcap

```
$ tshark -r <capture file> -Y "ipv6.nxt == 4" -T fields -e frame.len -e ipv6.dst -e ip.src
# PE1 to P2
154    fcb:b00:6:e008::    10.30.1.1    GREEN request
154    fcb:b00:1:e00a::    10.30.6.6    GREEN reply
# P2 to P4
154    fcb:b00:6:e008::    10.30.1.1    GREEN request
154    fcb:b00:1:e00a::    10.30.6.6    GREEN reply
```

- 這是遷移後的同樣兩條鏈路,用同樣的指令讀取。
- 兩個方向都使用 SRv6。目的位址是出口 PE 的 GREEN SID。這些客戶封包沒有 MPLS label。
- P2 轉送封包,不改變目的位址,和第 13 集的 BLUE 一樣。

Ep 21.6 · Step 3: who still uses label 16006?

來源：lab-evidence/21-srmpls-migration/srmpls-start-PE1.log, srmpls-migrated-PE1.log

```
Before the move:
RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32
Local   Outgoing   Prefix           Outgoing   Next Hop        Bytes
16006   16006       SR Pfx (idx 6)   Gi0/0/0/0   10.12.0.1       872

After the move, GREEN on SRv6:
RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32
Local   Outgoing   Prefix           Outgoing   Next Hop        Bytes
16006   16006       SR Pfx (idx 6)   Gi0/0/0/0   10.12.0.1       1415
```

- GREEN 現在使用 SRv6。SR-MPLS 還有承載流量嗎？
- 遷移之前,PE6 的 label 項目顯示轉送了 872 位元組。
- 遷移之後,計數器顯示 1415 位元組。仍有流量使用 label 16006。我們之後測試了原因。沒有客戶流量的六十秒內,計數器增加了 99 位元組。
- 擷取顯示兩個帶 label 的封包,屬於通往 10.0.0.6 的 IPv4 BGP 會話。BGP keepalive 是 59 位元組。TCP 確認封包是 40 位元組。合計就是增加的 99 位元組。
- 我們移除這個會話後,計數器不再增加。舊的傳輸仍然承載控制流量。移除舊的傳輸之前,要找出所有使用它的流量。

Ep 21.7 · Step 3: remove the IPv4 VPN session

路由器 / 工具 : PE1 · 來源 : lab-evidence/label-16006/S2-PE1-nobgp.log, S2-before.log, S2-after.log

```
! On PE1. PE6 does the same with neighbor 10.0.0.1
router bgp 65000
  no neighbor 10.0.0.6

RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32      (00:28:51)
16006  16006          SR Pfx (idx 6)      Gi0/0/0/0      10.12.0.1      1251

RP/0/RP0/CPU0:PE1#show mpls forwarding prefix 10.0.0.6/32      (00:29:53)
16006  16006          SR Pfx (idx 6)      Gi0/0/0/0      10.12.0.1      1251

RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/2/2 ms
```

- 第 3 步移除 label 16006 最後的使用者:IPv4 VPN 會話。
- 在 PE1 上,我們移除 neighbor 10.0.0.6。PE6 移除 neighbor 10.0.0.1。
- 接著我們讀兩次 label 16006 的計數器,間隔大約一分鐘。兩次都是 1251。計數器停止增加了。
- GREEN 仍然正常,走的是 SRv6。

Ep 21.8 · Step 4: remove SR-MPLS

路由器 / 工具 : PE1 · 來源 : lab-evidence/label-16006/PE1-cleanup.log, after-PE1.log

```
! On every router, here PE1
RP/0/RP0/CPU0:PE1(config)#no router isis CORE address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#no router isis CORE interface Loopback0 address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#no router isis CORE interface GigabitEthernet0/0/0/0 address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#no router isis CORE interface GigabitEthernet0/0/0/1 address-family ipv4 unicast
RP/0/RP0/CPU0:PE1(config)#commit

RP/0/RP0/CPU0:PE1#show running-config router isis CORE
(only SRv6 lines remain: segment-routing srv6, microloop avoidance)
```

- 第 4 步移除 SR-MPLS。要最後才做,而且要在計數器停止之後。
- 在每台路由器上,我們從 IS-IS 移除 IPv4 位址族,包括 instance 和每個介面。這也同時移除了 prefix SID 和 MPLS 的 segment routing。
- 然後 commit。
- IS-IS 的 running configuration 現在只剩 SRv6 的設定。

Ep 22.1 · The legacy IGP, added to every router

路由器 / 工具 : P3 · 來源 : lab-evidence/22-ospf-migration/P3-commit.log

```
router ospf LEGACY
router-id 10.0.0.3
area 0
interface Loopback0
passive enable
interface GigabitEthernet0/0/0/0
cost 40
network point-to-point
interface GigabitEthernet0/0/0/2
cost 10
network point-to-point
router ospfv3 LEGACY
router-id 10.0.0.3
area 0
interface GigabitEthernet0/0/0/0
cost 40
```

- 六台路由器都使用 IPv4 的 OSPF 和 IPv6 的 OSPFv3。這是 P3 的設定。
- 我們使用 area 0, 並把 loopback 設為 passive。
- 鏈路成本對應 IS-IS metric。下面的鏈路使用 40。其他鏈路使用 10。我們確認兩個協定選擇相同路徑。這樣遷移會改變路由歸屬, 而不改變這些路徑。
- OSPFv3 使用相同的鏈路成本。IS-IS 設定維持不變。

Ep 22.2 · Two IGPs: who owns what

來源 : lab-evidence/22-ospf-migration/both-igps-PE1.log

```
RP/0/RP0/CPU0:PE1#show ospf neighbor
10.0.0.2          1      FULL/  -      00:00:37    10.12.0.1      GigabitEthernet0/0/0/0
10.0.0.3          1      FULL/  -      00:00:37    10.13.0.1      GigabitEthernet0/0/0/1

RP/0/RP0/CPU0:PE1#show route ipv6 2001:db8:100::6/128
  Known via "ospf LEGACY", distance 110, metric 30, type intra area
    fe80::5454:ff:fe66:75e1, from ::, via GigabitEthernet0/0/0/0

RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 115, metric 31, SRv6-locator, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
    fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)

RP/0/RP0/CPU0:PE1#show ospfv3 database | include fcbb
```

- 兩個 IGP 運作時,每條路由由哪個協定提供?
- OSPF 鄰接顯示 Full。IS-IS 鄰接也維持 up。
- OSPF 提供選中的 PE6 loopback 路由。兩個協定都知道這個 prefix。OSPF 的 distance 是 110,比 IS-IS 的 115 低。選中的路由只有一條路徑,沒有備援。
- IS-IS 仍然提供 PE6 的 locator 路由,帶著 TI-LFA 備援。在這個設定中,OSPF 不通告 locator。
- OSPFv3 資料庫沒有 SID block 的 prefix。BLUE 的 ping 收到回應。

Ep 22.3 · Step 1: prefer IS-IS everywhere

來源：lab-evidence/22-ospf-migration/step1-PE1-commit.log, step1-isis-preferred-PE1.log

```
On every router:
router isis CORE
 address-family ipv4 unicast
   distance 100
 address-family ipv6 unicast
   distance 100

RP/0/RP0/CPU0:PE1#show route ipv6 2001:db8:100::6/128
  Known via "isis CORE", distance 100, metric 30, type level-2
    fe80::5454:ff:fe66:75e1, from 2001:db8:100::6, via GigabitEthernet0/0/0/0, Protected
    fe80::5454:ff:fecl:58eb, from 2001:db8:100::6, via GigabitEthernet0/0/0/1, Backup (TI-LFA)
RP/0/RP0/CPU0:PE1#show route ipv4 10.0.0.6/32
  Known via "isis CORE", distance 100, metric 30, labeled SR
```

- 第一步讓 IS-IS 優先。兩個協定都保留設定。
- 我們在六台路由器的兩個位址族把 IS-IS distance 設為 100。它比 OSPF 的 110 低。OSPF 鄰接維持 Full,所以它的路由仍然可用。
- IS-IS 現在提供選中的 PE6 loopback 路由。這條路由有 TI-LFA 保護。
- IPv4 loopback 也使用 IS-IS。它保留第 21 集的 SR-MPLS label。服務測試通過。改變優先順序就是遷移步驟。移除 distance 設定就能回退。
- IPv4 metric 是 30。在 OSPFv2 下是 31,因為 OSPFv2 加上 loopback 介面的成本 1。鏈路成本相同,總成本卻不同。依賴混合網路之前,要比較總成本和選中的路徑。

Ep 22.4 · Step 2: remove OSPF

來源：lab-evidence/22-ospf-migration/step2-PE1-commit.log, step2-ospf-removed-PE1.log

```
On every router:
no router ospf LEGACY
no router ospfv3 LEGACY

RP/0/RP0/CPU0:PE1#show ospf neighbor
RP/0/RP0/CPU0:PE1#show route ipv6 fcbb:bb00:6::/48
  Known via "isis CORE", distance 100, metric 31, SRv6-locator, type level-2
RP/0/RP0/CPU0:PE1#ping vrf BLUE 10.100.6.6 source 10.100.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 2/3/4 ms
RP/0/RP0/CPU0:PE1#ping vrf GREEN 10.30.6.6 source 10.30.1.1 count 2
Success rate is 100 percent (2/2), round-trip min/avg/max = 3/3/4 ms
```

- 第二步移除 OSPF。
- 我們在每台路由器輸入兩行指令。
- OSPF 鄰居全部消失。選中的路由表沒有改變,因為這些路由已經由 IS-IS 提供。
- BLUE 和 GREEN 收到回應。第一步改變路由優先順序。第二步移除未使用的協定。最後的移除不應改變選中的路徑。

Ep 23.1 · The same command, on both

路由器 / 工具 : PE6 · 來源 : lab-evidence/17-oam/v3/L-PE6.log

```
RP/0/RP0/CPU0:PE6#show segment-routing srv6 manager
  Signaled parameters:
    Max-SL           : 3
    Max-End-Pop-SRH  : 3
    Max-H-Insert     : 3 sids
    Max-H-Encap      : 3 sids
    Max-End-D        : 4
  Configurable parameters (under srv6):
    Encapsulation:
      Hop-Limit      : value=Yes, propagate=No
      Traffic-class   : value=Yes, propagate=Yes
  Max Locators: 16
  Max SIDs: 64000
```

- 在兩種平台執行相同的 manager 指令。這是第 17 集的 XRd 輸出。
- 輸出列出 segments-left、插入、封裝和解封裝的限制。IS-IS 通告這些能力。headend 或 PCE 建立路徑時可以使用這些資訊。
- 輸出也描述標頭欄位傳遞。這裡沒有啟用 hop limit 傳遞。traffic class 傳遞則可用。第 13 集量測了相應的 DSCP 行為。
- 下面列出資源限制。完成設計之前,先比較這份輸出和硬體輸出。每份輸出都要記錄平台和版本。

fcbb:bb00

0006

e004

0000

0000

0000

0000

帶走三句話

程式已經到達最後一個節點。

- DA 是一段程式：找出 active instruction，以及誰擁有它。
- Reachable、installed、delivered 是三件不同的事。
- 先預測，再觀察。